UMBRA



MANUAL FUNCIONAL Y TÉCNICO

La aplicación, explicada.

Todas sus funciones, el papel de cada componente y el código fuente completo para consultar y estudiar.

Edición del 08/11/2026 · Frontend local con React y Vite
50 archivos de texto reproducidos · 2892 líneas de código y documentación
Incluye accesibilidad independiente por perfil y guía para la futura API REST.

MAPA DEL DOCUMENTO

Contenido

Los títulos y enlaces permiten saltar entre explicación y código. El PDF conserva texto seleccionable; el ZIP contiene los archivos originales para editarlos.

1. Qué incluye este documento

2. Todas las funciones de la app

3. Pantallas y rutas

4. Cómo se conecta el código

5. Modelo de datos y almacenamiento

6. Recorridos explicados paso a paso

7. Accesibilidad por perfil

8. Ejecución y pruebas

9. Futura integración con Spring Boot

10. Índice de archivos y código

> Las explicaciones se basan en los archivos presentes al generar este documento. Cada listado tiene su ruta y números de línea; los cambios posteriores de la app requieren regenerar el manual.

1 · PUNTO DE PARTIDA

Qué incluye este documento

UMBRA es una aplicación para crear, organizar y conservar personajes de fantasía. Esta versión ejecuta su lógica y guarda sus datos en el navegador.

Este manual describe el comportamiento disponible, la arquitectura React, las entradas y responsabilidades de cada componente y las funciones de apoyo. Después de cada explicación aparece el código íntegro del archivo correspondiente, incluido el CSS que hace visibles los ajustes.

- Frontend: React, React DOM, React Router, Vite y JavaScript con módulos.
- Datos: bibliotecas, cuentas locales, sesiones y ajustes de accesibilidad.
- Pruebas: archivos unitarios y recorridos automatizados de navegador.
- Recursos: imágenes, icono y fuentes. Los binarios viajan en el ZIP; sus licencias se conservan.

Estado real de la versión

No está conectada todavía a un backend Spring Boot ni a una base de datos del servidor. No hay roles, cuenta administradora precreada, envío de correo ni sincronización entre dispositivos. Una cuenta con nombre Administrador tiene los mismos controles que cualquier otra cuenta local.

Los accesos locales organizan bibliotecas en un navegador; no son una autorización de servidor. Web Crypto requiere un contexto seguro, como localhost o HTTPS.

Cómo utilizar el paquete

El archivo `UMBRA-manual-completo.pdf` sirve para leer, buscar e imprimir. El HTML es una versión independiente para el navegador. Dentro del ZIP, la carpeta `codigo/` conserva los archivos editables y los recursos. Tras extraerla se pueden instalar las dependencias y ejecutar Vite.

> El código de dependencias, node_modules, dist y el historial .git no se imprimen. package-lock.json sí se conserva en el ZIP, pero no se reproduce en el PDF por ser un archivo generado de resolución de paquetes.

2 · INVENTARIO FUNCIONAL

Todas las funciones de la app

Función	Comportamiento y ubicación
Inicio	Portada, enlace a crear y dos personajes recientes; estado vacío y acceso a la cuenta. src/pages/Home.jsx
Biblioteca	Busca por nombre, clase, raza u origen; ignora tildes y mayúsculas; filtra clase y favoritos; ordena por fecha, nombre o nivel. src/pages/Library.jsx
Consultar ficha	Retrato, historia, raza, clase, nivel, origen, rasgos y cuatro atributos en un diálogo. src/App.jsx
Crear y editar	Valida los campos, personaliza la ficha y guarda en la biblioteca del perfil activo. src/pages/CreateCharacter.jsx
Retratos	Dos recursos locales, iniciales o archivo PNG/JPEG/WebP de hasta 750 KB, con vista previa. src/components/charactercard/Portrait.jsx
Aleatorio	Genera un borrador a partir de una ficha base, nombre y atributos; todavía requiere guardar. src/data/characters.js
Favoritos	Marca y desmarca fichas; el filtro de Biblioteca muestra solo las marcadas. src/hooks/useCharacters.js
Duplicado	Crea otro id, añade (copia) al nombre y comienza sin marcar favorito. src/App.jsx
Eliminación	Solicita confirmación y guarda una biblioteca sin la ficha eliminada. src/App.jsx
Inspiración	Ideas narrativas y semillas aleatorias; pasa el borrador al editor por estado de navegación. src/pages/Information.jsx
Registro	Crea una cuenta local y entrega su código de recuperación; no inicia automáticamente la sesión. src/pages/Auth.jsx

Función	Comportamiento y ubicación
Acceso	Comprueba credenciales locales, permite ver la contraseña y elegir Recordarme. src/pages/Login.jsx
Recuperación	Requiere correo y código guardado para sustituir la contraseña. src/utils/localAccounts.js
Perfil	Identidad o Invitado, recuento de fichas y favoritos, cierre de sesión y accesos a ajustes. src/pages/Profile.jsx
Copias JSON	Descarga biblioteca y retratos; valida una copia y la restaura solo tras confirmación. src/utils/characterStore.js
Texto	Pequeño, normal, grande y extra grande mediante radios y deslizador. src/pages/Accessibility.jsx
Contraste	Normal, alto o invertido; tiene prioridad sobre los colores de tema y acento. src/pages/Accessibility.jsx
Lectura	Anuncios de cambio de pantalla, lectura voluntaria de fichas y velocidades 0.75, 1 y 1.25. src/components/feedback/ReadAloud.jsx
Teclado	Tab, Mayús+Tab, opciones nativas, Escape, foco reforzado y flechas/Inicio/Fin en barra inferior. src/components/bottomnavigation/BottomNavigation.jsx
Apariencia	Oscuro, claro o sepia; controles normales, grandes o extra grandes; seis acentos; movimiento reducido. src/pages/Accessibility.jsx
Perfiles independientes	Cada cuenta e Invitado conservan biblioteca y accesibilidad diferentes. src/utils/preferenceStore.js
Menús y diálogos	Cabecera desplegable, barra inferior activa y diálogos que devuelven el foco. src/components/modal/Modal.jsx
Ayuda y privacidad	Preguntas desplegables y explicación del tratamiento local de datos. src/pages/Information.jsx
Conexión y 404	Avisa de eventos de red y presenta una pantalla para rutas inexistentes. src/components/feedback/ConnectionStatus.jsx

3 · NAVEGACIÓN

Pantallas y rutas

Las rutas están definidas en App.jsx. La cabecera y la barra inferior toman sus enlaces de navigation.js; el título de cada pantalla también procede de ese catálogo.

Ruta	Contenido
`/`	Redirección a /inicio
`/inicio`	Home
`/biblioteca`	Library
`/crear-personaje`	CreateCharacter + CharacterEditor
`/personaje/:id/editar`	El mismo editor con una ficha existente
`/inspiracion`	Inspiration
`/perfil`	Profile
`/login`	Login; redirige a Perfil si hay sesión
`/registro`	Auth; redirige a Perfil si hay sesión
`/recuperar`	Auth con recovery; redirige si hay sesión
`/accesibilidad`	Menú de cinco secciones
`/accesibilidad/texto`	Tamaño de texto
`/accesibilidad/contraste`	Modo de contraste
`/accesibilidad/lectura`	Lectores de pantalla y voz
`/accesibilidad/teclado`	Atajos y foco
`/accesibilidad/apariencia`	Tema, controles, acentos y movimiento
`/ayuda`	Help
`/privacidad`	Privacy

Ruta	Contenido
`Cualquier otra ruta`	NotFound

Al cambiar de ruta se actualiza el título, se enfoca el contenido principal y se vuelve al comienzo. Si el alojamiento publica BrowserRouter, debe devolver index.html para que las rutas internas carguen directamente.

4 · ORGANIZACIÓN DEL CÓDIGO

Cómo se conecta el código

index.html → main.jsx
El navegador recibe #root y React monta la aplicación dentro de BrowserRouter.

↓

App → Workspace
Sesión, propietario actual, rutas, biblioteca, ajustes y diálogos.

↓

Páginas + componentes
Las páginas organizan una tarea; tarjetas, formularios, cabecera y diálogos se reutilizan.

↓

Hooks
useCharacters y usePreferences conservan estado React y coordinan su persistencia.

↓

Utilidades → almacenamiento
Validación, cuentas, JSON y localStorage/sessionStorage.

Props, estado y eventos

Una prop es un dato o una función que el padre entrega al hijo. Por ejemplo, CharacterCard recibe character y onSelect. Su botón llama onSelect(character); Workspace cambia modal y React muestra la ficha.

useState conserva valores que deben volver a dibujar la interfaz. useRef guarda referencias al DOM o a una locución sin forzar otro render. useEffect conecta y limpia efectos externos, como listeners de conexión, foco y diálogos.

La clave que separa perfiles

App entrega a Workspace una key igual al id de la cuenta o guest. Al cambiar de perfil, React desmonta el espacio anterior y monta otro. Así los hooks cargan la biblioteca y los ajustes del nuevo propietario, en lugar de reutilizar el estado anterior.

5 · DATOS PERSISTENTES

Modelo y almacenamiento

Una ficha es un objeto JavaScript. La serialización JSON permite guardarla como texto y recuperarla al abrir de nuevo la app.

Propiedad	Tipo y significado
`id`	string Identificador estable; nuevo UUID cuando se crea una ficha.
`name`	string Nombre entre 2 y 60 caracteres tras recortar extremos.
`type / race`	string Valores de characterTypes y characterRaces.
`level`	number Entero entre 1 y 20.
`realm / story`	string Origen hasta 100 caracteres e historia hasta 3000.
`portrait`	number Índice del retrato base; sin recurso disponible se muestran iniciales.
`imageDataUrl / imageName`	string Imagen local serializada y su nombre; se guardan dentro de la ficha.
`traits`	string[] Hasta seis rasgos; la validación admite etiquetas de hasta 40 caracteres.
`stats`	number[4] Fuerza, Destreza, Inteligencia y Carisma; enteros de 1 a 20.
`favorite`	boolean Estado que utiliza el filtro de favoritos.
`updatedAt`	number Fecha en milisegundos; sirve para ordenar fichas recientes.

Claves del navegador

Clave	Almacén y contenido
`umbra.characters.v1.<owner>`	localStorage Biblioteca del propietario. owner es el id de la cuenta o guest.
`umbra.accessibility.v2.<owner>`	localStorage Ajustes de accesibilidad y apariencia del propietario.
`umbra.accounts.v1`	localStorage Array de cuentas con identificadores, nombres, correos y verificadores.
`umbra.session.v1`	sessionStorage o localStorage Id de la cuenta activa. Recordarme decide dónde se conserva.
`umbra.accessibility.v1`	localStorage; legado Solo se considera para recuperar los antiguos ajustes de Invitado.

Las claves pertenecen al origen del sitio. localhost:5173 y 127.0.0.1:5173 tienen almacenes distintos, igual que dos puertos o navegadores diferentes. El prefijo guest identifica al Invitado y no a una cuenta registrada.

Las contraseñas se verifican mediante PBKDF2-SHA-256 con sal y 100000 iteraciones; el código de recuperación se verifica con SHA-256. Estos verificadores no convierten localStorage en un servidor protegido. El ZIP de código no incluye datos guardados en los navegadores.

6 · DE UN CLIC AL RESULTADO

Recorridos explicados

Crear o editar una ficha

CharacterEditor → AppForm → submit → saveCharacter → commit → writeCharacters

1. El editor prepara los campos y el retrato. AppForm recoge FormData y valida las reglas.
2. Si hay errores, los muestra y enfoca el primer campo. Si son válidos, llama al callback del editor.
3. submit construye la ficha. saveCharacter normaliza, crea id si falta, actualiza la fecha y añade o reemplaza por id.
4. commit guarda la lista. Solo después actualiza characters en React.
5. El editor vuelve a Biblioteca con un mensaje. Si falla la escritura, permanece en el formulario y muestra el error.

Registro y acceso

Auth → registerAccount → código de recuperación
Login → loginAccount → App.user → Workspace del propietario

El registro verifica campos y correo duplicado, genera sal y verificadores y devuelve el código. El acceso compara el verificador calculado; Recordarme escribe el id de sesión en localStorage y la opción temporal utiliza sessionStorage. App recibe la identidad y vuelve a montar Workspace para ese perfil.

Recuperar la contraseña

Auth con recovery pide correo, código y una contraseña nueva confirmada. recoverAccount verifica el código, cambia sal y contraseña y borra las referencias de sesión de esa cuenta. La identidad id permanece, por lo que su biblioteca sigue perteneciendo a la misma cuenta.

Exportar una copia

Profile → downloadCharacters → JSON.stringify → Blob → enlace temporal → descarga

El archivo incluye app: UMBRA, version: 1 y characters. Se conservan favoritos y retratos subidos porque forman parte de cada ficha. El nombre incluye una fecha UTC. La URL temporal se libera después de iniciar la descarga.

Restaurar una copia

readBackup → importCharacters → parseCharacters → confirmar → onImport → replaceCharacters

1. Perfil lee un archivo de hasta 10 MB con file.text().
2. La utilidad analiza el JSON y comprueba formato, versión, lista y fichas; rechaza ids repetidos.
3. La copia validada queda temporalmente en backup. El diálogo indica cuántas fichas sustituirá.
4. Confirmar llama replaceCharacters y guarda la nueva biblioteca del propietario activo. Cancelar conserva la biblioteca anterior.

Una copia de personajes no contiene cuentas, credenciales o accesibilidad. Una lista vacía importada se guarda vacía y no se rellena con ejemplos al recargar.

Errores y conexión

Los formularios, diálogos y Perfil capturan fallos y muestran feedback. En personajes se conserva el estado anterior si falla el guardado. En accesibilidad el ajuste se aplica en memoria aunque no se pueda persistir. ConnectionStatus anuncia los eventos de red, pero no permite recargar sin conexión por sí mismo.

7 · AJUSTES INDEPENDIENTES

Accesibilidad por perfil

Menú → Accesibilidad abre cinco entradas. Cada una muestra el nombre del perfil al que se aplican los cambios.

Menú	Opciones
Tamaño de texto	small, normal, large y extra-large. El deslizador y los radios cambian el mismo valor.
Modo de contraste	normal, high e inverted. Los modos de contraste sustituyen los colores de la interfaz y mantienen la selección de tema.
Lectores de pantalla	announceNavigation añade el nombre de la pantalla a una región de estado. readAloud muestra lectura voluntaria de fichas. readingRate elige 0.75, 1 o 1.25.
Navegación por teclado	enhancedFocus refuerza el contorno. La guía explica Tab, Mayús+Tab, Enter/Espacio, flechas, Inicio/Fin y Escape.
Personalización visual	theme: dark/light/sepia; elementSize: normal/large/extra-large; accent: red/purple/blue/green/orange/slate; reduceMotion.

Control → Accessibility.update → usePreferences.updatePreferences → preferenceStore → data-* de Workspace → variables CSS

La normalización admite solo valores del catálogo. Cada propietario utiliza su clave v2. Invitado puede recuperar la antigua configuración global; una cuenta nueva recibe defaults. Restablecer modifica únicamente el perfil activo.

Las etiquetas semánticas, textos alternativos y el diálogo nativo se ofrecen siempre. La voz adicional depende de las capacidades y las voces del dispositivo; no activa ni sustituye TalkBack, VoiceOver o NVDA.

Las reglas CSS de contraste aparecen después de los temas y acentos para que prevalezcan. Al elegir contraste normal vuelven a verse los colores guardados. La app también respeta la preferencia de movimiento reducido del sistema.

8 · TRABAJAR CON EL PROYECTO

Ejecución y pruebas

Extrae codigo/ en una carpeta y abre una terminal allí. En PowerShell puede utilizarse npm.cmd para evitar el bloqueo de npm.ps1 por la política de ejecución.

Comando	Para qué sirve
`npm.cmd install`	Instala dependencias según el proyecto. Con un lock compatible, npm.cmd ci ofrece instalación reproducible.
`npm.cmd run dev`	Inicia Vite para editar; se abre la URL que muestre la terminal.
`npm.cmd test`	Ejecuta las 13 pruebas unitarias declaradas mediante node --test.
`npm.cmd run lint`	Comprueba reglas de JavaScript, JSX y hooks.
`npm.cmd run build`	Compila el frontend en dist/.
`npm.cmd run preview`	Sirve la compilación para comprobarla localmente.
`npm.cmd run test:browser`	Ejecuta recorridos con app activa y Chrome/Edge con CDP en el puerto 9222.

Pruebas unitarias incluidas: 13

Archivo	Caso
characterStore.test.js	una biblioteca vacía persiste y las cuentas conservan bibliotecas independientes
characterStore.test.js	una copia restaura los datos y rechaza identificadores repetidos o contenido inválido
characterStore.test.js	un fallo de almacenamiento no destruye la copia anterior
localAccounts.test.js	registro, verificación de contraseña, sesiones temporales y Recordarme
localAccounts.test.js	recuperación requiere el código y cambia la contraseña conservando la identidad
preferenceStore.test.js	cada perfil y el invitado conservan todos sus ajustes independientes
preferenceStore.test.js	los ajustes globales antiguos se recuperan solo para el invitado
preferenceStore.test.js	los valores desconocidos no se aplican y el fallo de guardado no sobrescribe ajustes anteriores

Archivo	Caso
validateForm.test.js	un nombre vacío o compuesto de espacios requiere una corrección
validateForm.test.js	el correo exige usuario, arroba y dominio
validateForm.test.js	el registro exige longitud suficiente, un carácter especial y contraseñas iguales
validateForm.test.js	el nivel debe ser entero y estar entre 1 y 20
validateForm.test.js	una casilla obligatoria debe estar marcada

Recorridos de navegador

browser-smoke.mjs utiliza un contexto temporal aislado mediante CDP. Además del CRUD, verifica cuentas, copias, separación de perfiles, tamaños, contrastes, foco, estructura accesible, pantallas estrechas, conexión y 404. UMBRA_URL cambia la URL base y UMBRA_SCREENSHOT guarda una captura.

Las voces se simulan en algunos casos; esas pruebas comprueban la conexión de los controles, no una salida de audio real ni el uso de un lector de pantalla externo. La existencia de pruebas no garantiza por sí sola que se hayan ejecutado después de cada cambio.

Publicación

build produce dist/. El alojamiento debe resolver las rutas de BrowserRouter a index.html y utilizar HTTPS cuando se sirva fuera de localhost. La configuración actual de Vite no contiene endpoints ni proxy para una API.

9 · PRÓXIMA CONEXIÓN

Integración futura con Spring Boot

Esta sección es una propuesta de integración. Los servicios HTTP y endpoints descritos aquí todavía no existen en este frontend.

> Página → hook → servicio HTTP → controlador REST → servicio Spring → repositorio ORM → base de datos

El punto principal de sustitución es la persistencia. useCharacters podría conservar su interfaz para las páginas y llamar a un servicio de personajes mediante fetch. El acceso, registro y recuperación pasarían a depender de respuestas del backend.

Pieza actual	Cambio al conectar el servidor
useCharacters + characterStore	Cambiar lecturas y escrituras por operaciones HTTP, con estados de carga, error y respuesta del servidor.
localAccounts + App/Auth/Login	Sustituir verificadores y sesiones locales por la autenticación real del backend. Los roles deben asignarse y comprobarse en el servidor.
preferenceStore	Decidir si los ajustes permanecen en el dispositivo o se sincronizan en una entidad o recurso del usuario.
Modelo de personaje	Acordar DTO: ids de cadena o numéricos, nombres de campos, listas de rasgos, orden de atributos y representación o subida del retrato.
Configuración	Añadir URL base, por ejemplo VITE_API_URL, y acordar CORS o un proxy de desarrollo. No colocar secretos del servidor en variables del frontend.

Un contrato posible para personajes sería GET y POST /api/personajes, más GET, PUT y DELETE /api/personajes/{id}. Las rutas definitivas deben coincidir con los controladores que se creen. La normalización actual de id espera cadenas; habrá que adaptar ids numéricos si los DTO del ORM los utilizan.

La base de datos y el ORM se quedan en Spring Boot. El navegador envía y recibe datos HTTP; no utiliza directamente repositorios ORM ni credenciales de la base de datos.

10 · CONSULTA DEL CÓDIGO

Índice de archivos

A continuación aparecen las explicaciones y los listados completos. Las líneas son las del archivo original; una línea larga puede ocupar varias líneas impresas conservando un único número.

Ruta	Responsabilidad
index.html	Documento HTML que recibe el navegador y punto de entrada de Vite.
src/main.jsx	Arranca React, conecta el enrutador y carga los estilos globales.
src/App.jsx	Coordina la sesión, las rutas, la biblioteca del perfil activo, sus ajustes y los diálogos de la aplicación.
src/pages/Home.jsx	Pantalla de inicio: portada, acceso a crear personajes y las dos fichas más recientes.
src/pages/Library.jsx	Explora toda la biblioteca con búsqueda, filtros, ordenación y estados vacíos.
src/pages/CreateCharacter.jsx	Comparte un editor entre la creación y la modificación de fichas.
src/pages/Login.jsx	Formulario de acceso a una cuenta local del navegador.
src/pages/Auth.jsx	Utiliza AppForm para registrar una cuenta o recuperar su contraseña.
src/pages/Profile.jsx	Muestra la identidad activa, sus recuentos, el cierre de sesión y los controles de copia de seguridad.
src/pages/Accessibility.jsx	Contiene el menú principal de accesibilidad y los cinco submenús, con configuración independiente por perfil.
src/pages/Information.jsx	Agrupa Inspiración, Ayuda, Privacidad y la pantalla de ruta inexistente.
src/components/header/Header.jsx	Cabecera compartida con marca, menú de secciones y acceso a avisos o ayuda.
src/components/bottomnavigation/BottomNavigation.jsx	Barra inferior para Inicio, Biblioteca, Inspiración y Perfil.
src/components/hero/Hero.jsx	Portada de Inicio con paisaje, mensaje de bienvenida y acceso al editor.
src/components/charactercard/CharacterCard.jsx	Tarjeta pulsable de un personaje en Inicio y Biblioteca.

Ruta	Responsabilidad
src/components/charactercard/Portrait.jsx	Presenta un retrato y ofrece iniciales si no hay imagen utilizable.
src/components/modal/Modal.jsx	Ventana modal reutilizable basada en el elemento nativo dialog.
src/components/forms/AppForm.jsx	Motor de formularios reutilizable para personajes, registro y recuperación.
src/components/feedback/ConnectionStatus.jsx	Aviso de pérdida y recuperación de conexión del navegador.
src/components/feedback/ReadAloud.jsx	Botón para leer texto en español mediante las voces del navegador o dispositivo.
src/components/icon/Icon.jsx	Catálogo de iconos SVG reutilizables sin archivos externos.
src/hooks/useCharacters.js	Mantiene la biblioteca React del perfil y coordina las escrituras persistentes.
src/hooks/usePreferences.js	Carga y aplica los ajustes de accesibilidad del perfil activo.
src/utils/characterStore.js	Valida fichas, lee y escribe bibliotecas y prepara las copias JSON.
src/utils/localAccounts.js	Implementa cuentas y sesiones locales mediante Web Crypto y almacenamiento del navegador.
src/utils/preferenceStore.js	Define valores predeterminados y persistencia de accesibilidad separada por perfil.
src/utils/validateForm.js	Validación declarativa compartida por AppForm y Login.
src/data/characters.js	Seis fichas iniciales y generador de semillas de personaje.
src/data/characterOptions.js	Catálogos de clases, razas, rasgos y nombres de atributos, además de la ficha vacía.
src/data/navigation.js	Fuente común de enlaces y títulos de pantalla.
src/data/accessibility.js	Catálogos de valores y etiquetas de los cinco menús de accesibilidad.
src/index.css	Fuentes locales, reinicio de estilos y variables base de color, texto y controles.
src/App.css	Composición general, tarjetas, formularios, diálogos y reglas globales de accesibilidad.

Ruta	Responsabilidad
src/pages/Login.css	Composición específica de la pantalla de acceso.
src/pages/Accessibility.css	Presentación del menú y los controles de accesibilidad.
package.json	Identidad del paquete, dependencias directas y comandos de desarrollo.
vite.config.js	Configuración del compilador y del servidor Vite.
eslint.config.js	Reglas de análisis estático para JavaScript y JSX.
tests/characterStore.test.js	Pruebas del almacenamiento y las copias de personajes.
tests/localAccounts.test.js	Pruebas de registro, verificadores, sesiones y recuperación.
tests/preferenceStore.test.js	Pruebas de accesibilidad independiente por perfil.
tests/validateForm.test.js	Pruebas de las reglas compartidas de formularios.
scripts/browser-smoke.mjs	Recorridos de integración en Chrome o Edge mediante Chrome DevTools Protocol, sin dependencia de Playwright.
public/favicon.svg	Icono vectorial de la pestaña del navegador.
public/images/ASSETS.md	Descripción y procedencia de los recursos gráficos incluidos.
.gitignore	Patrones que Git ignora para evitar añadir dependencias, compilados y archivos temporales.
README.md	Guía breve original del repositorio con comandos, funciones y límites del almacenamiento local.
LICENSE	Licencia del proyecto incluida en el repositorio.
public/fonts/Inter-OFL.txt	Licencia de fuente distribuida con los recursos.
public/fonts/Cormorant-Garamond-OFL.txt	Licencia de fuente distribuida con los recursos.

ARCHIVO 01 DE 50

index.html

`index.html`

Documento HTML que recibe el navegador y punto de entrada de Vite.

- Declara el idioma español, la codificación UTF-8, el viewport móvil, el icono y la descripción.
- El div #root es el lugar donde React monta la interfaz. El script de tipo module carga src/main.jsx.

Funciones y elementos principales

Nombre	Qué hace	Línea
`#root`	Contenedor de montaje de React; no es una función de JavaScript.	—

Código completo

15 líneas · SHA-256 del original: `a4b5b04a163ff023f3c8102426002f20b1b158dc381205f4221eb128a55c1dac`

```
1  <!doctype html>
2  <html lang="es">
3    <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6      <meta name="theme-color" content="#090f0f" />
7      <link rel="icon" type="image/svg+xml" href="/favicon.svg" />
8      <meta name="description" content="UMBRA: crea personajes y da vida a tus historias." />
9      <title>UMBRA · Tus historias</title>
10   </head>
11   <body>
12     <div id="root"></div>
13     <script type="module" src="/src/main.jsx"></script>
14   </body>
15 </html>
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 02 DE 50

main.jsx

`src/main.jsx`

Arranca React, conecta el enrutador y carga los estilos globales.

- createRoot(document.getElementById('root')) crea la raíz de React.
- BrowserRouter permite interpretar las rutas de la barra de direcciones. StrictMode ayuda a detectar efectos y limpiezas incorrectos durante el desarrollo.

Funciones y elementos principales

Nombre	Qué hace	Línea
`createRoot(...).render(...)`	Monta App dentro de BrowserRouter y StrictMode.	—

Código completo

13 líneas · SHA-256 del original: `a0c30b0734ced3997d30279cc261a3ab1c2dc13a7a16f736b62a6d8e132b012d`

```
1  import { StrictMode } from 'react'
2  import { createRoot } from 'react-dom/client'
3  import { BrowserRouter } from 'react-router'
4  import App from './App.jsx'
5  import './index.css'
6
7  createRoot(document.getElementById('root')).render(
8    <StrictMode>
9      <BrowserRouter>
10       <App />
11     </BrowserRouter>
12   </StrictMode>,
13 )
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 03 DE 50

App.jsx

`src/App.jsx`

Coordina la sesión, las rutas, la biblioteca del perfil activo, sus ajustes y los diálogos de la aplicación.

Entradas / props. Workspace recibe user, onLogin, onLogout y accountError. App es el componente raíz y no necesita props.

Estado. App conserva la sesión inicial y user. Workspace conserva modal y actionError; obtiene characters y preferences de los hooks.

- Workspace se monta con una key basada en el identificador del usuario o guest. Cambiar de cuenta vuelve a cargar biblioteca y preferencias de ese perfil.
- El efecto de navegación actualiza document.title, dirige el foco al contenido principal y vuelve al principio de la pantalla.
- Los atributos data-text-size, data-contrast, data-theme, data-element-size y data-accent conectan los ajustes con CSS.
- La ficha se abre en Modal y ofrece favoritos, edición, duplicado y eliminación con confirmación. Las notificaciones muestran información local de la biblioteca.
- Las rutas de acceso, registro y recuperación redirigen a Perfil cuando ya existe una sesión. La ruta desconocida muestra NotFound.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Workspace`	Compone cabecera, contenido, rutas, navegación inferior, avisos y diálogos.	24
`selectCharacter`	Recibe un personaje y abre su ficha mediante el identificador.	44
`act`	Ejecuta una acción y convierte sus errores en feedback dentro del diálogo.	45
`leaveModal`	Borra el error de acción y cierra el diálogo activo.	49
`App`	Recupera la sesión local y entrega sus cambios a Workspace.	105
`login`	Espera loginAccount y actualiza user con la identidad devuelta.	111
`logout`	Elimina las referencias de sesión y cambia al perfil Invitado.	112

Código completo

114 líneas · SHA-256 del original: `6d07d99caa4e8e73f692ef39422f56836d7e14909aec8f62c07cf6b95a6d7f0d`

```
import { useEffect, useRef, useState } from 'react'
import { Link, Navigate, Route, Routes, useLocation, useNavigate } from 'react-router'
import Header from './components/header/Header.jsx'
import BottomNavigation from './components/bottomnavigation/BottomNavigation.jsx'
import ConnectionStatus from './components/feedback/ConnectionStatus.jsx'
import Modal from './components/modal/Modal.jsx'
import Portrait from './components/charactercard/Portrait.jsx'
import Home from './pages/Home.jsx'
import Login from './pages/Login.jsx'
import Library from './pages/Library.jsx'
import CreateCharacter from './pages/CreateCharacter.jsx'
import Auth from './pages/Auth.jsx'
import Profile from './pages/Profile.jsx'
import Accessibility from './pages/Accessibility.jsx'
import { Help, Inspiration, NotFound, Privacy } from './pages/Information.jsx'
import ReadAloud from './components/feedback/ReadAloud.jsx'
import { useCharacters } from './hooks/useCharacters.js'
import { usePreferences } from './hooks/usePreferences.js'
import { loginAccount, logoutAccount, readSession } from './utils/localAccounts.js'
import { attributeNames } from './data/characterOptions.js'
import { pageTitles } from './data/navigation.js'
import './App.css'

function Workspace({ user, onLogin, onLogout, accountError }) {
  const { pathname } = useLocation()
  const navigate = useNavigate()
  const main = useRef(null)
  const [modal, setModal] = useState(null)
  const [actionError, setActionError] = useState('')
  const { characters, storageError, saveCharacter, deleteCharacter, toggleFavorite, replaceCharacters } = useCharacters(user?.id || 'guest')
  const { preferences, updatePreferences, preferenceError } = usePreferences(user?.id || 'guest')
  const path = pathname.replace(/\/$/, '') || '/'
  const editing = /^\/personaje\/[^/]+\/editar$/.test(path)
  const page = pageTitles[path] || (editing ? 'Editar personaje' : 'Página no encontrada')
  const isLogin = path === '/login'
  const character = modal?.id ? characters.find(item => item.id === modal.id) : null

  useEffect(() => {
    document.title = `UMBRA · ${page}`
    main.current.focus({ preventScroll: true })
    window.scrollTo(0, 0)
  }, [pathname, page])

  function selectCharacter(item) { setActionError(''); setModal({ type: 'character', id: item.id }) }
  function act(callback) {
    try { callback(); setActionError('') }
    catch (error) { setActionError(error.message) }
  }
  function leaveModal() { setActionError(''); setModal(null) }

  const className = ['app-frame', isLogin && 'login-frame', preferences.enhancedFocus && 'enhanced-focus', preferences.reduceMotion && 'reduce-motion'].filter(Boolean).join(' ')

```

```
  return <div className={className} data-text-size={preferences.textSize} data-contrast={preferences.contrast} data-theme={preferences.theme} data-element-size={preferences.elementSize} data-accent={preferences.accent}>
    <a className="skip-link" href="#main-content">Saltar al contenido</a>
    <Header login={isLogin} user={user} onNotify={() => setModal({ type: 'notifications' })} />
    <ConnectionStatus />
    {preferences.announceNavigation && <p className="visually-hidden" role="status" aria-live="polite" aria-atomic="true">Página: {page}</p>}
    {(storageError || accountError) && <p className="storage-notice" role="alert">{storageError || accountError}</p>}
    <main ref={main} className="main-content" id="main-content" tabIndex={-1}>
      {path !== '/inicio' && path !== '/' && !isLogin && <div className="page-heading"><p className="eyebrow">UMBRA · TUS HISTORIAS</p><h1>{page}</h1></div>}
      <Routes>
        <Route path="/" element={<Navigate to="/inicio" replace />} />
        <Route path="/inicio" element={<Home characters={characters} user={user} onSelect={selectCharacter} />} />
        <Route path="/biblioteca" element={<Library characters={characters} onSelect={selectCharacter} />} />
        <Route path="/crear-personaje" element={<CreateCharacter key="create" characters={characters} onSave={saveCharacter} />} />
        <Route path="/personaje/:id/editar" element={<CreateCharacter key={path} characters={characters} onSave={saveCharacter} />} />
        <Route path="/inspiracion" element={<Inspiration />} />
        <Route path="/perfil" element={<Profile user={user} characters={characters} onLogout={onLogout} onImport={replaceCharacters} />} />
        <Route path="/login" element={user ? <Navigate to="/perfil" replace /> : <Login onLogin={onLogin} />} />
        <Route path="/registro" element={user ? <Navigate to="/perfil" replace /> : <Auth key="register" />} />
        <Route path="/recuperar" element={user ? <Navigate to="/perfil" replace /> : <Auth key="recover" recovery />} />
        <Route path="/ayuda" element={<Help />} />
        <Route path="/accesibilidad/:section?" element={<Accessibility preferences={preferences} onChange={updatePreferences} error={preferenceError} user={user} />} />
        <Route path="/privacidad" element={<Privacy />} />
        <Route path="*" element={<NotFound />} />
      </Routes>
    </main>
    {!isLogin && <BottomNavigation />}
    {modal && <Modal title={character ? (modal.type === 'delete' ? `¿Eliminar a ${character.name}?` : character.name) : 'Notificaciones'} onClose={leaveModal}>
      {modal.type === 'notifications' ? <div className="action-stack"><p className="page-description">No tienes avisos nuevos. Tu biblioteca tiene {characters.length} personajes y se guarda en este navegador.</p><p className="form-hint">Recuerda exportar una copia desde Perfil para conservar tus historias.</p><Link className="secondary-button" to="/perfil" onClick={leaveModal}>Ir a Perfil</Link></div>
        : character && (modal.type === 'delete' ? <div className="action-stack">
          <p className="page-description">La ficha se eliminará de esta biblioteca. Esta acción no se puede deshacer.</p>
          <button className="primary-button" onClick={() => act(() => { deleteCharacter(character.id); leaveModal() })}>Eliminar personaje</button>
          <button className="secondary-button" onClick={() => setModal({ type: 'character', id: character.id })}>Conservar personaje</button>
        </div> : <>
          <Portrait character={character} />
          <p className="eyebrow">{character.race} · {character.type} · Nivel {character.level}</p>
          {character.realm && <p className="page-description">{character.realm}</p>}
          <p className="character-story">{character.story || 'Esta historia está por escribir.'}</p>
```

```
          {preferences.readAloud && <ReadAloud key={character.id} text={`${character.name}. ${character.race}. ${character.type}. Nivel ${character.level}. ${character.realm}. ${character.story}. Rasgos: ${character.traits.join(', ')}. ${attributeNames.map((name, index) => `${name}: ${character.stats[index]}`).join('. ')}.`} rate={preferences.readingRate} />}
          <ul className="trait-list">{character.traits.map(trait => <li key={trait}>{trait}</li>)}</ul>
          <dl className="attribute-grid">{attributeNames.map((name, index) => <div key={name}><dt>{name}</dt><dd>{character.stats[index]}</dd></div>)}</dl>
          <div className="action-stack">
            <button className="secondary-button" aria-pressed={character.favorite} onClick={() => act(() => toggleFavorite(character.id))}>{character.favorite ? 'Quitar de favoritos' : 'Guardar en favoritos'}</button>
            <Link className="primary-button" to={`/personaje/${character.id}/editar`} onClick={leaveModal}>Editar personaje</Link>
            <button className="secondary-button" onClick={() => act(() => { const copy = saveCharacter({ ...character, id: undefined, name: `${character.name.slice(0, 52)} (copia)`, favorite: false }); leaveModal(); navigate('/biblioteca', { state: { message: `${copy.name}: personaje duplicado.` } }) })}>Duplicar personaje</button>
            <button className="text-button danger-button" onClick={() => setModal({ type: 'delete', id: character.id })}>Eliminar personaje</button>
          </div>
        </>)}
      {actionError && <p className="feedback error" role="alert">{actionError}</p>}
    </Modal>}
  </div>
}

export default function App() {
  const [initial] = useState(() => {
    try { return { user: readSession(), error: '' } }
    catch { return { user: null, error: 'No se pudo leer la sesión local. Comprueba los permisos de almacenamiento del navegador.' } }
  })
  const [user, setUser] = useState(initial.user)
  async function login(data) { setUser(await loginAccount(data)) }
  function logout() { logoutAccount(); setUser(null) }
  return <Workspace key={user?.id || 'guest'} user={user} onLogin={login} onLogout={logout} accountError={initial.error} />
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 04 DE 50

Home.jsx

`src/pages/Home.jsx`

Pantalla de inicio: portada, acceso a crear personajes y las dos fichas más recientes.

Entradas / props. characters: biblioteca activa; onSelect: abre una ficha; user: nombre de la cuenta o null.

- Ordena una copia de characters por updatedAt y toma los dos primeros. No cambia el orden del array original.
- Utiliza Hero y CharacterCard. Muestra un estado vacío cuando no existen personajes y un acceso al perfil o al login.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Home`	Construye Inicio a partir de la biblioteca y de la sesión activa.	5

Código completo

21 líneas · SHA-256 del original: `035f7ac6a58d3da2d7ef29f9f44eccc24f3b657da27da6622153ee4a702501f3`

```
1  import { Link } from 'react-router'
2  import Hero from '../components/hero/Hero.jsx'
3  import CharacterCard from '../components/charactercard/CharacterCard.jsx'
4
5  export default function Home({ onSelect, characters, user }) {
6    const recent = [...characters].sort((a, b) => b.updatedAt - a.updatedAt).slice(0, 2)
7    return <>
8      <Hero />
9      <div className="ornament" aria-hidden="true"><span>❖</span></div>
10     <section aria-labelledby="characters-title">
11       <div className="section-heading">
12         <h2 id="characters-title">Personajes recientes</h2>
13         <Link className="text-button" to="/biblioteca">Ver todos <span aria-hidden="true">→</span></Link>
14       </div>
15       <div className="character-grid">{recent.map(character => <CharacterCard key={character.id}
   character={character} onSelect={onSelect} />)}</div>
16       {!recent.length && <p className="page-description">Tu biblioteca está vacía. Crea un personaje para
   comenzar tu historia.</p>}
17     </section>
18     <p className="closing-quote">«Incluso en la oscuridad, una historia perdura.»</p>
19     <p className="guest-note">{user ? <>Bienvenido, {user.name}. <Link to="/perfil">Ver perfil</Link></>
   : <>Estás explorando como invitado. <Link to="/login">Iniciar sesión</Link></>}</p>
20   </>
21 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 05 DE 50

Library.jsx

`src/pages/Library.jsx`

Explora toda la biblioteca con búsqueda, filtros, ordenación y estados vacíos.

Entradas / props. characters: biblioteca del perfil; onSelect: acción al pulsar una tarjeta.

Estado. search, type, favorites y sort representan los filtros que está usando la persona.

- La búsqueda abarca nombre, clase, raza y reino. Normaliza mayúsculas y tildes para que angelica encuentre Angélica.
- Combina la búsqueda con la clase seleccionada y con Solo favoritos. Ordena por actualización, nombre o mayor nivel.
- Lee location.state.message para mostrar la confirmación recibida tras crear, editar o duplicar. Limpiar filtros no elimina personajes.

Funciones y elementos principales

Nombre	Qué hace	Línea
`simplify`	Normaliza una cadena con NFD, elimina marcas diacríticas y convierte a minúsculas españolas.	7
`Library`	Calcula los resultados y presenta los filtros, el contador y las tarjetas.	9

Código completo

34 líneas · SHA-256 del original: `936556e606f01ff84c8273acc319594f9b70d9321d590c3b6a9d8da92436cec3`

```
1  import { useState } from 'react'
2  import { Link, useLocation } from 'react-router'
3  import CharacterCard from '../components/charactercard/CharacterCard.jsx'
4  import Icon from '../components/icon/Icon.jsx'
5  import { characterTypes } from '../data/characterOptions.js'
6
7  const simplify = value => value.normalize('NFD').replace(/[\u0300-\u036f]/g, '').toLocaleLowerCase('es')
8
9  export default function Library({ onSelect, characters }) {
10   const [search, setSearch] = useState('')
11   const [type, setType] = useState('')
12   const [favorites, setFavorites] = useState(false)
13   const [sort, setSort] = useState('recent')
14   const { state } = useLocation()
15   const results = characters.filter(character =>
```

```
16      simplify(`${character.name} ${character.type} ${character.race}
    ${character.realm}`).includes(simplify(search.trim())) &&
17      (!type || character.type === type) && (!favorites || character.favorite))
18      .sort((a, b) => sort === 'name' ? a.name.localeCompare(b.name, 'es') : sort === 'level' ? b.level -
    a.level : b.updatedAt - a.updatedAt)
19
20    return <section aria-labelledby="library-title">
21      <div className="section-heading"><h2 id="library-title">Tus personajes</h2><Link className="text-
    button" to="/crear-personaje">+ Crear personaje</Link></div>
22      {state?.message && <p className="feedback success" role="status">{state.message}</p>}
23      <label className="search-field"><Icon name="search" /><input aria-label="Buscar por nombre, clase,
    raza o reino" type="search" value={search} onChange={event => setSearch(event.target.value)}
    placeholder="Buscar personaje…" /></label>
24      <div className="form-row library-filters">
25        <label>Clase<select value={type} onChange={event => setType(event.target.value)}><option
    value="">Todas las clases</option>{characterTypes.map(item => <option key={item}>{item}</option>)}
    </select></label>
26        <label>Ordenar<select value={sort} onChange={event => setSort(event.target.value)}><option
    value="recent">Más recientes</option><option value="name">Nombre</option><option value="level">Mayor
    nivel</option></select></label>
27      </div>
28      <label className="preference-option"><input type="checkbox" checked={favorites} onChange={event =>
    setFavorites(event.target.checked)} />Solo favoritos</label>
29      <p className="search-results" role="status">{results.length} {results.length === 1 ? 'personaje
    encontrado' : 'personajes encontrados'}.</p>
30      <div className="character-grid">{results.map(character => <CharacterCard key={character.id}
    character={character} onSelect={onSelect} />)}</div>
31      {results.length === 0 && <div className="empty-state"><p>{characters.length ? 'No hay personajes que
    coincidan con los filtros.' : 'Tu biblioteca está vacía. Empieza una nueva leyenda.'}</p>
    {characters.length ? <button className="secondary-button" onClick={() => { setSearch(''); setType('');
    setFavorites(false) }}>Limpiar filtros</button> : <Link className="primary-button" to="/crear-
    personaje">Crear mi primer personaje</Link>}</div>}
32      <p className="page-description">Tus cambios se guardan en este navegador. Puedes exportar una copia
    de seguridad desde Perfil.</p>
33    </section>
34  }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 06 DE 50

CreateCharacter.jsx

`src/pages/CreateCharacter.jsx`

Comparte un editor entre la creación y la modificación de fichas.

Entradas / props. CreateCharacter recibe characters y onSave. CharacterEditor recibe initial, onSave y editing.

Estado. El contenedor mantiene draft y revision. El editor mantiene el retrato, el estilo, el error de imagen y si está leyéndose un archivo.

- La ruta con :id busca la ficha existente. Sin :id utiliza blankCharacter o el borrador enviado desde Inspiración. Un id ausente muestra NotFound.
- Declara campos de nombre, clase, raza, nivel, origen, historia y cuatro atributos para AppForm. Ofrece hasta seis rasgos de personalidad.
- El retrato puede ser uno de los recursos incluidos, las iniciales o una imagen PNG/JPG/WebP de hasta 750 KB. FileReader convierte la imagen a data URL y un objeto Image comprueba que se pueda decodificar.
- Generar personaje aleatorio sustituye el borrador del editor; revision cambia la key para inicializar de nuevo los campos. Esta acción no guarda nada hasta pulsar Guardar.
- Cancelar muestra confirmación si AppForm ha detectado cambios; no existe un bloqueo general de todos los enlaces internos ni una recuperación automática de borradores.

Funciones y elementos principales

Nombre	Qué hace	Línea
`CharacterEditor`	Presenta AppForm, los rasgos y la selección o carga del retrato.	10
`upload`	Revisa tipo y tamaño del archivo, lee su contenido, comprueba la imagen y actualiza la vista previa.	26
`submit`	Construye la ficha con FormData, llama onSave y navega a Biblioteca con un mensaje.	54
`CreateCharacter`	Decide si se está creando o editando y prepara la ficha inicial.	80

Código completo

92 líneas · SHA-256 del original: `2e37c5f882c86bf1c5fb2133faa692ef872a89558ce7f28bfcadddac1f647a14`

```
import { useState } from 'react'
import { useLocation, useNavigate, useParams } from 'react-router'
import AppForm from '../components/forms/AppForm.jsx'
import Portrait from '../components/charactercard/Portrait.jsx'
import { attributeNames, blankCharacter, characterRaces, characterTypes, personalityTraits } from '../data/characterOptions.js'
import { randomCharacter } from '../data/characters.js'
import { MAX_IMAGE_BYTES } from '../utils/characterStore.js'
import { NotFound } from './Information.jsx'

function CharacterEditor({ initial, onSave, editing }) {
  const navigate = useNavigate()
  const [image, setImage] = useState({ imageDataUrl: initial.imageDataUrl || '', imageName: initial.imageName || '' })
  const [portrait, setPortrait] = useState(initial.portrait <= 1 ? initial.portrait : 2)
  const [imageError, setImageError] = useState('')
  const [readingImage, setReadingImage] = useState(false)
  const fields = [
    { name: 'name', label: 'Nombre del personaje', required: true, minLength: 2, maxLength: 60, defaultValue: initial.name, placeholder: 'El nombre de tu leyenda' },
    { name: 'type', label: 'Clase', required: true, options: characterTypes, defaultValue: initial.type },
    { name: 'race', label: 'Raza', required: true, options: characterRaces, defaultValue: initial.race },
    { name: 'level', label: 'Nivel', type: 'number', required: true, min: 1, max: 20, defaultValue: initial.level, hint: 'Un número entero entre 1 y 20.' },
    { name: 'realm', label: 'Reino u origen', maxLength: 100, defaultValue: initial.realm },
    { name: 'story', label: 'Historia', type: 'textarea', maxLength: 3000, defaultValue: initial.story, placeholder: 'Toda leyenda tiene un comienzo…' },
    ...attributeNames.map((label, index) => ({ name: `stat${index}`, label, type: 'number', required: true, min: 1, max: 20, defaultValue: initial.stats[index], hint: 'Entre 1 y 20.' })),
  ]

  async function upload(event) {
    const file = event.target.files[0]
    if (!file) return
    setImageError('')
    if (!['image/png', 'image/jpeg', 'image/webp'].includes(file.type) || file.size > MAX_IMAGE_BYTES) {
      setImageError('Selecciona una imagen PNG, JPG o WebP de hasta 750 KB.')
      event.target.value = ''
      return
    }
    setReadingImage(true)
    try {
      const imageDataUrl = await new Promise((resolve, reject) => {
        const reader = new FileReader()
        reader.onload = () => resolve(reader.result)
        reader.onerror = () => reject(new Error('No se pudo leer el retrato.'))
        reader.readAsDataURL(file)
      })
      await new Promise((resolve, reject) => {
        const preview = new Image()
        preview.onload = resolve
        preview.onerror = () => reject(new Error('El archivo no contiene una imagen válida.'))
        preview.src = imageDataUrl
      })
      setImage({ imageDataUrl, imageName: file.name })
    } catch (error) { setImageError(error.message) }
    finally { setReadingImage(false) }
  }
```

```
53
54    function submit(values) {
55      if (readingImage) throw new Error('Espera a que termine de cargarse el retrato.')
56      const character = onSave({
57        ...initial, ...image, portrait,
58        name: values.get('name'), type: values.get('type'), race: values.get('race'),
59        level: Number(values.get('level')), realm: values.get('realm'), story: values.get('story'),
60        stats: attributeNames.map((_, index) => Number(values.get(`stat${index}`))), traits:
values.getAll('traits'),
61      })
62      navigate('/biblioteca', { state: { message: `${character.name}: ${editing ? 'cambios guardados' :
'personaje creado'}.` } })
63    }
64
65    return <AppForm fields={fields} submitLabel={editing ? 'Guardar cambios' : 'Guardar personaje'}
onSubmit={submit} cancelTo="/biblioteca">
66      <fieldset className="traits-field"><legend>Rasgos de personalidad (hasta 6)</legend><div
className="traits-grid">{[...new Set([...personalityTraits, ...initial.traits])].map(trait => <label
className="checkbox-label" key={trait}><input type="checkbox" name="traits" value={trait} defaultChecked=
{initial.traits.includes(trait)} />{trait}</label>)}</div></fieldset>
67      <div className="portrait-editor">
68        <h2>Retrato</h2>
69        <Portrait character={{ ...initial, ...image, portrait, name: initial.name || 'tu personaje' }} />
70        <label>Estilo de retrato<select value={portrait} onChange={event => {
setPortrait(Number(event.target.value)); setImage({ imageDataUrl: '', imageName: '' }) }}><option value=
{0}>Hechicera</option><option value={1}>Pícaro</option><option value={2}>Iniciales del personaje</option>
</select></label>
71        <label>Subir imagen<input type="file" accept="image/png,image/jpeg,image/webp" onChange={upload}
disabled={readingImage} /></label>
72        <p className="form-hint">PNG, JPG o WebP. Máximo 750 KB.{image.imageName && ` Imagen:
${image.imageName}`}</p>
73        {readingImage && <p role="status">Cargando retrato…</p>}
74        {image.imageDataUrl && <button type="button" className="secondary-button" onClick={() => setImage({
imageDataUrl: '', imageName: '' })}>Quitar imagen subida</button>}
75        {imageError && <p className="field-error" role="alert">{imageError}</p>}
76      </div>
77    </AppForm>
78  }
79
80  export default function CreateCharacter({ characters, onSave }) {
81    const { id } = useParams()
82    const { state } = useLocation()
83    const [draft, setDraft] = useState(() => ({ ...blankCharacter(), ...state?.draft, id: undefined }))
84    const [revision, setRevision] = useState(0)
85    const initial = id ? characters.find(character => character.id === id) : draft
86    if (!initial) return <NotFound />
87
88    return <section className="editor-page">
89      {!id && <button type="button" className="secondary-button random-button" onClick={() => { setDraft({
...randomCharacter(), id: undefined }); setRevision(value => value + 1) }}>Generar personaje
aleatorio</button>}
90      <CharacterEditor key={`${id || 'new'}-${revision}`} initial={initial} onSave={onSave} editing=
{Boolean(id)} />
91    </section>
92  }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 07 DE 50

Login.jsx

`src/pages/Login.jsx`

Formulario de acceso a una cuenta local del navegador.

Entradas / props. onLogin: callback asíncrono que verifica credenciales y actualiza la sesión.

Estado. showPassword controla la visibilidad; errors contiene errores de campos; message muestra fallos del acceso; pending evita envíos repetidos.

- Valida que correo y contraseña estén presentes y que el correo tenga formato válido. Los requisitos de creación de contraseña se comprueban en Registro, no se vuelven a exigir al iniciar sesión.
- Permite mostrar u ocultar la contraseña, elegir Recordarme y abrir Registro o Recuperar acceso.
- No existe una cuenta de administrador precreada ni un sistema de roles. Una cuenta llamada Administrador tiene las mismas funciones que las demás.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Login`	Dibuja el formulario y sus mensajes accesibles.	12
`handleSubmit`	Valida, enfoca el primer error o espera onLogin; al tener éxito navega a Perfil.	19

Código completo

90 líneas · SHA-256 del original: `cf0768a1da6013d2d61e81398a606491589ef07d03658fef6b4e72b62dd55d7d`

```
1  import { useState } from 'react'
2  import { Link, useNavigate } from 'react-router'
3  import Icon from '../components/icon/Icon.jsx'
4  import { validateForm } from '../utils/validateForm.js'
5  import './Login.css'
6
7  const fields = [
8    { name: 'email', label: 'Correo electrónico', type: 'email', required: true },
9    { name: 'password', label: 'Contraseña', type: 'password', required: true },
10 ]
11
12 export default function Login({ onLogin }) {
13   const navigate = useNavigate()
14   const [showPassword, setShowPassword] = useState(false)
15   const [errors, setErrors] = useState({})
16   const [message, setMessage] = useState('')
```

```
  const [pending, setPending] = useState(false)

  async function handleSubmit(event) {
    event.preventDefault()
    if (pending) return
    const form = event.currentTarget
    const nextErrors = validateForm(fields, new FormData(form))
    setErrors(nextErrors)
    setMessage('')
    const firstError = Object.keys(nextErrors)[0]
    if (firstError) {
      form.elements.namedItem(firstError).focus()
      return
    }
    setPending(true)
    try {
      const values = new FormData(form)
      await onLogin({ email: values.get('email'), password: values.get('password'), remember:
values.has('remember') })
      navigate('/perfil', { replace: true })
    } catch (error) {
      setMessage(error.message || 'No se pudo acceder. Comprueba el almacenamiento del navegador.')
    } finally { setPending(false) }
  }

  return <section className="login-page" aria-labelledby="login-title">
    <div className="login-intro">
      <Icon name="sigil" size={60} />
      <h1 id="login-title">Cruza el umbral</h1>
      <p>Tus historias te esperan en la oscuridad.</p>
    </div>

    <div className="login-divider" aria-hidden="true"><span>❖</span></div>

    <p className="form-hint" id="login-notice">Accede a tu cuenta local de este navegador. Los campos con
* son obligatorios.</p>
    <form className="login-form" noValidate aria-describedby="login-notice" onSubmit={handleSubmit}>
      <div className="login-field">
        <label htmlFor="login-email">Correo electrónico *</label>
        <div className="login-input">
          <Icon name="mail" size={18} />
          <input id="login-email" name="email" type="email" autoComplete="email" required maxLength={254}
            placeholder="Tu correo electrónico" aria-invalid={Boolean(errors.email)} aria-describedby=
{errors.email ? 'login-email-error' : undefined} />
        </div>
        {errors.email && <p className="field-error" id="login-email-error" role="alert">{errors.email}
</p>}
      </div>

      <div className="login-field">
        <label htmlFor="login-password">Contraseña *</label>
        <div className="login-input">
          <Icon name="lock" size={18} />
          <input id="login-password" name="password" type={showPassword ? 'text' : 'password'}
autoComplete="current-password" required
            placeholder="Introduce tu contraseña" aria-invalid={Boolean(errors.password)} aria-
describedby={errors.password ? 'login-password-error' : undefined} />
          <button type="button" className="login-password-toggle" aria-label={showPassword ? 'Ocultar
contraseña' : 'Mostrar contraseña'}
```

```
69              aria-pressed={showPassword} onClick={() => setShowPassword(!showPassword)}>
70              <Icon name={showPassword ? 'eye' : 'eye-off'} size={18} />
71            </button>
72          </div>
73          {errors.password && <p className="field-error" id="login-password-error" role="alert">
    {errors.password}</p>}
74        </div>
75
76        <div className="login-options">
77          <label className="login-remember"><input name="remember" type="checkbox" />
    <span>Recordarme</span></label>
78          <Link className="login-recovery" to="/recuperar">¿Olvidaste tu contraseña?</Link>
79        </div>
80
81        <button className="primary-button login-submit" type="submit" disabled={pending}><Icon name="login"
    size={18} />{pending ? 'Entrando…' : 'Iniciar sesión'}</button>
82        {message && <p className="feedback error" role="alert">{message}</p>}
83      </form>
84
85      <div className="login-divider login-account-divider"><span>¿Aún no tienes una cuenta?</span></div>
86      <Link className="login-create-account" to="/registro"><Icon name="user-plus" size={18} />Crear
    cuenta</Link>
87      <p className="login-terms"><Link to="/privacidad">Privacidad y condiciones de uso</Link></p>
88      <p className="login-motto">Cada alma tiene una historia.</p>
89    </section>
90  }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 08 DE 50

Auth.jsx

`src/pages/Auth.jsx`

Utiliza AppForm para registrar una cuenta o recuperar su contraseña.

Entradas / props. recovery: false para registro y true para recuperación.

Estado. recoveryCode conserva el código recién entregado; recovered indica que terminó el cambio de contraseña.

- Registro solicita nombre, correo, contraseña de 8 a 256 caracteres con carácter especial, confirmación y aceptación del almacenamiento local.
- La recuperación solicita correo, código y una nueva contraseña confirmada. No envía un email.
- Al crear la cuenta muestra el código de recuperación completo solo en esa pantalla. La persona debe guardarlo; después accede desde Login.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Auth`	Elige los campos y cambia entre formulario, código de recuperación y confirmación de cambio.	10
`submit`	Convierte FormData en un objeto y llama registerAccount o recoverAccount.	19

Código completo

39 líneas · SHA-256 del original: `f9a2022caaa080adac64a076ebe3dd187b626744e6b0631539777c41177ca152`

```
1  import { useState } from 'react'
2  import { Link } from 'react-router'
3  import AppForm from '../components/forms/AppForm.jsx'
4  import { recoverAccount, registerAccount } from '../utils/localAccounts.js'
5
6  const email = { name: 'email', label: 'Correo electrónico', type: 'email', required: true, autoComplete:
   'email', maxLength: 254 }
7  const password = { name: 'password', label: 'Contraseña', type: 'password', required: true, minLength: 8,
   maxLength: 256, requireSpecial: true, autoComplete: 'new-password', hint: 'Al menos 8 caracteres y un
   carácter especial, como !, @, # o $.' }
8  const confirmation = { name: 'confirmPassword', label: 'Repetir contraseña', type: 'password', required:
   true, matches: 'password', autoComplete: 'new-password' }
9
10 export default function Auth({ recovery = false }) {
11   const [recoveryCode, setRecoveryCode] = useState('')
```

```
  const [recovered, setRecovered] = useState(false)
  const fields = recovery ? [email, { name: 'recoveryCode', label: 'Código de recuperación', required: true, maxLength: 32, hint: 'El código que guardaste al crear tu cuenta local.' }, password, confirmation] : [
    { name: 'name', label: 'Nombre', required: true, minLength: 2, maxLength: 60, autoComplete: 'name' },
    email, password, confirmation,
    { name: 'localAcknowledged', label: 'Entiendo que la cuenta y mis personajes se guardan solo en este navegador.', type: 'checkbox', required: true },
  ]

  async function submit(values) {
    const data = Object.fromEntries(values)
    if (recovery) {
      await recoverAccount(data)
      setRecovered(true)
    } else {
      setRecoveryCode(await registerAccount(data))
    }
  }

  return <section className="auth-page">
    {recoveryCode ? <div className="action-stack">
      <p className="feedback success" role="status">Tu cuenta local está creada. Guarda este código antes de continuar: lo necesitarás si olvidas la contraseña.</p>
      <label className="form-field">Código de recuperación<input aria-label="Código de recuperación" readOnly value={recoveryCode} onFocus={event => event.target.select()} /></label>
      <p className="form-hint">Guárdalo en un lugar seguro. No podemos recuperarlo ni enviarlo por correo.</p>
      <Link className="primary-button" to="/login">Ya he guardado el código. Iniciar sesión</Link>
    </div> : recovered ? <div className="action-stack"><p className="feedback success" role="status">Contraseña actualizada. Ya puedes entrar con la nueva contraseña.</p><Link className="primary-button" to="/login">Iniciar sesión</Link></div> : <AppForm fields={fields} onSubmit={submit} submitLabel={recovery ? 'Cambiar contraseña' : 'Crear cuenta local'} notice="La cuenta funciona en este navegador. No se envían correos ni se sincronizan datos entre dispositivos." />}
    {!recoveryCode && !recovered && <p className="account-link">¿Ya tienes cuenta? <Link to="/login">Iniciar sesión</Link></p>}
    <Link className="inline-link" to="/privacidad">Privacidad y uso</Link>
  </section>
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 09 DE 50

Profile.jsx

`src/pages/Profile.jsx`

Muestra la identidad activa, sus recuentos, el cierre de sesión y los controles de copia de seguridad.

Entradas / props. user, characters, onLogout y onImport. onImport está conectado a replaceCharacters en App.

Estado. backup conserva temporalmente la copia validada; message y error dan feedback.

- Exportar prepara un JSON con la biblioteca actual. Importar lee un archivo de hasta 10 MB, lo valida y abre un diálogo antes de sustituir datos.
- La restauración reemplaza toda la biblioteca del perfil activo, incluidos sus favoritos y retratos; no mezcla listas ni afecta a otras cuentas.
- Las copias no incluyen cuentas, contraseñas ni ajustes de accesibilidad. La pantalla incluye enlaces a accesibilidad y privacidad.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Profile`	Presenta la información del perfil y los controles de biblioteca y copia.	7
`readBackup`	Lee el archivo elegido, revisa su tamaño y utiliza importCharacters para validarlo.	12
`restore`	Llama onImport con la copia validada y muestra el resultado; los errores no se anuncian como éxito.	24

Código completo

50 líneas · SHA-256 del original: `7f687b3c23093c5669458a57c18a8a9e94d5954c9c68a1d60e05c81402a7e7f5`

```
1  import { useState } from 'react'
2  import { Link } from 'react-router'
3  import Icon from '../components/icon/Icon.jsx'
4  import Modal from '../components/modal/Modal.jsx'
5  import { downloadCharacters, importCharacters } from '../utils/characterStore.js'
6
7  export default function Profile({ user, characters, onLogout, onImport }) {
8    const [backup, setBackup] = useState(null)
9    const [message, setMessage] = useState('')
```

```
10   const [error, setError] = useState('')
11
12   async function readBackup(event) {
13     const file = event.target.files[0]
14     event.target.value = ''
15     if (!file) return
16     setError('')
17     setMessage('')
18     try {
19       if (file.size > 10 * 1024 * 1024) throw new Error('La copia no puede superar los 10 MB.')
20       setBackup(importCharacters(await file.text()))
21     } catch (failure) { setError(failure.message) }
22   }
23
24   function restore() {
25     try {
26       onImport(backup)
27       setBackup(null)
28       setError('')
29       setMessage('Copia restaurada. Tu biblioteca ya está actualizada.')
30     } catch (failure) { setBackup(null); setError(failure.message) }
31   }
32
33   return <section className="profile-panel">
34     <Icon name="user" size={36} />
35     <h2>{user ? user.name : 'Exploras como invitado'}</h2>
36     <p>{user ? user.email : 'Puedes crear y guardar personajes sin una cuenta. Cada cuenta local y el
   modo invitado tienen su propia biblioteca.'}</p>
37     <div className="profile-stats"><p><strong>{characters.length}</strong> personajes</p><p><strong>
   {characters.filter(character => character.favorite).length}</strong> favoritos</p></div>
38     <Link className="primary-button" to="/biblioteca">Abrir mi biblioteca</Link>
39     {user ? <button className="secondary-button" onClick={() => { try { onLogout() } catch { setError('No
   se pudo cerrar la sesión. Comprueba los permisos del navegador.') } }}>Cerrar sesión</button> : <><Link
   className="secondary-button" to="/login">Iniciar sesión</Link><Link className="secondary-button"
   to="/registro">Crear cuenta local</Link></>}
40     <h2>Copia de seguridad</h2>
41     <p>Exporta tus personajes, favoritos y retratos para conservarlos o llevarlos a otro navegador. Las
   cuentas y contraseñas no se incluyen.</p>
42     <button className="secondary-button" onClick={() => { try { downloadCharacters(characters);
   setMessage('Copia de seguridad preparada para descargar.'); setError('') } catch { setError('No se pudo
   preparar la descarga.') } }}>Exportar biblioteca (JSON)</button>
43     <label className="form-field">Importar copia de seguridad<input type="file"
   accept=".json,application/json" onChange={readBackup} /></label>
44     {message && <p className="feedback success" role="status">{message}</p>}
45     {error && <p className="feedback error" role="alert">{error}</p>}
46     <Link className="inline-link" to="/accesibilidad">Preferencias de accesibilidad</Link>
47     <Link className="inline-link" to="/privacidad">Privacidad y uso</Link>
48     {backup && <Modal title="¿Restaurar la biblioteca?" onClose={() => setBackup(null)}><div
   className="action-stack"><p className="page-description">La copia tiene {backup.length} personajes y
   sustituirá los {characters.length} actuales. Exporta tu biblioteca primero si quieres conservarla.</p>
   <button className="primary-button" onClick={restore}>Sustituir y restaurar</button><button
   className="secondary-button" onClick={() => setBackup(null)}>Cancelar</button></div></Modal>}
49   </section>
50 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 10 DE 50

Accessibility.jsx

`src/pages/Accessibility.jsx`

Contiene el menú principal de accesibilidad y los cinco submenús, con configuración independiente por perfil.

Entradas / props. preferences, onChange, error y user; useParams aporta el submenú solicitado.

Estado. message confirma cambios. Los valores permanentes viven en usePreferences y no en esta pantalla.

- Texto: cuatro tamaños y deslizador. Contraste: normal, alto e invertido. Lectura: avisos de pantalla, voz voluntaria y velocidad.
- Teclado: foco reforzado y tabla de atajos nativos. Apariencia: tres temas, tres tamaños de elementos, seis acentos y movimiento reducido.
- Choices utiliza radios reales dentro de fieldset y legend para que se puedan recorrer con teclado y lectores de pantalla. Preview permite ver el efecto.
- Restablecer entrega defaultPreferences al perfil activo. Un submenú desconocido muestra NotFound. El modo de contraste prevalece sobre el color del tema sin borrar la elección.

Funciones y elementos principales

Nombre	Qué hace	Línea
`labelFor`	Encuentra el texto visible correspondiente a un valor de configuración.	11
`Choices`	Construye un grupo de opciones con radios, descripciones y marca de selección.	13
`Preview`	Presenta una ficha de texto de ejemplo que hereda los ajustes actuales.	25
`Accessibility`	Elige el menú a mostrar y compone los controles de configuración.	34
`update`	Entrega cambios a onChange y prepara la confirmación para el perfil activo.	41

Código completo

114 líneas · SHA-256 del original: `4e170d4f623bc21880cc444e25c33fc1863a323af2163d87fb929dba4836a285`

```
1  import { useId, useState } from 'react'
```

```
import { Link, useParams } from 'react-router'
import Icon from '../components/icon/Icon.jsx'
import ReadAloud from '../components/feedback/ReadAloud.jsx'
import { accessibilityMenus, accentColors, contrastModes, elementSizes, textSizes, themes } from '../data/accessibility.js'
import { defaultPreferences } from '../utils/preferenceStore.js'
import { NotFound } from './Information.jsx'
import './Accessibility.css'

const sample = 'Elara Valen, hechicera humana. Una guerrera marcada por la pérdida que busca una segunda oportunidad entre las sombras. Toda historia merece ser contada.'
const labelFor = (options, value) => options.find(option => option.value === value)?.label

function Choices({ name, legend, value, options, onChange }) {
  const id = useId()
  return <fieldset className="settings-choices"><legend>{legend}</legend>
    {options.map(option => <label className={`choice-card${value === option.value ? ' is-selected' : ''}`} key={option.value}>
      <input id={`${id}-${option.value}`} type="radio" name={name} value={option.value} checked={value === option.value} onChange={() => onChange(option.value)} />
      {option.color && <span className="color-swatch" style={{ backgroundColor: option.color }} aria-hidden="true" />}
      <span><strong>{option.label}</strong>{option.description && <small>{option.description}</small>}</span>
      {value === option.value && <Icon name="check" size={18} />}
    </label>)}
  </fieldset>
}

function Preview() {
  return <section className="settings-preview" aria-label="Vista previa de los ajustes">
    <p className="eyebrow">VISTA PREVIA</p><h2>Elara Valen</h2>
    <p className="preview-tags">Humana · Hechicera</p>
    <p>Una guerrera marcada por la pérdida que busca una segunda oportunidad entre las sombras.</p>
    <p>Toda historia merece ser contada.</p>
  </section>
}

export default function Accessibility({ preferences, onChange, error, user }) {
  const { section } = useParams()
  const [message, setMessage] = useState('')
  const profile = user ? user.name : 'Invitado'
  const menu = accessibilityMenus.find(item => item.id === section)
  if (section && !menu) return <NotFound />

  function update(changes) {
    onChange(changes)
    setMessage(`Ajustes aplicados al perfil ${profile}.`)
  }

  const summaries = {
    texto: labelFor(textSizes, preferences.textSize), contraste: labelFor(contrastModes, preferences.contrast),
    lectura: preferences.readAloud ? 'Lectura en voz alta activada' : 'Lector del dispositivo',
    teclado: preferences.enhancedFocus ? 'Foco reforzado' : 'Foco estándar',
    apariencia: `${labelFor(themes, preferences.theme)} · ${labelFor(elementSizes, preferences.elementSize)}`,
  }

```

```
  return <section className="accessibility-page">
    {section && <Link className="settings-back" to="/accesibilidad"><Icon name="back" size={18} />Menú de accesibilidad</Link>}
    <div className="settings-profile"><Icon name="user" size={20} /><p>Ajustes de <strong>{profile}</strong><small>Cada perfil conserva su propia configuración.</small></p></div>
    {!section ? <>
      <p className="settings-description">Adapta Umbra a tu forma de leer y navegar. Elige una opción para abrir su menú.</p>
      <nav aria-label="Menú de accesibilidad" className="settings-menu">
        {accessibilityMenus.map(item => <Link className="settings-menu-item" key={item.id} to={`/accesibilidad/${item.id}`}>
          <span className="settings-icon"><Icon name={item.icon} size={25} /></span>
          <span><strong>{item.label}</strong><small>{item.description}</small><span className="settings-current">{summaries[item.id]}</span></span>
          <Icon name="chevron" size={18} />
        </Link>)}
      </nav>
      <button className="secondary-button" type="button" onClick={() => update(defaultPreferences)}>Restablecer ajustes de este perfil</button>
    </> : <>
      {section === 'texto' && <>
        <p className="settings-description">Aumenta o reduce el texto. El cambio se aplica en todas las pantallas, incluidas las fichas y los formularios.</p>
        <label className="text-size-slider">Tamaño de texto: {labelFor(textSizes, preferences.textSize)}<span><span aria-hidden="true">A</span><input type="range" name="textSizeSlider" min="0" max="3" step="1" value={textSizes.findIndex(item => item.value === preferences.textSize)} aria-valuetext={labelFor(textSizes, preferences.textSize)} onChange={event => update({ textSize: textSizes[Number(event.target.value)].value })} /><span aria-hidden="true">A</span></span></label>
        <Choices name="textSize" legend="Elige un tamaño" value={preferences.textSize} options={textSizes} onChange={textSize => update({ textSize })} />
        <Preview />
      </>}
      {section === 'contraste' && <>
        <p className="settings-description">Elige cómo distinguir el texto, los bordes y los controles. Las imágenes conservan sus colores.</p>
        <Choices name="contrast" legend="Modo de contraste" value={preferences.contrast} options={contrastModes} onChange={contrast => update({ contrast })} />
        <Preview />
        <p className="form-hint">El contraste normal utiliza el tema y el acento elegidos en Personalización visual.</p>
      </>}
      {section === 'lectura' && <>
        <p className="settings-description">Puedes usar el lector de pantalla de tu dispositivo, como TalkBack, VoiceOver o NVDA. Los controles tienen etiquetas, las imágenes tienen alternativas y los diálogos mantienen el foco.</p>
        <label className="settings-toggle"><input type="checkbox" name="announceNavigation" checked={preferences.announceNavigation} onChange={event => update({ announceNavigation: event.target.checked })} /><span>Anunciar cambios de pantalla<small>Añade un aviso con el nombre de la pantalla al navegar.</small></span></label>
        <label className="settings-toggle"><input type="checkbox" name="readAloud" checked={preferences.readAloud} onChange={event => update({ readAloud: event.target.checked })} /><span>Lectura en voz alta<small>Muestra un botón para escuchar el contenido de las fichas.</small></span></label>
        {preferences.readAloud && <>
          <label className="form-field">Velocidad de lectura<select name="readingRate" value={preferences.readingRate} onChange={event => update({ readingRate: Number(event.target.value) })}><option value={0.75}>Pausada</option><option value={1}>Normal</option><option value={1.25}>Rápida</option></select></label>
          <Preview /><ReadAloud text={sample} rate={preferences.readingRate} />
```

```
          <p className="form-hint">La voz depende de las voces disponibles en tu navegador o dispositivo y solo empieza al pulsar el botón.</p>
        </>}
      </>}
      {section === 'teclado' && <>
        <p className="settings-description">Recorre Umbra con el teclado. El foco indica qué control vas a activar.</p>
        <label className="settings-toggle"><input type="checkbox" name="enhancedFocus" checked={preferences.enhancedFocus} onChange={event => update({ enhancedFocus: event.target.checked })} /> <span>Reforzar el indicador de foco<small>Aumenta el grosor y el espacio del contorno al usar el teclado.</small></span></label>
        <table className="keyboard-shortcuts"><caption>Atajos principales</caption><tbody>
          <tr><th scope="row"><kbd>Tab</kbd></th><td>Siguiente control</td></tr>
          <tr><th scope="row"><kbd>Mayús</kbd> + <kbd>Tab</kbd></th><td>Control anterior</td></tr>
          <tr><th scope="row"><kbd>Enter</kbd> / <kbd>Espacio</kbd></th><td>Activar enlaces o botones; marcar opciones</td></tr>
          <tr><th scope="row"><kbd>←</kbd> <kbd>→</kbd></th><td>Cambiar de opción en los grupos; recorrer la barra inferior</td></tr>
          <tr><th scope="row"><kbd>Inicio</kbd> / <kbd>Fin</kbd></th><td>Primer o último enlace de la barra inferior</td></tr>
          <tr><th scope="row"><kbd>Esc</kbd></th><td>Cerrar un menú o diálogo</td></tr>
        </tbody></table>
        <p className="form-hint">Al empezar con Tab aparece «Saltar al contenido». Los lectores de pantalla pueden utilizar también sus propios atajos.</p>
        <Link className="secondary-button" to="/biblioteca">Practicar en la biblioteca</Link>
      </>}
      {section === 'apariencia' && <>
        <Choices name="theme" legend="Tema" value={preferences.theme} options={themes} onChange={theme => update({ theme })} />
        <Choices name="elementSize" legend="Tamaño de elementos" value={preferences.elementSize} options={elementSizes} onChange={elementSize => update({ elementSize })} />
        <Choices name="accent" legend="Color de acento" value={preferences.accent} options={accentColors} onChange={accent => update({ accent })} />
        <label className="settings-toggle"><input type="checkbox" name="reduceMotion" checked={preferences.reduceMotion} onChange={event => update({ reduceMotion: event.target.checked })} /> <span>Reducir movimiento<small>Elimina las transiciones y el movimiento de las tarjetas.</small></span></label>
        {preferences.contrast !== 'normal' && <p className="form-hint">El modo de contraste tiene prioridad sobre los colores. Tu tema y acento se conservan para cuando vuelvas al contraste normal.</p>}
        <Preview />
      </>}
    </>}
    <p className={`feedback${error ? ' error' : ''}`} role="status">{error || message || `Los ajustes de ${profile} se guardan automáticamente en este navegador.`}</p>
  </section>
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 11 DE 50

Information.jsx

`src/pages/Information.jsx`

Agrupa Inspiración, Ayuda, Privacidad y la pantalla de ruta inexistente.

Estado. Inspiration mantiene el índice de la idea narrativa y un personaje aleatorio.

- Las ideas narrativas se encuentran en el array prompts. El enlace a crear pasa un borrador mediante el estado de navegación; todavía no lo guarda.
- Ayuda utiliza details y summary para preguntas desplegables. Privacidad explica la persistencia local. NotFound permite volver a Inicio o Ayuda.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Inspiration`	Muestra una idea y una semilla de personaje para comenzar una ficha.	15
`generate`	Escoge una idea distinta de la actual y vuelve a generar el personaje.	18
`Help`	Presenta preguntas frecuentes y accesos a ayuda de accesibilidad.	32
`Privacy`	Explica qué datos se conservan y qué contiene la copia de personajes.	47
`NotFound`	Presenta el estado 404 para rutas o fichas que no se encuentran.	58

Código completo

66 líneas · SHA-256 del original: `acce321b2bc66ac25b5807fc9fda5e5c4f9a8f5b53d44cbc890cf559e482b09e`

```
1  import { useState } from 'react'
2  import { Link } from 'react-router'
3  import Icon from '../components/icon/Icon.jsx'
4  import { randomCharacter } from '../data/characters.js'
5
6  const prompts = [
7    'Tu personaje recibe una carta escrita con su propia letra, fechada cien años en el futuro. ¿Qué advertencia contiene?',
8    'Una ciudad olvida el nombre de uno de sus habitantes cada amanecer. Hoy ha desaparecido el tuyo. ¿Quién te recuerda?',
9    'La espada que heredaste se niega a herir a tu enemigo. ¿Qué secreto comparten?',
10   'Un dragón te ofrece su último deseo a cambio de un recuerdo. ¿Qué estás dispuesto a perder?',
11   'Todas las estatuas del reino han empezado a llorar. Una de ellas tiene tu rostro. ¿Qué anuncia?',
12   'Tu sombra abandona tus pasos para proteger a alguien que nunca has visto. ¿La sigues?',
13 ]
```

```
14
15 export function Inspiration() {
16   const [prompt, setPrompt] = useState(0)
17   const [character, setCharacter] = useState(randomCharacter)
18   function generate() {
19     setPrompt(index => (index + 1 + Math.floor(Math.random() * (prompts.length - 1))) % prompts.length)
20     setCharacter(randomCharacter())
21   }
22   return <section className="inspiration-panel">
23     <Icon name="sparkle" size={30} />
24     <h2>El comienzo de una leyenda</h2>
25     <p role="status">{prompts[prompt]}</p>
26     <div className="inspiration-seed"><h2>{character.name}</h2><p>{character.race} · {character.type} ·
   Nivel {character.level}</p><p>{character.traits.join(' · ')}</p></div>
27     <button className="secondary-button" onClick={generate}>Otra inspiración</button>
28     <Link className="primary-button" to="/crear-personaje" state={{ draft: { ...character, story:
   prompts[prompt] } }}>Crear a partir de esta idea</Link>
29   </section>
30 }
31
32 export function Help() {
33   return <section className="information-page">
34     <p>Umbra es un espacio para imaginar personajes, guardar sus fichas y dar vida a sus historias.</p>
35     <details><summary>¿Cómo navego por Umbra?</summary><p>Usa el menú superior para acceder a todas las
   secciones y la barra inferior para Inicio, Biblioteca, Inspiración y Perfil.</p></details>
36     <details><summary>¿Necesito iniciar sesión?</summary><p>Puedes crear y guardar personajes como
   invitado. Si creas una cuenta local, tendrás una biblioteca independiente en este navegador. Guarda el
   código de recuperación que aparece al registrarte.</p></details>
37     <details><summary>¿Dónde se guardan mis personajes?</summary><p>En el almacenamiento de este
   navegador. No se sincronizan entre dispositivos. Desde Perfil puedes exportar una copia JSON y
   restaurarla en otro navegador o cuenta. Borrar los datos del sitio elimina las cuentas y las bibliotecas
   locales.</p></details>
38     <details><summary>¿Cómo edito o elimino una ficha?</summary><p>Abre un personaje desde Inicio o
   Biblioteca. En su ficha puedes marcarlo como favorito, editarlo, duplicarlo o eliminarlo. Se pide
   confirmación antes de eliminar.</p></details>
39     <details><summary>¿Qué ocurre si pierdo la conexión?</summary><p>Se muestra un aviso y puedes seguir
   trabajando mientras la app esté abierta. Para volver a cargarla necesitas conexión al sitio o tener el
   servidor local en marcha.</p></details>
40     <details><summary>¿Cómo recupero mi contraseña?</summary><p>Abre «¿Olvidaste tu contraseña?» e
   introduce tu correo y el código que guardaste al crear la cuenta. La recuperación es local y no envía
   correos. Si borraste los datos del navegador, necesitarás crear otra cuenta e importar tu copia de
   personajes.</p></details>
41     <details><summary>¿Cómo mejoro la legibilidad?</summary><p>En Accesibilidad puedes aumentar el
   contraste y el tamaño del texto. La interfaz también respeta la preferencia del sistema de reducir el
   movimiento.</p></details>
42     <Link className="secondary-button" to="/accesibilidad">Abrir Accesibilidad</Link>
43     <Link className="inline-link" to="/inicio">Volver a Inicio</Link>
44   </section>
45 }
46
47 export function Privacy() {
48   return <section className="information-page">
49     <h2>Datos en tu navegador</h2>
50     <p>UMBRA guarda personajes, retratos, favoritos, preferencias y cuentas en el almacenamiento local
   del navegador. No envía estos datos a un servidor ni los sincroniza entre dispositivos.</p>
51     <p>Las cuentas locales permiten organizar bibliotecas independientes en este navegador. El código que
   recibes al registrarte sirve para recuperar el acceso si olvidas tu contraseña. No se envían correos y
   las copias de seguridad no incluyen las credenciales.</p>
52     <h2>Conservar tus historias</h2>
```

```
53      <p>Exporta una copia desde Perfil antes de borrar los datos del sitio o cambiar de navegador. La
    copia incluye las fichas y sus retratos, sin cuentas ni contraseñas. Usa solo imágenes que tengas permiso
    para utilizar.</p>
54      <Link className="secondary-button" to="/perfil">Ir a Perfil</Link>
55    </section>
56  }
57
58  export function NotFound() {
59    return <section className="information-page error-page">
60      <p className="error-code" aria-hidden="true">404</p>
61      <h2>Este camino no lleva a ninguna historia</h2>
62      <p>La dirección no existe o ha cambiado. Puedes volver a Inicio o consultar Ayuda para continuar.</p>
63      <Link className="primary-button" to="/inicio">Volver a Inicio</Link>
64      <Link className="secondary-button" to="/ayuda">Consultar Ayuda</Link>
65    </section>
66  }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 12 DE 50

Header.jsx

`src/components/header/Header.jsx`

Cabecera compartida con marca, menú de secciones y acceso a avisos o ayuda.

Entradas / props. onNotify abre los avisos; login adapta la cabecera del acceso; user permite ocultar Iniciar sesión cuando ya hay sesión.

Estado. menuOpen indica si el menú está abierto. Las refs identifican la cabecera y su botón.

- Cierra el menú al elegir una sección, pulsar su botón de cierre, usar Escape o pulsar fuera de la cabecera.
- aria-expanded y aria-controls describen el desplegable. El listener pointerdown se añade solo con el menú abierto y se limpia después.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Header`	Dibuja la cabecera y el menú de navegación.	6
`closeOutside`	Detecta un clic o toque fuera y cierra el menú.	13
`closeMenu`	Cierra el menú y devuelve el foco al botón que lo abre.	20

Código completo

45 líneas · SHA-256 del original: `bc63b3517248772123ab66ef9a73d28aa449b2c50fadd77cbf49182acf312fec`

```
1  import { useEffect, useRef, useState } from 'react'
2  import { Link, NavLink } from 'react-router'
3  import Icon from '../icon/Icon.jsx'
4  import { menuItems } from '../../data/navigation.js'
5
6  export default function Header({ onNotify, login = false, user }) {
7    const [menuOpen, setMenuOpen] = useState(false)
8    const header = useRef(null)
9    const menuButton = useRef(null)
10
11   useEffect(() => {
12     if (!menuOpen) return
13     function closeOutside(event) {
14       if (!header.current.contains(event.target)) setMenuOpen(false)
15     }
16     document.addEventListener('pointerdown', closeOutside)
```

```
    return () => document.removeEventListener('pointerdown', closeOutside)
  }, [menuOpen])

  function closeMenu() {
    setMenuOpen(false)
    menuButton.current.focus()
  }

  return (
    <header ref={header} className="header" onKeyDown={event => {
      if (event.key === 'Escape' && menuOpen) closeMenu()
    }}>
      {login ? <Link className="icon-button" to="/inicio" aria-label="Volver al inicio"><Icon name="back" /></Link> : <button ref={menuButton} className="icon-button" aria-label={menuOpen ? 'Cerrar menú' : 'Abrir menú'}
        aria-expanded={menuOpen} aria-controls="header-menu" onClick={() => setMenuOpen(!menuOpen)}>
        <Icon name="menu" />
      </button>}
      <Link className="brand" to="/inicio" aria-label="Umbra, inicio" onClick={() => setMenuOpen(false)}>
        <Icon name="crest" size={27} /><span>UMBRA</span>
      </Link>
      {login ? <Link className="icon-button" to="/ayuda" aria-label="Ayuda y soporte"><Icon name="help" /></Link> : <button className="icon-button notification-trigger" aria-label="Notificaciones" onClick={onNotify}>
        <Icon name="bell" />
      </button>}
      {!login && menuOpen && <nav id="header-menu" className="header-popover" aria-label="Menú de secciones">
        {menuItems.filter(item => !user || item.path !== '/login').map(item => <NavLink key={item.path} to={item.path} onClick={() => setMenuOpen(false)}>{item.label}</NavLink>)}
        <button onClick={closeMenu}>Cerrar menú</button>
      </nav>}
    </header>
  )
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 13 DE 50

BottomNavigation.jsx

`src/components/bottomnavigation/BottomNavigation.jsx`

Barra inferior para Inicio, Biblioteca, Inspiración y Perfil.

- Lee mainTabs de navigation.js. NavLink identifica la ruta activa.
- Las flechas izquierda/derecha mueven el foco entre enlaces de forma circular; Inicio y Fin van al primero y al último. Enter activa el enlace enfocado.

Funciones y elementos principales

Nombre	Qué hace	Línea
`BottomNavigation`	Construye los cuatro enlaces con icono y texto.	5
`handleKeys`	Calcula el enlace que debe recibir el foco según la tecla pulsada.	6

Código completo

24 líneas · SHA-256 del original: `2e6b222d69ace0e5eab439918671e1652cea516540324a4d8087656a898fc8ae`

```
1  import { NavLink } from 'react-router'
2  import Icon from '../icon/Icon.jsx'
3  import { mainTabs } from '../../data/navigation.js'
4
5  export default function BottomNavigation() {
6    function handleKeys(event) {
7      if (!['ArrowLeft', 'ArrowRight', 'Home', 'End'].includes(event.key)) return
8      const links = [...event.currentTarget.querySelectorAll('a')]
9      const index = links.indexOf(document.activeElement)
10     const next = event.key === 'Home' ? 0 : event.key === 'End' ? links.length - 1
11       : (index + (event.key === 'ArrowRight' ? 1 : -1) + links.length) % links.length
12     event.preventDefault()
13     links[next].focus()
14   }
15
16   return (
17     <nav className="bottom-navigation" aria-label="Navegación principal" onKeyDown={handleKeys}>
18       {mainTabs.map(tab => <NavLink key={tab.path} to={tab.path}
19         className={({ isActive }) => isActive ? 'nav-item active' : 'nav-item'}>
20         <Icon name={tab.icon} /><span className="nav-label">{tab.label}</span>
21       </NavLink>)}
22     </nav>
23   )
24 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 14 DE 50

Hero.jsx

`src/components/hero/Hero.jsx`

Portada de Inicio con paisaje, mensaje de bienvenida y acceso al editor.

- El paisaje tiene un nombre accesible. Crear personaje es un Link a /crear-personaje, de modo que no necesita guardar estado ni recibir callbacks.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Hero`	Renderiza la portada y el acceso a crear una ficha.	3

Código completo

13 líneas · SHA-256 del original: `122bddc95a161c7f53e81d9faac748ef8533d6e2a76129a0705eb8c661a610c1`

```
1  import { Link } from 'react-router'
2
3  export default function Hero() {
4    return <section className="hero" aria-labelledby="hero-title">
5      <div className="hero-landscape" role="img" aria-label="Castillo entre montañas y niebla" />
6      <div className="hero-copy">
7        <p className="welcome">Bienvenido a Umbra</p>
8        <h1 id="hero-title">Que tus historias<br />nunca se apaguen.</h1>
9        <p className="hero-description">Aquí comienza tu leyenda.</p>
10       <Link className="primary-button" to="/crear-personaje"><span aria-hidden="true">+</span> Crear
   personaje</Link>
11     </div>
12   </section>
13 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 15 DE 50

CharacterCard.jsx

`src/components/charactercard/CharacterCard.jsx`

Tarjeta pulsable de un personaje en Inicio y Biblioteca.

Entradas / props. character aporta la ficha; onSelect recibe ese personaje al pulsar.

- Es un button real con nombre accesible que incluye nombre, clase y nivel. Reutiliza Portrait y muestra una marca si la ficha es favorita.

Funciones y elementos principales

Nombre	Qué hace	Línea
`CharacterCard`	Presenta la tarjeta y comunica su selección al componente padre.	3

Código completo

8 líneas · SHA-256 del original: `1a4d5ca86bed78f929eb7e2c5813904ec0a21f6217cc00e9d31817eaceb4da98`

```
1 import Portrait from './Portrait.jsx'
2
3 export default function CharacterCard({ character, onSelect }) {
4   return <button className="character-card" onClick={() => onSelect(character)} aria-label={`Ver ficha de ${character.name}, ${character.type}, nivel ${character.level}`}>
5     <Portrait character={character} />
6     <div className="character-info"><h3>{character.name}</h3><p>{character.type} · Nivel {character.level}</p>{character.favorite && <span className="favorite-label">★ Favorito</span>}</div>
7   </button>
8 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 16 DE 50

Portrait.jsx

`src/components/charactercard/Portrait.jsx`

Presenta un retrato y ofrece iniciales si no hay imagen utilizable.

Entradas / props. character determina la imagen, el nombre y la posición; el resto de props se reenvía al elemento.

Estado. failedImage guarda la URL de una imagen que falló para no volver a mostrarla.

- Prioridad de imagen: imageDataUrl, image y finalmente el retrato local indicado por portrait. Los recursos locales disponibles son Elara y Kael.
- Si no hay imagen o su carga falla, genera hasta dos iniciales y las describe mediante aria-label. Las imágenes tienen alt con el nombre de la persona ficticia.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Portrait`	Decide entre imagen e iniciales y aplica las alternativas accesibles.	3

Código completo

26 líneas · SHA-256 del original: `fb13bc4be4580e567430468de02d95b4aa232ed266d8850e6268ac8e976f0668`

```
import { useState } from 'react'

export default function Portrait({ character, ...props }) {
  const [failedImage, setFailedImage] = useState('')
  const portraits = ['/images/elara-valen.png', '/images/kael-grim.png']
  const image = character.imageDataUrl || character.image || portraits[character.portrait]

  if (!image || failedImage === image) {
    return <div className="character-portrait portrait-placeholder" role="img" aria-label={`Iniciales de ${character.name}`} {...props}>
      <span aria-hidden="true">{character.name.split(' ').map(word => word[0]).slice(0, 2).join('')}</span>
    </div>
  }

  return (
    <img
      className="character-portrait"
      src={image}
      onError={() => setFailedImage(image)}
      alt={`Retrato de ${character.name}`}
```

```
      width="400"
      height="400"
      style={{ objectPosition: character.imagePosition || 'center' }}
      {...props}
    />
  )
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 17 DE 50

Modal.jsx

`src/components/modal/Modal.jsx`

Ventana modal reutilizable basada en el elemento nativo dialog.

Entradas / props. title, onClose y children. useId relaciona el título con aria-labelledby.

- showModal coloca el diálogo en la capa superior y activa el comportamiento modal del navegador.
- Puede cerrarse con el botón, Escape o un clic en el fondo. Al desmontarse cierra el diálogo y devuelve el foco anterior si el elemento sigue existiendo.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Modal`	Abre y presenta un diálogo accesible, ejecuta la limpieza y comunica su cierre.	4

Código completo

29 líneas · SHA-256 del original: `16f7bc8b7d3092477abecf3e47eb5926dec44bd540964f20ec9aea17eac39d94`

```
1  import { useEffect, useId, useRef } from 'react'
2  import Icon from '../icon/Icon.jsx'
3
4  export default function Modal({ title, onClose, children }) {
5    const dialog = useRef(null)
6    const titleId = useId()
7
8    useEffect(() => {
9      const element = dialog.current
10     const previousFocus = document.activeElement
11     element.showModal()
12     return () => {
13       element.close()
14       if (previousFocus?.isConnected) previousFocus.focus({ preventScroll: true })
15     }
16   }, [])
17
18   return (
19     <dialog ref={dialog} className="umbra-dialog" aria-labelledby={titleId}
20       onClose={() => { if (!dialog.current.open) onClose() }}
21       onClick={event => { if (event.target === event.currentTarget) event.currentTarget.close() }}>
22       <div className="dialog-content">
23         <button className="icon-button dialog-close" aria-label="Cerrar diálogo" onClick={() =>
   dialog.current.close()}><Icon name="close" /></button>
24         <h2 id={titleId}>{title}</h2>
```

```
25          {children}
26        </div>
27      </dialog>
28    )
29  }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 18 DE 50

AppForm.jsx

`src/components/forms/AppForm.jsx`

Motor de formularios reutilizable para personajes, registro y recuperación.

Entradas / props. fields describe los campos; onSubmit ejecuta el guardado; submitLabel, successMessage, children, cancelTo y notice personalizan el formulario.

Estado. errors, success, dirty, confirmCancel, failure y pending controlan validación, confirmaciones y envío.

- Genera inputs, selects, textareas y casillas según la definición recibida. Asocia etiquetas, ayudas y errores mediante identificadores únicos.
- La validación es propia: noValidate evita mensajes distintos entre navegadores. El primer campo inválido recibe foco después de actualizar React.
- Mientras espera el envío desactiva los campos y el botón. La cancelación confirma el descarte si hubo cambios detectados dentro del formulario.

Funciones y elementos principales

Nombre	Qué hace	Línea
`AppForm`	Genera los controles y las zonas de feedback y cancelación.	6
`handleSubmit`	Valida FormData, impide envíos repetidos, espera onSubmit y gestiona éxito o error.	16

Código completo

81 líneas · SHA-256 del original: `4a8ae6f5bc75be07c4937e00dc05883937ce1be83b04afbe677e730131fef18e`

```
1  import { useId, useState } from 'react'
2  import { useNavigate } from 'react-router'
3  import Modal from '../modal/Modal.jsx'
4  import { validateForm } from '../../utils/validateForm.js'
5
6  export default function AppForm({ fields, submitLabel, successMessage, onSubmit, children, cancelTo =
   '/inicio', notice }) {
7    const id = useId()
8    const navigate = useNavigate()
9    const [errors, setErrors] = useState({})
10   const [success, setSuccess] = useState('')
11   const [dirty, setDirty] = useState(false)
12   const [confirmCancel, setConfirmCancel] = useState(false)
```

```
  const [failure, setFailure] = useState('')
  const [pending, setPending] = useState(false)

  async function handleSubmit(event) {
    event.preventDefault()
    if (pending) return
    const form = event.currentTarget
    const nextErrors = validateForm(fields, new FormData(form))
    setErrors(nextErrors)
    setSuccess('')
    setFailure('')
    if (Object.keys(nextErrors).length) {
      requestAnimationFrame(() => form.querySelector('[aria-invalid="true"]')?.focus())
      return
    }
    setPending(true)
    try {
      await onSubmit(new FormData(form))
      setDirty(false)
      setSuccess(successMessage || '')
    } catch (error) {
      setFailure(error.message || 'No se pudo completar la operación. Vuelve a intentarlo.')
    } finally {
      setPending(false)
    }
  }

  return <>
    {notice && <p className="form-notice" id={`${id}-notice`}>{notice}</p>}
    <form className="character-form" noValidate aria-describedby={notice ? `${id}-notice` : undefined}
onSubmit={handleSubmit}
      onChange={() => { setDirty(true); setSuccess('') }}>
      <p className="form-hint">Los campos marcados con * son obligatorios.</p>
      <fieldset className="form-contents" disabled={pending}>
      {Object.keys(errors).length > 0 && <p className="feedback error" role="alert">Revisa los campos
indicados antes de continuar.</p>}
      {fields.map(field => {
        const inputId = `${id}-${field.name}`
        const props = {
          id: inputId, name: field.name, required: field.required,
          defaultValue: field.type === 'checkbox' ? undefined : field.defaultValue, minLength:
field.minLength, maxLength: field.maxLength,
          'aria-invalid': Boolean(errors[field.name]),
          'aria-describedby': [field.hint && `${inputId}-hint`, errors[field.name] && `${inputId}-
error`].filter(Boolean).join(' ') || undefined,
        }
        return <div className={field.type === 'checkbox' ? 'form-field checkbox-field' : 'form-field'}
key={field.name}>
          {field.type === 'checkbox' ? <label className="checkbox-label"><input {...props}
type="checkbox" defaultChecked={field.defaultValue === true} />{field.label}{field.required && ' *'}
</label> : <>
            <label htmlFor={inputId}>{field.label}{field.required && ' *'}</label>
            {field.options ? <select {...props}>{field.options.map(option => <option key={option}>
{option}</option>)}</select>
              : field.type === 'textarea' ? <textarea {...props} rows="4" placeholder={field.placeholder}
/>
                : <input {...props} type={field.type || 'text'} autoComplete={field.autoComplete} min=
{field.min} max={field.max} placeholder={field.placeholder} />}
          </>}
```

```
          {field.hint && <p id={`${inputId}-hint`} className="form-hint">{field.hint}</p>}
          {errors[field.name] && <p id={`${inputId}-error`} className="field-error">{errors[field.name]}
</p>}
        </div>
      })}
      {children}
      </fieldset>
      {failure && <p className="feedback error" role="alert">{failure}</p>}
      <p className={success ? 'feedback success' : 'visually-hidden'} role="status" aria-live="polite">
{success}</p>
      <button className="primary-button" type="submit" disabled={pending}>{pending ? 'Guardando…' :
submitLabel}</button>
      <button className="secondary-button" type="button" disabled={pending} onClick={() => dirty ?
setConfirmCancel(true) : navigate(cancelTo)}>Cancelar</button>
    </form>
    {confirmCancel && <Modal title="¿Descartar lo escrito?" onClose={() => setConfirmCancel(false)}>
      <p className="page-description">Los campos introducidos se perderán si sales de este formulario.
</p>
      <div className="action-stack">
        <button className="primary-button" onClick={() => setConfirmCancel(false)}>Seguir
editando</button>
        <button className="secondary-button" onClick={() => navigate(cancelTo)}>Descartar y
salir</button>
      </div>
    </Modal>}
  </>
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 19 DE 50

ConnectionStatus.jsx

`src/components/feedback/ConnectionStatus.jsx`

Aviso de pérdida y recuperación de conexión del navegador.

Estado. message conserva el aviso que se anuncia mediante una región de estado.

- Escucha los eventos offline y online y elimina ambos listeners al desmontarse.
- navigator.onLine describe el estado que comunica el navegador, no comprueba la disponibilidad de un backend. No implementa caché ni un service worker.

Funciones y elementos principales

Nombre	Qué hace	Línea
`ConnectionStatus`	Inicia y presenta el estado de conexión.	3
`offline`	Prepara el aviso de falta de conexión.	7
`online`	Prepara el aviso de conexión restablecida.	8

Código completo

18 líneas · SHA-256 del original: `661240da7ee93895c744b93593d0078c7983655d346fa477b5c26a7da245c1bf`

```
1  import { useEffect, useState } from 'react'
2
3  export default function ConnectionStatus() {
4    const [message, setMessage] = useState(() => navigator.onLine ? '' : 'Sin conexión. Puedes seguir
   trabajando mientras la app esté abierta.')
5
6    useEffect(() => {
7      const offline = () => setMessage('Sin conexión. Puedes seguir trabajando mientras la app esté
   abierta.')
8      const online = () => setMessage('Conexión restablecida.')
9      window.addEventListener('offline', offline)
10     window.addEventListener('online', online)
11     return () => {
12       window.removeEventListener('offline', offline)
13       window.removeEventListener('online', online)
14     }
15   }, [])
16
17   return <div className={message ? 'connection-status feedback' : 'visually-hidden'} role="status" aria-
   live="polite" aria-atomic="true">{message}</div>
18 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 20 DE 50

ReadAloud.jsx

`src/components/feedback/ReadAloud.jsx`

Botón para leer texto en español mediante las voces del navegador o dispositivo.

Entradas / props. text contiene lo que se leerá; rate define la velocidad, normalmente 0.75, 1 o 1.25.

Estado. reading y error controlan la interfaz; la ref current conserva la locución en curso.

- Comprueba SpeechSynthesis y SpeechSynthesisUtterance antes de ofrecer el botón. Prefiere una voz cuyo idioma empiece por es.
- La lectura comienza solo por acción de la persona. Al detenerla o desmontarse el componente cancela la locución.
- La reproducción depende de las voces disponibles y puede fallar con un mensaje. Esta función es distinta del lector de pantalla del sistema.

Funciones y elementos principales

Nombre	Qué hace	Línea
`ReadAloud`	Presenta el botón o la explicación de que no hay soporte.	4
`toggle`	Inicia una locución con el texto y velocidad actuales o detiene la que se está leyendo.	18

Código completo

48 líneas · SHA-256 del original: `6b8db696e2080055c48acb39dbaf6904f444089ce5c135a8a09fb4c9c3414069`

```
1  import { useEffect, useRef, useState } from 'react'
2  import Icon from '../icon/Icon.jsx'
3
4  export default function ReadAloud({ text, rate = 1 }) {
5    const supported = 'speechSynthesis' in window && 'SpeechSynthesisUtterance' in window
6    const current = useRef(null)
7    const [reading, setReading] = useState(false)
8    const [error, setError] = useState('')
9
10   useEffect(() => () => {
11     if (current.current) {
12       current.current.onend = null
13       current.current.onerror = null
14       window.speechSynthesis.cancel()
15     }
16   }, [])
```

```
17
18    function toggle() {
19      if (reading) {
20        window.speechSynthesis.cancel()
21        current.current = null
22        setReading(false)
23        return
24      }
25      setError('')
26      window.speechSynthesis.cancel()
27      const speech = new SpeechSynthesisUtterance(text)
28      speech.lang = 'es-ES'
29      speech.rate = rate
30      const voice = window.speechSynthesis.getVoices().find(item => item.lang.startsWith('es'))
31      if (voice) speech.voice = voice
32      speech.onend = () => { current.current = null; setReading(false) }
33      speech.onerror = event => {
34        current.current = null
35        setReading(false)
36        if (!['canceled', 'interrupted'].includes(event.error)) setError('No se pudo reproducir la voz.
    Comprueba que el dispositivo tenga una voz en español disponible.')
37      }
38      current.current = speech
39      setReading(true)
40      window.speechSynthesis.speak(speech)
41    }
42
43    if (!supported) return <p className="form-hint">Este navegador no ofrece lectura en voz alta. Puedes
    utilizar el lector de pantalla de tu dispositivo.</p>
44    return <div className="read-aloud">
45      <button className="secondary-button" type="button" onClick={toggle} aria-pressed={reading}><Icon
    name="speaker" />{reading ? 'Detener lectura' : 'Leer en voz alta'}</button>
46      {error && <p className="feedback error" role="alert">{error}</p>}
47    </div>
48  }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 21 DE 50

Icon.jsx

`src/components/icon/Icon.jsx`

Catálogo de iconos SVG reutilizables sin archivos externos.

Entradas / props. name elige el dibujo y size fija el tamaño; por defecto es 22.

- paths contiene los trazados de cada icono. aria-hidden evita que un dibujo decorativo repita el nombre que ya aporta su botón o enlace.

Funciones y elementos principales

Nombre	Qué hace	Línea
`Icon`	Genera un svg con el trazado, tamaño y estilo comunes.	33

Código completo

35 líneas · SHA-256 del original: `5de85065d74d5b71fce2b4886147b91b6860d5666e8c4cbf7f682cc48e63de1e`

```
const paths = {
  mail: <><rect x="3" y="5" width="18" height="14" rx="2" /><path d="m3 6 9 7 9-7" /></>,
  lock: <><rect x="5" y="10" width="14" height="11" rx="2" /><path d="M8 10V7a4 4 0 0 1 8 0v3M12 14v3" /></>,
  'eye-off': <><path d="m3 3 18 18M10.5 5.1 12 5c6 0 10 7 10 7a20 20 0 0 1-3 3.5M6 6.5A22 22 0 0 0 2 12s4 7 10 7a11 11 0 0 0 5-1.2M10 10a3 3 0 0 0 4 4" /></>,
  login: <><path d="M14 3h6v18h-6M3 12h13m-5-5 5 5-5 5" /></>,
  'user-plus': <><circle cx="9" cy="7" r="4" /><path d="M2 22v-3a7 7 0 0 1 14 0v3M19 8v8M15 12h8" /></>,
  sigil: <><path d="m12 2 10 5-2.5 11-7.5 4-7.5-4L2 7ZM5 7l7 12 7-12M12 2v10M8 11h8M9.5 15h5" /></>,
  close: <path d="m6 6 12 12M6 18 18 6" />,
  back: <path d="m14 5-7 7 7 7" />,
  chevron: <path d="m9 5 7 7-7 7" />,
  search: <><circle cx="10" cy="10" r="6" /><path d="m15 15 6 6" /></>,
  edit: <><path d="m15 4 5 5M4 20l5-1L21 7l-5-5L4 14Z" /></>,
  trash: <><path d="M3 6h18M9 6V3h6v3M6 6l1 15h10l1-15M10 10v7M14 10v7" /></>,
  bookmark: <path d="M6 3h12v19l-6-4-6 4Z" />,
  help: <><circle cx="12" cy="12" r="9" /><path d="M9 9a3 3 0 1 1 4 3c-1 0-1 1-1 2M12 17h.01" /></>,
  logout: <><path d="M9 3H4v18h5M10 12h12m-5-5 5 5-5 5" /></>,
  palette: <><path d="M12 3a9 9 0 0 1 0 0 18h1a2 2 0 0 0 0-4h-1a2 2 0 0 1 0-4h4a5 5 0 0 0 5-5c0-3-5-5-9-5Z" /><path d="M7 9h.01M11 6h.01M6 14h.01M16 7h.01" /></>,
  contrast: <><circle cx="12" cy="12" r="9" /><path d="M12 3v18" /><path d="M12 3a9 9 0 0 0 1 0 18Z" fill="currentColor" /></>,
  type: <path d="m3 20 7-16 7 16M6 14h8M17 12h6M20 12v8" />,
  speaker: <><path d="m11 4-6 5H2v6h3l6 5ZM15 8a6 6 0 0 1 0 8M18 5a10 10 0 0 1 0 14" /></>,
  keyboard: <><rect x="2" y="5" width="20" height="14" rx="2" /><path d="M6 9h1m3 0h1m3 0h1m3 0h1M6 12h1m3 0h1m3 0h1M7 16h10" /></>,
  check: <path d="m4 12 5 5L20 6" />,
  eye: <><path d="M2 12s4-7 10-7 10 7 10 7-4 7-10 7S2 12 2 12Z" /><circle cx="12" cy="12" r="3" /></>,
```

```
  menu: <><path d="M5 6h14M5 12h14M5 18h14" /></>,
  bell: <><path d="M18 8a6 6 0 0 0-12 0c0 7-3 7-3 9h18c0-2-3-2-3-9M10 21h4" /></>,
  home: <><path d="m3 10 9-7 9 7v11h-6v-7H9v7H3Z" /></>,
  book: <><path d="M12 5v16M3 3l9 2 9-2v16l-9 2-9-2Z" /></>,
  sparkle: <><path d="m12 3 2.5 6.5L21 12l-6.5 2.5L12 21l-2.5-6.5L3 12l6.5-2.5ZM20 2v4M18 4h4" /></>,
  user: <><circle cx="12" cy="7" r="4" /><path d="M4 22v-3a8 8 0 0 1 16 0v3" /></>,
  crest: <><path d="m12 2 8 5-2 10-6 5-6-5L4 7ZM12 2v20M8 6l1 8 3 3 3-3 1-8" /></>,
}

export default function Icon({ name, size = 22 }) {
  return <svg width={size} height={size} viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="1.3" strokeLinecap="round" strokeLinejoin="round" aria-hidden="true">{paths[name]}</svg>
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 22 DE 50

useCharacters.js

`src/hooks/useCharacters.js`

Mantiene la biblioteca React del perfil y coordina las escrituras persistentes.

Entradas / props. owner es el identificador estable de la cuenta o guest.

Estado. loaded contiene el resultado inicial; characters es la biblioteca visible; storageError informa de fallos.

- Carga una vez al montarse. App vuelve a montar Workspace cuando cambia el perfil.
- Actualiza el estado visible solo después de guardar correctamente. Si el almacenamiento falla conserva la biblioteca anterior y lanza un error para el formulario o diálogo.

Funciones y elementos principales

Nombre	Qué hace	Línea
`useCharacters`	Devuelve personajes, errores y operaciones de biblioteca.	4
`commit`	Valida la nueva lista, la escribe y actualiza el estado; transforma fallos de almacenamiento en un mensaje.	12
`saveCharacter`	Normaliza una ficha, crea identificador cuando hace falta y actualiza updatedAt. Sustituye por id o añade al inicio.	25
`deleteCharacter`	Filtra la ficha con el id indicado y guarda la nueva lista.	34
`toggleFavorite`	Invierte favorite en la ficha indicada y guarda los datos.	35
`replaceCharacters`	Expone commit para sustituir la biblioteca al restaurar una copia.	36

Código completo

38 líneas · SHA-256 del original: `a885b8a5c772dc0c583fbb3ceaf109b45df7b3b28ec9c0074243925f6b04aa14`

```
1  import { useState } from 'react'
2  import { normalizeCharacter, parseCharacters, readCharacters, writeCharacters } from
   '../utils/characterStore.js'
3
4  export function useCharacters(owner) {
5    const [loaded] = useState(() => {
6      try { return { characters: readCharacters(owner), error: '' } }
```

```
    catch { return { characters: [], error: 'No se pudo leer la biblioteca guardada. Puedes importar una copia desde Perfil. Comprueba que el navegador permita el almacenamiento local.' } }
  })
  const [characters, setCharacters] = useState(loaded.characters)
  const [storageError, setStorageError] = useState(loaded.error)

  function commit(next) {
    parseCharacters(next)
    try {
      const checked = writeCharacters(owner, next)
      setCharacters(checked)
      setStorageError('')
    } catch {
      const message = 'No se pudo guardar. El almacenamiento puede estar lleno o bloqueado. Exporta una copia desde Perfil y prueba con un retrato más pequeño.'
      setStorageError(message)
      throw new Error(message)
    }
  }

  function saveCharacter(draft) {
    const character = normalizeCharacter({ ...draft, id: draft.id || crypto.randomUUID(), updatedAt: Date.now() })
    const exists = characters.some(item => item.id === character.id)
    commit(exists ? characters.map(item => item.id === character.id ? character : item) : [character, ...characters])
    return character
  }

  return {
    characters, storageError, saveCharacter,
    deleteCharacter: id => commit(characters.filter(character => character.id !== id)),
    toggleFavorite: id => commit(characters.map(character => character.id === id ? { ...character, favorite: !character.favorite } : character)),
    replaceCharacters: commit,
  }
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 23 DE 50

usePreferences.js

`src/hooks/usePreferences.js`

Carga y aplica los ajustes de accesibilidad del perfil activo.

Entradas / props. owner es el identificador del perfil o guest.

Estado. preferences conserva la configuración; preferenceError informa de lectura o guardado fallidos.

- Si no puede leer los ajustes utiliza los valores predeterminados y muestra un aviso.
- A diferencia de los personajes, un ajuste de accesibilidad se aplica visualmente incluso si no se puede persistir. Se avisa de que solo durará mientras la app siga abierta.

Funciones y elementos principales

Nombre	Qué hace	Línea
`usePreferences`	Recupera ajustes y devuelve su valor, su error y la operación de cambio.	4
`updatePreferences`	Combina cambios parciales, normaliza, aplica al estado y trata de guardar para owner.	12

Código completo

24 líneas · SHA-256 del original: `473006af0a4bb3363458de9ab2d2b34b1a6357d3fcf7c297af1ddc6caa3a5208`

```
1  import { useState } from 'react'
2  import { defaultPreferences, normalizePreferences, readPreferences, writePreferences } from
   '../utils/preferenceStore.js'
3
4  export function usePreferences(owner) {
5    const [loaded] = useState(() => {
6      try { return { preferences: readPreferences(owner), error: '' } }
7      catch { return { preferences: { ...defaultPreferences }, error: 'No se pudieron recuperar los ajustes
   de este perfil. Puedes volver a elegirlos.' } }
8    })
9    const [preferences, setPreferences] = useState(loaded.preferences)
10   const [preferenceError, setPreferenceError] = useState(loaded.error)
11
12   function updatePreferences(changes) {
13     const updated = normalizePreferences({ ...preferences, ...changes })
14     setPreferences(updated)
15     try {
16       writePreferences(owner, updated)
17       setPreferenceError('')
18     } catch {
```

```
19        setPreferenceError('El navegador no permite guardar tus preferencias. Se aplicarán mientras la app esté abierta.')
20      }
21    }
22
23    return { preferences, updatePreferences, preferenceError }
24  }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 24 DE 50

characterStore.js

`src/utils/characterStore.js`

Valida fichas, lee y escribe bibliotecas y prepara las copias JSON.

- MAX_IMAGE_BYTES fija 750 KB para la carga de retratos; imagePattern limita el formato de data URL a PNG, JPEG o WebP.
- readCharacters inicia una biblioteca nueva con seis ejemplos si la clave no existe. Una lista guardada vacía se conserva vacía.
- La importación acepta hasta 500 fichas y una versión de copia reconocida; la pantalla Perfil limita también el archivo a 10 MB.

Funciones y elementos principales

Nombre	Qué hace	Línea
`normalizeCharacter`	Construye una ficha con las propiedades admitidas; comprueba nombre, clase, raza, nivel, atributos, rasgos e imagen. Añade un id nuevo si no recibe uno utilizable.	7
`text`	Ayudante interno que recorta y limita campos de texto.	9
`parseCharacters`	Comprueba que recibe una lista, valida cada ficha y rechaza identificadores repetidos.	35
`readCharacters`	Lee la clave del propietario y devuelve su biblioteca o los ejemplos iniciales.	42
`writeCharacters`	Valida la lista y la serializa en localStorage para el propietario.	47
`importCharacters`	Analiza el JSON, verifica app y version y devuelve las fichas validadas.	53
`downloadCharacters`	Serializa app, version y characters, crea un Blob y un enlace temporal y dispara la descarga; después libera la URL.	60

Código completo

68 líneas · SHA-256 del original: `ccf59fe9fab148b04558021ebbf4e148e02f272ec2b83bdaa779840457b1b510`

```
1  import { initialCharacters } from '../data/characters.js'
2  import { characterRaces, characterTypes } from '../data/characterOptions.js'
```

```
3
4  export const MAX_IMAGE_BYTES = 750 * 1024
5  const imagePattern = /^data:image\/(png|jpeg|webp)base64,[A-Za-z0-9+/]+=*$/
6
7  export function normalizeCharacter(value) {
8    if (!value || typeof value !== 'object') throw new Error('La ficha no es válida.')
9    const text = (key, max, required = false) => {
10     const result = typeof value[key] === 'string' ? value[key].trim() : ''
11     const label = { name: 'nombre', realm: 'origen', story: 'historia', imageDataUrl: 'retrato',
   imageName: 'nombre de imagen' }[key]
12     if ((required && result.length < 2) || result.length > max) throw new Error(`Revisa el campo ${label}
   de la ficha.`)
13     return result
14   }
15   const name = text('name', 60, true)
16   if (!characterTypes.includes(value.type) || !characterRaces.includes(value.race)) throw new Error('La
   clase o la raza no son válidas.')
17   const level = Number(value.level)
18   if (!Number.isInteger(level) || level < 1 || level > 20) throw new Error('El nivel debe estar entre 1 y
   20.')
19   if (!Array.isArray(value.stats) || value.stats.length !== 4 || value.stats.some(stat =>
   !Number.isInteger(stat) || stat < 1 || stat > 20)) throw new Error('Los cuatro atributos deben estar
   entre 1 y 20.')
20   if (!Array.isArray(value.traits) || value.traits.length > 6 || value.traits.some(trait => typeof trait
   !== 'string' || trait.length > 40)) throw new Error('Puedes elegir hasta seis rasgos.')
21   const imageDataUrl = text('imageDataUrl', MAX_IMAGE_BYTES * 1.4)
22   if (imageDataUrl && !imagePattern.test(imageDataUrl)) throw new Error('El retrato debe ser una imagen
   PNG, JPG o WebP.')
23   return {
24     id: typeof value.id === 'string' && /^[\w-]{1,80}$/.test(value.id) ? value.id : crypto.randomUUID(),
25     name, type: value.type, race: value.race, level,
26     realm: text('realm', 100), story: text('story', 3000),
27     portrait: Number.isInteger(value.portrait) && value.portrait >= 0 && value.portrait <= 5 ?
   value.portrait : 0,
28     imageDataUrl, imageName: text('imageName', 255),
29     traits: [...new Set(value.traits)], stats: [...value.stats],
30     favorite: value.favorite === true,
31     updatedAt: Number.isFinite(value.updatedAt) ? value.updatedAt : 0,
32   }
33 }
34
35 export function parseCharacters(values) {
36   if (!Array.isArray(values) || values.length > 500) throw new Error('La copia debe contener una lista de
   hasta 500 personajes.')
37   const characters = values.map(normalizeCharacter)
38   if (new Set(characters.map(character => character.id)).size !== characters.length) throw new Error('La
   copia contiene identificadores repetidos.')
39   return characters
40 }
41
42 export function readCharacters(owner, storage = localStorage) {
43   const saved = storage.getItem(`umbra.characters.v1.${owner}`)
44   return saved === null ? initialCharacters.map(normalizeCharacter) : parseCharacters(JSON.parse(saved))
45 }
46
47 export function writeCharacters(owner, characters, storage = localStorage) {
48   const checked = parseCharacters(characters)
49   storage.setItem(`umbra.characters.v1.${owner}`, JSON.stringify(checked))
50   return checked
```

```
}

export function importCharacters(contents) {
  let backup
  try { backup = JSON.parse(contents) } catch { throw new Error('El archivo no contiene JSON válido.') }
  if (backup?.app !== 'UMBRA' || backup.version !== 1) throw new Error('Selecciona una copia de seguridad de UMBRA compatible.')
  return parseCharacters(backup.characters)
}

export function downloadCharacters(characters) {
  const blob = new Blob([JSON.stringify({ app: 'UMBRA', version: 1, characters }, null, 2)], { type: 'application/json' })
  const url = URL.createObjectURL(blob)
  const link = document.createElement('a')
  link.href = url
  link.download = `umbra-personajes-${new Date().toISOString().slice(0, 10)}.json`
  link.click()
  setTimeout(() => URL.revokeObjectURL(url), 1000)
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 25 DE 50

localAccounts.js

`src/utils/localAccounts.js`

Implementa cuentas y sesiones locales mediante Web Crypto y almacenamiento del navegador.

- La cuenta guarda id, nombre, correo normalizado, sal y verificadores. No persiste la contraseña ni el código de recuperación en texto plano.
- PBKDF2-SHA-256 utiliza una sal aleatoria de 16 bytes, 100000 iteraciones y resultado de 256 bits. El código aleatorio de recuperación usa 16 bytes y se verifica con SHA-256.
- Con Recordarme el id de sesión queda en localStorage; sin marcarlo queda en sessionStorage. No son sesiones autenticadas por un servidor.
- No existe una cuenta semilla, rol ADMIN, token de backend, envío de correo ni control de permisos de servidor.

Funciones y elementos principales

Nombre	Qué hace	Línea
`bytesToHex`	Convierte un array de bytes en una cadena hexadecimal.	4
`normalizeEmail`	Elimina espacios en los extremos y convierte el correo a minúsculas.	5
`passwordHash`	Deriva el verificador de contraseña con PBKDF2 y la sal recibida.	7
`recoveryHash`	Calcula SHA-256 del código recortado y convertido a mayúsculas.	13
`accounts`	Lee y comprueba el array de cuentas locales.	17
`readSession`	Busca el id de sesión temporal o persistente y devuelve solo id, nombre y correo.	23
`registerAccount`	Rechaza correo duplicado, genera identificadores y verificadores, guarda la cuenta y devuelve el código de recuperación.	29
`loginAccount`	Verifica correo y contraseña, escribe la sesión en el almacén elegido y devuelve la identidad.	40
`logoutAccount`	Borra la referencia de sesión en los dos tipos de almacenamiento.	49

Nombre	Qué hace	Línea
`recoverAccount`	Verifica correo y código, cambia sal y contraseña y retira las referencias de sesión de esa cuenta.	54

Código completo

63 líneas · SHA-256 del original: `097f5b62a324833b5fa35bd1da92483cda3b00eb7fff68c072c2cce9ea0533cd`

```
1  const accountsKey = 'umbra.accounts.v1'
2  const sessionKey = 'umbra.session.v1'
3  const iterations = 100000
4  const bytesToHex = bytes => Array.from(bytes, byte => byte.toString(16).padStart(2, '0')).join('')
5  const normalizeEmail = email => email.trim().toLocaleLowerCase('es')
6
7  async function passwordHash(password, salt) {
8    const key = await crypto.subtle.importKey('raw', new TextEncoder().encode(password), 'PBKDF2', false,
   ['deriveBits'])
9    const bits = await crypto.subtle.deriveBits({ name: 'PBKDF2', salt: new TextEncoder().encode(salt),
   iterations, hash: 'SHA-256' }, key, 256)
10   return bytesToHex(new Uint8Array(bits))
11 }
12
13 async function recoveryHash(code) {
14   return bytesToHex(new Uint8Array(await crypto.subtle.digest('SHA-256', new
   TextEncoder().encode(code.trim().toUpperCase()))))
15 }
16
17 function accounts(storage) {
18   const saved = JSON.parse(storage.getItem(accountsKey) || '[]')
19   if (!Array.isArray(saved)) throw new Error('No se pueden leer las cuentas de este navegador.')
20   return saved
21 }
22
23 export function readSession(storage = localStorage, temporary = sessionStorage) {
24   const id = temporary.getItem(sessionKey) || storage.getItem(sessionKey)
25   const account = accounts(storage).find(item => item.id === id)
26   return account ? { id: account.id, name: account.name, email: account.email } : null
27 }
28
29 export async function registerAccount({ name, email, password }, storage = localStorage) {
30   const list = accounts(storage)
31   const normalizedEmail = normalizeEmail(email)
32   if (list.some(account => account.email === normalizedEmail)) throw new Error('Ya existe una cuenta con
   este correo en este navegador.')
33   const salt = bytesToHex(crypto.getRandomValues(new Uint8Array(16)))
34   const recoveryCode = bytesToHex(crypto.getRandomValues(new Uint8Array(16))).toUpperCase()
35   const account = { id: crypto.randomUUID(), name: name.trim(), email: normalizedEmail, salt,
   passwordHash: await passwordHash(password, salt), recoveryHash: await recoveryHash(recoveryCode) }
36   storage.setItem(accountsKey, JSON.stringify([...list, account]))
37   return recoveryCode
38 }
39
40 export async function loginAccount({ email, password, remember }, storage = localStorage, temporary =
   sessionStorage) {
```

```
41    const account = accounts(storage).find(item => item.email === normalizeEmail(email))
42    if (!account || await passwordHash(password, account.salt) !== account.passwordHash) throw new
    Error('El correo o la contraseña no coinciden con una cuenta de este navegador.')
43    const target = remember ? storage : temporary
44    target.setItem(sessionKey, account.id)
45    ;(remember ? temporary : storage).removeItem(sessionKey)
46    return { id: account.id, name: account.name, email: account.email }
47  }
48
49  export function logoutAccount(storage = localStorage, temporary = sessionStorage) {
50    storage.removeItem(sessionKey)
51    temporary.removeItem(sessionKey)
52  }
53
54  export async function recoverAccount({ email, recoveryCode, password }, storage = localStorage, temporary
    = sessionStorage) {
55    const list = accounts(storage)
56    const account = list.find(item => item.email === normalizeEmail(email))
57    if (!account || await recoveryHash(recoveryCode) !== account.recoveryHash) throw new Error('El correo o
    el código de recuperación no son correctos.')
58    const salt = bytesToHex(crypto.getRandomValues(new Uint8Array(16)))
59    const updated = { ...account, salt, passwordHash: await passwordHash(password, salt) }
60    storage.setItem(accountsKey, JSON.stringify(list.map(item => item.id === account.id ? updated : item)))
61    if (storage.getItem(sessionKey) === account.id) storage.removeItem(sessionKey)
62    if (temporary.getItem(sessionKey) === account.id) temporary.removeItem(sessionKey)
63  }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 26 DE 50

preferenceStore.js

`src/utils/preferenceStore.js`

Define valores predeterminados y persistencia de accesibilidad separada por perfil.

- defaultPreferences usa texto normal, contraste normal, tema oscuro, controles normales, acento rojo y anuncios de navegación. La voz, el foco reforzado y el movimiento reducido empiezan desactivados.
- La antigua clave global v1 solo se recupera para guest. Las cuentas nuevas empiezan con sus propios valores predeterminados.
- La normalización admite únicamente tamaños, contrastes, temas, acentos y velocidades conocidos.

Funciones y elementos principales

Nombre	Qué hace	Línea
`normalizePreferences`	Construye una configuración válida y sustituye opciones desconocidas por sus valores predeterminados.	8
`choose`	Ayudante interno que acepta un valor solo si aparece en el catálogo de opciones.	10
`readPreferences`	Lee la clave v2 del propietario, considera la migración de Invitado o devuelve defaults.	21
`writePreferences`	Normaliza y guarda la configuración del propietario.	32

Código completo

36 líneas · SHA-256 del original: `b8f0d16a392951b7a68d78af14e91fdd1f28ca3d40889207ca74ebcb701c70e9`

```
1  import { accentColors, contrastModes, elementSizes, textSizes, themes } from '../data/accessibility.js'
2
3  export const defaultPreferences = Object.freeze({
4    textSize: 'normal', contrast: 'normal', theme: 'dark', elementSize: 'normal', accent: 'red',
5    readAloud: false, readingRate: 1, announceNavigation: true, enhancedFocus: false, reduceMotion: false,
6  })
7
8  export function normalizePreferences(saved) {
9    const source = saved && typeof saved === 'object' ? saved : {}
10   const choose = (key, options) => options.some(option => option.value === source[key]) ? source[key] :
   defaultPreferences[key]
11   return {
12     textSize: choose('textSize', textSizes), contrast: choose('contrast', contrastModes),
```

```
    theme: choose('theme', themes), elementSize: choose('elementSize', elementSizes), accent: choose('accent', accentColors),
    readAloud: source.readAloud === true,
    readingRate: [0.75, 1, 1.25].includes(source.readingRate) ? source.readingRate : 1,
    announceNavigation: source.announceNavigation !== false,
    enhancedFocus: source.enhancedFocus === true, reduceMotion: source.reduceMotion === true,
  }
}

export function readPreferences(owner, storage = localStorage) {
  const saved = storage.getItem(`umbra.accessibility.v2.${owner}`)
  if (saved !== null) return normalizePreferences(JSON.parse(saved))
  // The previous app-wide settings belong only to the guest. Accounts start independently.
  if (owner === 'guest') {
    const legacy = JSON.parse(storage.getItem('umbra.accessibility.v1') || 'null')
    if (legacy) return normalizePreferences({ textSize: legacy.largeText ? 'large' : 'normal', contrast: legacy.highContrast ? 'high' : 'normal' })
  }
  return { ...defaultPreferences }
}

export function writePreferences(owner, preferences, storage = localStorage) {
  const checked = normalizePreferences(preferences)
  storage.setItem(`umbra.accessibility.v2.${owner}`, JSON.stringify(checked))
  return checked
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 27 DE 50

validateForm.js

`src/utils/validateForm.js`

Validación declarativa compartida por AppForm y Login.

- Recibe la definición de campos y un FormData. Devuelve un objeto de errores, con el nombre del campo como clave.
- Comprueba obligatoriedad, formato de correo, longitudes, opciones permitidas, carácter especial, enteros en rango y coincidencia de contraseñas.
- Se detiene en el primer error de cada campo. Las comprobaciones del navegador y de la interfaz no sustituyen la validación de un futuro backend.

Funciones y elementos principales

Nombre	Qué hace	Línea
`validateForm`	Recorre las reglas y devuelve mensajes listos para asociarse a los controles.	1

Código completo

25 líneas · SHA-256 del original: `97ab9fdf59e9edaeff458ef55a36ad19a9328d9cd8a61051f1cc0bcc9740a4c4`

```
export function validateForm(fields, values) {
  const errors = {}
  for (const field of fields) {
    const raw = values.get(field.name)
    const value = typeof raw === 'string' ? raw : ''
    if (field.required && !value.trim()) {
      errors[field.name] = field.type === 'checkbox' ? 'Debes marcar esta casilla para continuar.' : `Completa el campo «${field.label}».`
    } else if (value && field.type === 'email' && !/^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(value)) {
      errors[field.name] = 'Escribe un correo válido, por ejemplo nombre@dominio.es.'
    } else if (value && field.minLength && value.trim().length < field.minLength) {
      errors[field.name] = `Introduce al menos ${field.minLength} caracteres.`
    } else if (value && field.maxLength && value.length > field.maxLength) {
      errors[field.name] = `Introduce como máximo ${field.maxLength} caracteres.`
    } else if (value && field.options && !field.options.includes(value)) {
      errors[field.name] = 'Selecciona una opción de la lista.'
    } else if (value && field.requireSpecial && !/[^\p{L}\p{N}\s]/u.test(value)) {
      errors[field.name] = 'Incluye al menos un carácter especial, como !, @, # o $.'
    } else if (value && field.type === 'number' && (!Number.isInteger(Number(value)) || Number(value) < field.min || Number(value) > field.max)) {
      errors[field.name] = `Introduce un número entero entre ${field.min} y ${field.max}.`
    } else if (field.matches && value !== values.get(field.matches)) {
      errors[field.name] = 'Las contraseñas deben coincidir.'
```

```
    }
  }
  return errors
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 28 DE 50

characters.js

`src/data/characters.js`

Seis fichas iniciales y generador de semillas de personaje.

- initialCharacters incluye Elara, Kael, Seraphine, Draeg, Inquisidor y Angélica. Los arrays stats siguen el orden de attributeNames.
- El generador reutiliza una plantilla y cambia nombre, id de borrador y atributos; no escribe nada en el almacenamiento.

Funciones y elementos principales

Nombre	Qué hace	Línea
`randomCharacter`	Escoge plantilla y nombre aleatorios; genera cuatro atributos enteros entre 6 y 20.	40

Código completo

49 líneas · SHA-256 del original: `555900b706070b7ea3c05c190dcfdde891c2297e3771940afc53b6e18531188a`

```
export const initialCharacters = [
  {
    id: 'elara', name: 'Elara Valen', type: 'Hechicera', race: 'Humana', level: 8,
    realm: 'Reino de Aethoria', portrait: 0,
    story: 'Nacida en las ruinas de Velmora, Elara teje la magia de aquello que otros temen. Busca una verdad enterrada entre las sombras, aunque el precio sea su propia alma.',
    traits: ['Reservada', 'Leal', 'Determinada'], stats: [8, 14, 18, 12],
  },
  {
    id: 'kael', name: 'Kael Grim', type: 'Pícaro', race: 'Humano', level: 6,
    realm: 'Ciudad de Nox', portrait: 1,
    story: 'Entre los callejones de Nox, Kael aprendió que todo secreto tiene un precio. Ahora un antiguo juramento lo obliga a proteger aquello que una vez prometió robar.',
    traits: ['Astuto', 'Independiente', 'Irónico'], stats: [12, 19, 14, 15],
  },
  {
    id: 'seraphine', name: 'Seraphine', type: 'Hechicera', race: 'Elfa', level: 10,
    realm: 'Bosque de Lúmina', portrait: 2,
    story: 'La última guardiana de un bosque sin amanecer escucha las voces de los árboles. Una melodía olvidada podría devolver la luz a su pueblo.',
    traits: ['Serena', 'Sabia', 'Protectora'], stats: [7, 15, 20, 16],
  },
  {
    id: 'draeg', name: 'Draeg', type: 'Guerrero', race: 'Humano', level: 12,
    realm: 'Tierras del Norte', portrait: 3,
```

```
    story: 'Su armadura recuerda cada batalla, pero Draeg solo recuerda a quienes no pudo salvar. Camina hacia el norte buscando una paz que ninguna victoria le ha dado.',
    traits: ['Valiente', 'Leal', 'Obstinado'], stats: [20, 12, 10, 13],
  },
  {
    id: 'inquisidor', name: 'Inquisidor', type: 'Guerrero', race: 'Humano', level: 9,
    realm: 'Orden del Eclipse', portrait: 4,
    story: 'Tras la máscara del inquisidor se esconde una duda. Cuando la orden exige quemar una biblioteca, deberá elegir entre su deber y la memoria del mundo.',
    traits: ['Metódico', 'Severo', 'Cauto'], stats: [17, 11, 16, 10],
  },
  {
    id: 'angelica', name: 'Angélica Mangus', type: 'Hechicera', race: 'Celestial', level: 15,
    realm: 'Reino del Cielo', portrait: 5,
    story: 'Nacida de una pluma del arcángel Miguel, con carácter fuerte, valentía y poca paciencia. Descendió para encontrar un nombre perdido entre los mortales.',
    traits: ['Extrovertida', 'Dominante', 'Audaz'], stats: [8, 20, 18, 19],
  },
]

export function randomCharacter() {
  const template = initialCharacters[Math.floor(Math.random() * initialCharacters.length)]
  const names = ['Lyra del Alba', 'Aren Noctis', 'Nerea de Ceniza', 'Valen del Umbral', 'Angélica Mangus', 'Eira Sombraluna']
  return {
    ...template,
    id: `draft-${Date.now()}`,
    name: names[Math.floor(Math.random() * names.length)],
    stats: Array.from({ length: 4 }, () => 6 + Math.floor(Math.random() * 15)),
  }
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 29 DE 50

characterOptions.js

`src/data/characterOptions.js`

Catálogos de clases, razas, rasgos y nombres de atributos, además de la ficha vacía.

- Fuerza, Destreza, Inteligencia y Carisma determinan el orden del array stats en toda la app.
- blankCharacter prepara los campos iniciales del editor, con nivel 1 y cuatro atributos de valor 10.

Funciones y elementos principales

Nombre	Qué hace	Línea
`blankCharacter`	Devuelve un objeto nuevo de personaje todavía sin nombre ni identificador persistente.	6

Código completo

19 líneas · SHA-256 del original: `638e78935a9815f6be3f8f4f941d8f40c874ae40fff4fdecef8bdc0d1516c7e3`

```
1  export const characterTypes = ['Hechicera', 'Hechicero', 'Guerrera', 'Guerrero', 'Pícara', 'Pícaro',
   'Exploradora', 'Explorador', 'Sanadora', 'Sanador', 'Paladín', 'Bardo']
2  export const characterRaces = ['Humana', 'Humano', 'Elfa', 'Elfo', 'Enana', 'Enano', 'Orca', 'Orco',
   'Celestial', 'Tiefling', 'Otra']
3  export const personalityTraits = ['Leal', 'Valiente', 'Reservada', 'Determinada', 'Astuto',
   'Independiente', 'Serena', 'Sabia', 'Protectora', 'Irónico', 'Audaz', 'Cauto']
4  export const attributeNames = ['Fuerza', 'Destreza', 'Inteligencia', 'Carisma']
5
6  export function blankCharacter() {
7    return {
8      name: '',
9      type: 'Guerrero',
10     race: 'Humana',
11     level: 1,
12     realm: '',
13     portrait: 0,
14     imageDataUrl: '',
15     imageName: '',
16     story: '',
17     traits: [],
18     stats: [10, 10, 10, 10] }
19 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 30 DE 50

navigation.js

`src/data/navigation.js`

Fuente común de enlaces y títulos de pantalla.

- menuItems define las secciones de la cabecera; mainTabs define los cuatro accesos inferiores; pageTitles relaciona rutas con títulos.
- App utiliza estos títulos para el encabezado, la pestaña del navegador y el aviso de cambio de pantalla.

Código completo

24 líneas · SHA-256 del original: `1c2230c0b5cc331334eb1857b21ac7fcebaf74f9ee217d5c879250faa6668386`

```
export const menuItems = [
  { label: 'Inicio', path: '/inicio', icon: 'home' },
  { label: 'Biblioteca', path: '/biblioteca', icon: 'book' },
  { label: 'Crear personaje', path: '/crear-personaje' },
  { label: 'Inspiración', path: '/inspiracion', icon: 'sparkle' },
  { label: 'Perfil', path: '/perfil', icon: 'user' },
  { label: 'Ayuda', path: '/ayuda' },
  { label: 'Accesibilidad', path: '/accesibilidad' },
  { label: 'Iniciar sesión', path: '/login' },
]

export const mainTabs = [
  { label: 'Inicio', path: '/inicio', icon: 'home' },
  { label: 'Biblioteca', path: '/biblioteca', icon: 'book' },
  { label: 'Inspiración', path: '/inspiracion', icon: 'sparkle' },
  { label: 'Perfil', path: '/perfil', icon: 'user' },
]
export const pageTitles = {
  ...Object.fromEntries(menuItems.map(item => [item.path, item.label])),
  '/registro': 'Crear cuenta', '/recuperar': 'Recuperar acceso', '/privacidad': 'Privacidad y uso',
  '/accesibilidad/texto': 'Tamaño de texto', '/accesibilidad/contraste': 'Modo de contraste',
  '/accesibilidad/lectura': 'Lectores de pantalla', '/accesibilidad/teclado': 'Navegación por teclado',
  '/accesibilidad/apariencia': 'Personalización visual',
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 31 DE 50

accessibility.js

`src/data/accessibility.js`

Catálogos de valores y etiquetas de los cinco menús de accesibilidad.

- Define textSizes, contrastModes, themes, elementSizes y accentColors.
- accessibilityMenus contiene id, etiqueta, icono y descripción de cada submenú. La pantalla y la validación leen los mismos catálogos para evitar opciones divergentes.

Código completo

36 líneas · SHA-256 del original: `80a1383f7d244410cc28615ffe6b6f2a566c191656b219ed07995f320a452818`

```
export const textSizes = [
  { value: 'small', label: 'Pequeño' },
  { value: 'normal', label: 'Normal' },
  { value: 'large', label: 'Grande' },
  { value: 'extra-large', label: 'Extra grande' },
]
export const contrastModes = [
  { value: 'normal', label: 'Normal', description: 'Colores del tema elegido.' },
  { value: 'high', label: 'Alto contraste', description: 'Fondo negro, texto blanco y controles amarillos.' },
  { value: 'inverted', label: 'Invertido', description: 'Fondo claro, texto negro y controles azules.' },
]
export const themes = [
  { value: 'dark', label: 'Oscuro' },
  { value: 'light', label: 'Claro' },
  { value: 'sepia', label: 'Sepia' },
]
export const elementSizes = [
  { value: 'normal', label: 'Normal' },
  { value: 'large', label: 'Grande' },
  { value: 'extra-large', label: 'Extra grande' },
]
export const accentColors = [
  { value: 'red', label: 'Rojo', color: '#9e3537' },
  { value: 'purple', label: 'Violeta', color: '#7738a8' },
  { value: 'blue', label: 'Azul', color: '#12679a' },
  { value: 'green', label: 'Verde', color: '#27743e' },
  { value: 'orange', label: 'Naranja', color: '#a04d0b' },
  { value: 'slate', label: 'Gris', color: '#536177' },
]
export const accessibilityMenus = [
  { id: 'texto', label: 'Tamaño de texto', icon: 'type', description: 'Ajusta la lectura y prueba el resultado.' },
  { id: 'contraste', label: 'Modo de contraste', icon: 'contrast', description: 'Normal, alto contraste o invertido.' },
  { id: 'lectura', label: 'Lectores de pantalla', icon: 'speaker', description: 'Anuncios de navegación y lectura en voz alta.' },
```

```
34   { id: 'teclado', label: 'Navegación por teclado', icon: 'keyboard', description: 'Atajos y resaltado
     del foco.' },
35   { id: 'apariencia', label: 'Personalización visual', icon: 'palette', description: 'Tema, tamaño de
     elementos y color de acento.' },
36 ]
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 32 DE 50

index.css

`src/index.css`

Fuentes locales, reinicio de estilos y variables base de color, texto y controles.

- Declara Cormorant Garamond e Inter con font-face y recursos de public/fonts.
- Define los tokens base: foreground, background, surface, accent, text-scale, control-height y otros.
- Establece box-sizing, estilo base de enlaces y controles, foco visible y respeto a prefers-reduced-motion.

Código completo

77 líneas · SHA-256 del original: `c978c61f6a0b248b94abc4091f7f8418e86dceb6738676638b7eb73e1e113af0`

```
@font-face {
  font-family: 'Cormorant Garamond';
  font-style: normal;
  font-weight: 400;
  font-display: swap;
  src: url('/fonts/cormorant-garamond-regular.ttf') format('truetype');
}
@font-face {
  font-family: 'Cormorant Garamond';
  font-style: normal;
  font-weight: 500;
  font-display: swap;
  src: url('/fonts/cormorant-garamond-medium.ttf') format('truetype');
}
@font-face {
  font-family: 'Cormorant Garamond';
  font-style: italic;
  font-weight: 400;
  font-display: swap;
  src: url('/fonts/cormorant-garamond-italic.ttf') format('truetype');
}
@font-face {
  font-family: 'Inter';
  font-style: normal;
  font-weight: 400;
  font-display: swap;
  src: url('/fonts/inter-regular.ttf') format('truetype');
}

:root {
  font-family: 'Inter', sans-serif;
  color: #ded9c7;
  background: #202020;
  color-scheme: dark;
  font-synthesis: none;
  text-rendering: optimizeLegibility;
```

```
  -webkit-font-smoothing: antialiased;
  -moz-osx-font-smoothing: grayscale;
  --font-display: 'Cormorant Garamond', Georgia, serif;
  --background: #090f0f;
  --gold: #a99770;
  --muted: #9b9d91;
  --border: #303833;
  --accent: #9e3537;
  --foreground: #ded9c7;
  --surface: #111a18;
  --surface-raised: #25352c;
  --input-background: #0b1210;
  --feedback-background: #1b221d;
  --error: #ffbba5;
  --error-background: #302020;
  --success: #7d9c70;
  --accent-hover: #b44244;
  --accent-ink: #e09b82;
  --button-text: #fff8ef;
  --text-scale: 1;
  --control-height: 44px;
  --control-padding: 10px;
  --icon-size: 22px;
}

* { box-sizing: border-box; }
body { min-width: 280px; min-height: 100vh; margin: 0; }
button, input, select, textarea { font: inherit; }
button, a { -webkit-tap-highlight-color: transparent; }
button { color: inherit; cursor: pointer; border: 0; }
a { color: inherit; text-decoration: none; }
h1, h2, h3, p { margin: 0; }
button:focus-visible, a:focus-visible, input:focus-visible, select:focus-visible, textarea:focus-visible
{
  outline: 2px solid var(--gold);
  outline-offset: 4px;
}
img { max-width: 100%; }

@media (prefers-reduced-motion: reduce) {
  *, *::before, *::after { scroll-behavior: auto !important; transition: none !important; }
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 33 DE 50

App.css

`src/App.css`

Composición general, tarjetas, formularios, diálogos y reglas globales de accesibilidad.

- La interfaz tiene un marco máximo de 390 px y se adapta a pantallas estrechas. Grid y flex distribuyen tarjetas, barras y controles.
- Los selectores data-text-size multiplican los tamaños de fuente. data-element-size ajusta alturas y espacios de controles e iconos.
- Los selectores data-theme eligen oscuro, claro o sepia; data-accent cambia el acento. Las reglas de contraste se declaran después y tienen prioridad.
- enhanced-focus refuerza el contorno y reduce-motion elimina transiciones. Se ajustan las columnas para evitar desbordamiento en móvil.

Código completo

255 líneas · SHA-256 del original: `c31d037040bb76b8ec62d5837bec2c06213bc01fd399c35c84b561e884231a70`

```
.app-frame {
  width: 100%;
  max-width: 390px;
  min-height: 780px;
  margin: 32px auto;
  border: 1px solid var(--border);
  background: var(--background);
  color: var(--foreground);
  display: flex;
  flex-direction: column;
  position: relative;
}

.header {
  height: 62px;
  flex-shrink: 0;
  padding: 0 17px;
  display: flex;
  align-items: center;
  justify-content: space-between;
  border-bottom: 1px solid var(--border);
  position: relative;
  z-index: 2;
}
.icon-button { display: grid; place-items: center; width: 44px; height: 44px; padding: 0; color: var(--gold); background: transparent; }
.brand { display: flex; align-items: center; gap: 8px; font-family: var(--font-display); font-size: calc(25px * var(--text-scale)); letter-spacing: 1px; }
.brand svg { color: var(--gold); }
.notification-trigger { position: relative; }
```

```
.notification-dot { width: 5px; height: 5px; border-radius: 50%; background: var(--accent-ink); position: absolute; right: 8px; top: 6px; }
.header-popover { position: absolute; top: 55px; left: 16px; right: 16px; max-height: calc(100dvh - 85px); overflow-y: auto; padding: 8px; background: var(--surface); border: 1px solid var(--border); box-shadow: 0 12px 24px #0008; }
.header-popover a, .header-popover button { display: block; width: 100%; padding: 12px; font-size: calc(13px * var(--text-scale)); text-align: left; background: transparent; }
.header-popover a:hover, .header-popover button:hover { background: var(--surface-raised); }

.main-content { flex: 1; padding: 24px 23px 0; }
.hero { position: relative; padding-top: 106px; overflow: hidden; }
.hero-landscape {
  position: absolute;
  inset: 0 0 auto;
  height: 246px;
  background: linear-gradient(180deg, #090f0f05 5%, #090f0f40 35%, var(--background) 94%), url('/images/umbra-castle.png') center 38% / cover;
}
.hero-copy { position: relative; }
.welcome { color: var(--gold); font-family: var(--font-display); font-size: calc(17px * var(--text-scale)); margin-bottom: 7px; }
h1 { font-family: var(--font-display); font-weight: 400; font-size: calc(31px * var(--text-scale)); line-height: 1.04; }
.hero h1 { text-transform: uppercase; font-variant-caps: small-caps; letter-spacing: -.2px; }
.hero-description { margin: 12px 0 27px; color: var(--muted); font-size: calc(12px * var(--text-scale)); line-height: 1.65; }
.primary-button { display: flex; align-items: center; justify-content: center; gap: 9px; width: 100%; min-height: 47px; padding: 9px 12px; border-radius: 2px; background: var(--accent); color: var(--button-text); font-family: var(--font-display); font-size: calc(18px * var(--text-scale)); transition: background .2s; }
.primary-button:hover { background: var(--accent-hover); }
.primary-button > span { font-family: Georgia, serif; font-size: calc(23px * var(--text-scale)); line-height: 1; }
.ornament { display: flex; align-items: center; gap: 22px; margin: 16px 0 17px; color: var(--gold); font-size: calc(14px * var(--text-scale)); }
.ornament::before, .ornament::after { content: ''; height: 1px; flex: 1; background: var(--border); }
.section-heading { display: flex; justify-content: space-between; align-items: center; gap: 8px; margin-bottom: 13px; }
h2 { font-family: var(--font-display); font-size: calc(22px * var(--text-scale)); line-height: 1.2; font-weight: 500; }
.text-button { flex-shrink: 0; min-height: 44px; display: inline-flex; align-items: center; padding: 8px 0; color: var(--accent-ink); background: transparent; font-size: calc(11px * var(--text-scale)); }
.text-button span { margin-left: 4px; font-size: calc(15px * var(--text-scale)); }
.character-grid { display: grid; grid-template-columns: repeat(2, minmax(0, 1fr)); gap: 12px; }
.character-card { width: 100%; padding: 0; text-align: left; background: var(--surface); border: 1px solid var(--border); border-radius: 4px; overflow: hidden; transition: border-color .2s, transform .2s; }
.character-card:hover { border-color: var(--gold); transform: translateY(-2px); }
.character-portrait { display: block; width: 100%; height: auto; aspect-ratio: 1.18; object-fit: cover; object-position: center; }
.portrait-placeholder { display: grid; place-items: center; color: var(--gold); background: radial-gradient(ellipse at top, var(--surface-raised), var(--surface) 75%); font-family: var(--font-display); font-size: calc(42px * var(--text-scale)); }
.character-info { padding: 10px 10px 9px; }
.character-info h3 { font-family: var(--font-display); font-size: calc(20px * var(--text-scale)); line-height: 1.1; font-weight: 500; }
.character-info p { margin-top: 6px; color: var(--muted); font-size: calc(11px * var(--text-scale)); line-height: 1.5; }
.closing-quote { padding: 20px 0 25px; text-align: center; color: var(--muted); font-family: var(--font-display); font-size: calc(14px * var(--text-scale)); font-style: italic; }
```

```
.bottom-navigation { display: grid; grid-template-columns: repeat(4, minmax(0, 1fr)); gap: 4px; padding: 12px 8px 16px; border-top: 1px solid var(--border); background: var(--surface); }
.nav-item { min-height: 44px; display: flex; flex-direction: column; justify-content: center; align-items: center; gap: 7px; color: var(--muted); }
.nav-label { font-size: calc(11px * var(--text-scale)); }
.nav-item.active { color: var(--accent-ink); }

.page-heading { padding: 20px 0 28px; }
.page-heading h1 { margin-top: 12px; }
.eyebrow { color: var(--gold); font-size: calc(10px * var(--text-scale)); letter-spacing: 1.5px; }
.page-description, .character-story { color: var(--muted); font-size: calc(13px * var(--text-scale)); line-height: 1.8; padding: 16px 0; white-space: pre-wrap; overflow-wrap: anywhere; }
.search-field { display: flex; gap: 8px; align-items: center; margin-bottom: 18px; color: var(--gold); }
.search-field input { min-width: 0; width: 100%; }
.inspiration-panel, .profile-panel { display: grid; gap: 22px; padding: 0 0 32px; }
.inspiration-panel > svg, .profile-panel > svg { color: var(--gold); }
.inspiration-panel p, .profile-panel p { color: var(--muted); font-size: calc(13px * var(--text-scale)); line-height: 1.8; }
.storage-notice { color: var(--error); padding: 20px 0; font-size: calc(12px * var(--text-scale)); line-height: 1.6; }
.umbra-dialog { width: min(360px, calc(100% - 32px)); max-height: calc(100dvh - 48px); margin: auto; padding: 0; color: var(--foreground); background: var(--surface); border: 1px solid var(--border); }
.umbra-dialog::backdrop { background: #000b; }
.dialog-content { padding: 28px 22px; position: relative; }
.dialog-close { position: absolute; right: 7px; top: 7px; }
.dialog-content h2 { margin: 0 20px 24px 0; font-size: calc(29px * var(--text-scale)); }
.dialog-content > .character-portrait { aspect-ratio: 1; margin-bottom: 20px; border-radius: 3px; }
.character-form { display: grid; gap: 16px; }
.character-form label { display: grid; gap: 7px; font-size: calc(12px * var(--text-scale)); color: var(--muted); }
input, select, textarea { width: 100%; min-width: 0; padding: 10px; border: 1px solid var(--border); border-radius: 3px; background: var(--input-background); color: var(--foreground); font-size: calc(13px * var(--text-scale)); }
textarea { resize: vertical; }
.form-row { display: grid; grid-template-columns: 1fr 1fr; gap: 12px; }

@media (max-width: 480px) {
  .app-frame { min-height: 100dvh; margin: 0 auto; border: 0; }
  .bottom-navigation { padding-bottom: max(16px, env(safe-area-inset-bottom)); }
}
@media (max-width: 350px) {
  .header { height: 54px; }
  .brand { font-size: calc(22px * var(--text-scale)); }
  .main-content { padding: 21px 20px 0; }
  .hero { padding-top: 96px; }
  .hero h1 { font-size: calc(28px * var(--text-scale)); }
  .primary-button { min-height: 42px; }
  .section-heading { margin-bottom: 10px; }
  .text-button { padding-top: 4px; padding-bottom: 4px; }
  .character-info { padding-top: 8px; padding-bottom: 7px; }
  .character-info p { margin-top: 5px; }
  .closing-quote { padding-top: 16px; padding-bottom: 20px; }
  .character-info h3 { font-size: calc(18px * var(--text-scale)); }
  h2 { font-size: calc(20px * var(--text-scale)); }
  .bottom-navigation { padding-top: 10px; padding-bottom: max(12px, env(safe-area-inset-bottom)); }
  .nav-item { min-height: 44px; }
}

```

```
.visually-hidden { position: absolute; width: 1px; height: 1px; padding: 0; margin: -1px; overflow: hidden; clip-path: inset(50%); white-space: nowrap; border: 0; }
.skip-link { position: absolute; top: 8px; left: 8px; z-index: 5; padding: 12px; background: var(--foreground); color: var(--background); transform: translateY(-200%); }
.skip-link:focus { transform: translateY(0); }
.main-content:focus { outline: none; }
.guest-note { padding: 0 0 22px; color: var(--muted); font-size: calc(12px * var(--text-scale)); line-height: 1.7; }
.guest-note a, .account-link a, .inline-link { color: var(--accent-ink); text-decoration: underline; text-underline-offset: 3px; }
.inline-link { display: inline-flex; align-items: center; min-height: 44px; font-size: calc(13px * var(--text-scale)); }
.form-notice, .feedback { padding: 12px; border: 1px solid var(--border); border-radius: 3px; background: var(--feedback-background); color: var(--foreground); font-size: calc(13px * var(--text-scale)); line-height: 1.7; overflow-wrap: anywhere; }
.form-notice { margin-bottom: 20px; }
.connection-status { margin: 12px 16px 0; }
.feedback.error { color: var(--error); border-color: var(--error); background: var(--error-background); }
.feedback.success { border-color: var(--success); }
.form-field { display: grid; gap: 7px; }
.form-field label { display: block; font-size: calc(13px * var(--text-scale)); }
.form-hint { font-size: calc(12px * var(--text-scale)); color: var(--muted); line-height: 1.7; }
.field-error { color: var(--error); font-size: calc(13px * var(--text-scale)); line-height: 1.6; }
[aria-invalid="true"] { border-color: var(--error); }
.checkbox-label { display: flex !important; align-items: flex-start; gap: 10px; min-height: 44px; line-height: 1.7; }
input[type="checkbox"] { flex-shrink: 0; width: 20px; height: 20px; margin: 3px 0 0; accent-color: var(--gold); }
.secondary-button { display: flex; align-items: center; justify-content: center; min-height: 44px; width: 100%; padding: 10px; border: 1px solid var(--border); border-radius: 3px; background: var(--surface); color: var(--foreground); font-size: calc(13px * var(--text-scale)); text-align: center; line-height: 1.5; }
.secondary-button:hover { background: var(--surface-raised); }
.action-stack, .empty-state { display: grid; gap: 14px; }
.auth-page { padding-bottom: 24px; }
.account-link { margin-top: 24px; color: var(--muted); font-size: calc(13px * var(--text-scale)); line-height: 1.7; }
.search-results { margin-bottom: 16px; color: var(--muted); font-size: calc(12px * var(--text-scale)); }
.empty-state { padding: 16px 0; font-size: calc(13px * var(--text-scale)); color: var(--muted); line-height: 1.8; }
.information-page { display: grid; gap: 20px; padding-bottom: 30px; }
.information-page p { color: var(--muted); font-size: calc(14px * var(--text-scale)); line-height: 1.8; }
.information-page .feedback { color: var(--foreground); }
.information-page details { border-bottom: 1px solid var(--border); padding-bottom: 12px; }
.information-page summary { min-height: 44px; cursor: pointer; font-size: calc(14px * var(--text-scale)); line-height: 1.7; padding: 10px 0; }
.information-page details p { padding: 8px 0; }
.preference-option { display: flex; align-items: center; gap: 12px; min-height: 44px; font-size: calc(14px * var(--text-scale)); line-height: 1.6; }
.information-page .error-code { font-family: var(--font-display); font-size: calc(72px * var(--text-scale)); color: var(--gold); line-height: 1; }
.character-info h3, h1, h2 { overflow-wrap: anywhere; }
.editor-page { padding-bottom: 28px; }
.form-contents { display: contents; border: 0; margin: 0; padding: 0; min-width: 0; }
.traits-field { margin: 0; padding: 14px; border: 1px solid var(--border); min-width: 0; }
.traits-field legend { font-size: calc(13px * var(--text-scale)); line-height: 1.6; color: var(--gold); }
.traits-grid { display: grid; grid-template-columns: repeat(2, minmax(0, 1fr)); gap: 8px; }
.traits-grid .checkbox-label { font-size: calc(12px * var(--text-scale)); overflow-wrap: anywhere; }
.portrait-editor { display: grid; gap: 14px; }
```

```
.portrait-editor > .character-portrait { width: 160px; max-width: 100%; aspect-ratio: 1; margin: auto; }
.random-button { margin-bottom: 24px; }
.library-filters { margin-bottom: 12px; }
.library-filters label { display: grid; gap: 8px; font-size: calc(12px * var(--text-scale)); color: var(--muted); }
.search-field { margin-top: 18px; }
.favorite-label { display: block; margin-top: 8px; color: var(--gold); font-size: calc(11px * var(--text-scale)); }
.trait-list { list-style: none; display: flex; flex-wrap: wrap; gap: 8px; padding: 0; margin: 0 0 20px; }
.trait-list li { padding: 6px 9px; background: var(--surface-raised); border-radius: 3px; font-size: calc(12px * var(--text-scale)); }
.attribute-grid { display: grid; grid-template-columns: repeat(2, minmax(0, 1fr)); gap: 10px; margin: 0 0 24px; }
.attribute-grid div { padding: 12px; border: 1px solid var(--border); text-align: center; }
.attribute-grid dt { font-size: calc(12px * var(--text-scale)); color: var(--muted); }
.attribute-grid dd { margin: 8px 0 0; font-family: var(--font-display); font-size: calc(26px * var(--text-scale)); color: var(--gold); }
.danger-button { color: var(--error); justify-content: center; width: 100%; }
.profile-stats { display: flex; flex-wrap: wrap; gap: 20px; }
.profile-stats strong { color: var(--gold); font-size: calc(22px * var(--text-scale)); }
.inspiration-seed { display: grid; gap: 12px; padding: 16px; border: 1px solid var(--border); }
.storage-notice { padding: 16px; }
button:disabled { cursor: wait; opacity: .6; }
@media (max-width: 350px) {
  .section-heading { flex-wrap: wrap; }
}

.app-frame[data-text-size="small"] { --text-scale: .9; }
.app-frame[data-text-size="large"] { --text-scale: 1.2; }
.app-frame[data-text-size="extra-large"] { --text-scale: 1.4; }
.app-frame[data-element-size="large"] { --control-height: 52px; --control-padding: 13px; --icon-size: 26px; }
.app-frame[data-element-size="extra-large"] { --control-height: 60px; --control-padding: 16px; --icon-size: 30px; }

.app-frame[data-accent="purple"] { --accent: #7738a8; --accent-hover: #8b45bd; --accent-ink: #d4a6fa; }
.app-frame[data-accent="blue"] { --accent: #12679a; --accent-hover: #1977ac; --accent-ink: #87d2ff; }
.app-frame[data-accent="green"] { --accent: #27743e; --accent-hover: #308747; --accent-ink: #9bdaa5; }
.app-frame[data-accent="orange"] { --accent: #a04d0b; --accent-hover: #b05915; --accent-ink: #ffbf82; }
.app-frame[data-accent="slate"] { --accent: #536177; --accent-hover: #64748b; --accent-ink: #c5d4eb; }

.app-frame[data-theme="light"] {
  color-scheme: light;
  --background: #f5f5f0; --foreground: #252d28; --muted: #50594f; --gold: #726039;
  --surface: #fff; --surface-raised: #e4e8df; --input-background: #fff;
  --border: #818c7d; --feedback-background: #e6eddf; --error: #9c2b28; --error-background: #ffe5df; --success: #34733d;
  --accent-ink: var(--accent);
}
.app-frame[data-theme="sepia"] {
  color-scheme: light;
  --background: #efe2c8; --foreground: #3e3020; --muted: #61503b; --gold: #77551f;
  --surface: #f9efd9; --surface-raised: #e1d0b2; --input-background: #fff7e7;
  --border: #8e7958; --feedback-background: #e5d9bb; --error: #922c24; --error-background: #f7d4bb; --success: #38642d;
  --accent-ink: var(--accent);
}
.app-frame[data-contrast="high"] {
  color-scheme: dark;
```

```
  --background: #000; --foreground: #fff; --muted: #fff; --gold: #ffed00;
  --surface: #000; --surface-raised: #171717; --input-background: #000;
  --border: #fff; --feedback-background: #000; --error: #ffbcae; --error-background: #200000; --success: #b1ff9f;
  --accent: #ffed00; --accent-hover: #fff68a; --accent-ink: #ffed00; --button-text: #000;
}
.app-frame[data-contrast="inverted"] {
  color-scheme: light;
  --background: #fff; --foreground: #000; --muted: #111; --gold: #004b7a;
  --surface: #fff; --surface-raised: #e0f2ff; --input-background: #fff;
  --border: #000; --feedback-background: #eaf7ff; --error: #960000; --error-background: #ffe8e8; --success: #125122;
  --accent: #005180; --accent-hover: #003a5c; --accent-ink: #004b7a; --button-text: #fff;
}

.app-frame .primary-button, .app-frame .secondary-button, .app-frame .icon-button,
.app-frame .text-button, .app-frame .nav-item, .app-frame .inline-link,
.app-frame .header-popover a, .app-frame .header-popover button,
.app-frame .login-create-account, .app-frame .login-recovery, .app-frame .login-remember {
  min-height: var(--control-height);
}
.app-frame .icon-button { width: var(--control-height); height: var(--control-height); flex-shrink: 0; }
.app-frame .icon-button svg, .app-frame .nav-item svg { width: var(--icon-size); height: var(--icon-size); }
.app-frame .header { height: auto; min-height: 62px; padding-top: 6px; padding-bottom: 6px; }
.app-frame input:not([type="checkbox"]):not([type="radio"]):not([type="range"]),
.app-frame select, .app-frame textarea { min-height: var(--control-height); padding: var(--control-padding); }
.app-frame .login-input input { height: auto; padding-left: 34px; padding-right: 44px; }
.app-frame .login-password-toggle { height: var(--control-height); width: var(--control-height); }
.app-frame input[type="checkbox"], .app-frame input[type="radio"] { accent-color: var(--accent); }
.app-frame .section-heading { flex-wrap: wrap; }
.app-frame .nav-item { min-width: 0; }
.app-frame .nav-label { text-align: center; overflow-wrap: anywhere; }
.app-frame p, .app-frame label, .app-frame small, .app-frame legend { overflow-wrap: anywhere; }
.app-frame .read-aloud { display: grid; gap: 12px; margin-bottom: 16px; }
.app-frame .hero-copy { color: var(--foreground); }
.app-frame[data-theme="light"] .hero-landscape, .app-frame[data-theme="sepia"] .hero-landscape,
.app-frame[data-contrast="inverted"] .hero-landscape { opacity: .18; }
.app-frame[data-contrast="high"] .hero-landscape { opacity: .25; }
.app-frame.enhanced-focus :focus-visible { outline: 4px solid var(--gold); outline-offset: 5px; }
.app-frame.reduce-motion *, .app-frame.reduce-motion *::before, .app-frame.reduce-motion *::after {
  animation: none !important; transition: none !important; scroll-behavior: auto !important;
}
.app-frame.reduce-motion .character-card:hover { transform: none; }
@media (prefers-reduced-motion: reduce) {
  .character-card:hover { transform: none; }
}
@media (max-width: 350px) {
  .app-frame[data-text-size="large"] .character-grid,
  .app-frame[data-text-size="extra-large"] .character-grid,
  .app-frame[data-text-size="extra-large"] .traits-grid,
  .app-frame[data-element-size="extra-large"] .library-filters { grid-template-columns: 1fr; }
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 34 DE 50

Login.css

`src/pages/Login.css`

Composición específica de la pantalla de acceso.

- Estiliza el marco de login, el símbolo, los campos con iconos, el botón de visibilidad, las opciones y los separadores.
- Utiliza las variables de color y text-scale para respetar los ajustes del perfil Invitado que se ven antes del acceso.

Código completo

238 líneas · SHA-256 del original: `9171b7bceded35939c7f54794fde857b8e4a5636fff99f6c79b6ff68fd34d886`

```
.app-frame.login-frame {
  max-width: 336px;
  min-height: 728px;
}

.login-frame .header {
  height: 54px;
  padding: 0 8px;
}

.login-frame .brand {
  gap: 7px;
  font-size: calc(20px * var(--text-scale));
  letter-spacing: .5px;
}

.login-frame .brand svg {
  width: 25px;
  height: 25px;
}

.login-frame .header .icon-button svg {
  width: 18px;
  height: 18px;
}

.login-frame .main-content {
  display: flex;
  padding: 0 20px;
}

.login-page {
  display: flex;
  flex-direction: column;
  width: 100%;
  min-width: 0;
```

```
}

.login-intro {
  padding-top: 37px;
  text-align: center;
}

.login-intro>svg {
  color: var(--gold);
  stroke-width: .65;
}

.login-intro h1 {
  margin-top: 0;
  font-size: calc(26px * var(--text-scale));
  text-transform: uppercase;
  line-height: 1.2;
}

.login-intro p {
  margin-top: 11px;
  color: var(--muted);
  font-family: var(--font-display);
  font-size: calc(15px * var(--text-scale));
  line-height: 1.5;
}

.login-divider {
  display: flex;
  align-items: center;
  gap: 19px;
  margin: 28px 0 18px;
  color: var(--gold);
  font-size: calc(13px * var(--text-scale));
}

.login-divider::before,
.login-divider::after {
  content: '';
  flex: 1;
  height: 1px;
  background: var(--border);
}

.login-form {
  display: grid;
  gap: 16px;
}

.login-field {
  display: grid;
  gap: 6px;
}

.login-field>label {
  font-family: var(--font-display);
  font-size: calc(14px * var(--text-scale));
}

```

```
.login-input {
  position: relative;
  display: flex;
  align-items: center;
}

.login-input>svg {
  position: absolute;
  left: 10px;
  color: var(--gold);
  pointer-events: none;
}

.login-input input {
  height: 42px;
  padding: 10px 38px 10px 34px;
  border-color: var(--border);
  border-radius: 2px;
  background: var(--input-background);
  font-size: calc(10px * var(--text-scale));
}

.login-input input::placeholder {
  color: var(--muted);
  opacity: 1;
}

.login-input input[aria-invalid="true"] {
  border-color: var(--error);
}

.login-password-toggle {
  position: absolute;
  right: 0;
  display: grid;
  place-items: center;
  width: 38px;
  height: 42px;
  padding: 0;
  background: transparent;
  color: var(--muted);
}

.login-options {
  display: flex;
  align-items: center;
  justify-content: space-between;
  flex-wrap: wrap;
  gap: 6px 12px;
  margin-top: -7px;
  font-size: calc(9px * var(--text-scale));
}

.login-remember {
  display: flex;
  align-items: center;
  gap: 7px;
  min-height: 28px;
  color: var(--muted);
```

```
  cursor: pointer;
}

.login-remember input[type="checkbox"] {
  appearance: none;
  width: 12px;
  height: 12px;
  margin: 0;
  padding: 0;
  border: 1px solid var(--gold);
  border-radius: 2px;
  background: transparent;
}

.login-remember input:checked {
  background: var(--gold);
  box-shadow: inset 0 0 0 2px var(--background);
}

.login-options .login-recovery {
  display: inline-flex;
  align-items: center;
  min-height: 28px;
  padding: 0;
  background: transparent;
  color: var(--gold);
  font-size: inherit;
}

.login-form .login-submit {
  min-height: 40px;
  margin-top: -10px;
  gap: 8px;
  font-size: calc(16px * var(--text-scale));
}

.login-account-divider {
  gap: 10px;
  margin: 20px 0 19px;
  color: var(--muted);
  font-family: var(--font-display);
  font-size: calc(13px * var(--text-scale));
}

.login-create-account {
  display: flex;
  align-items: center;
  justify-content: center;
  gap: 8px;
  min-height: 40px;
  border: 1px solid var(--border);
  border-radius: 2px;
  background: var(--surface);
  font-family: var(--font-display);
  font-size: calc(16px * var(--text-scale));
}

.login-create-account:hover {
  background: var(--surface-raised);
```

```
}

.login-terms {
  margin-top: 20px;
  color: var(--muted);
  text-align: center;
  font-size: calc(8px * var(--text-scale));
  line-height: 1.7;
}

.login-motto {
  margin-top: auto;
  padding: 40px 0 17px;
  color: var(--muted);
  text-align: center;
  font-family: var(--font-display);
  font-size: calc(13px * var(--text-scale));
  font-style: italic;
}

@media (max-width: 480px) {
  .app-frame.login-frame {
    max-width: 390px;
  }
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 35 DE 50

Accessibility.css

`src/pages/Accessibility.css`

Presentación del menú y los controles de accesibilidad.

- Da forma a las filas de navegación, el perfil activo, radios, marcas de selección, muestras de color, deslizador y vista previa.
- La tabla keyboard-shortcuts describe atajos. Todos los estilos heredan las variables del tema, el contraste y el tamaño elegido.

Código completo

44 líneas · SHA-256 del original: `b0f317a6716b445e7c105892c481a0541c33db47c70fd5f05d5537523f27822f`

```
.accessibility-page { display: grid; gap: 22px; padding-bottom: 30px; }
.settings-back { display: inline-flex; align-items: center; gap: 6px; min-height: var(--control-height); color: var(--accent-ink); font-size: calc(13px * var(--text-scale)); }
.settings-profile { display: flex; align-items: flex-start; gap: 12px; padding: 15px; border: 1px solid var(--border); background: var(--surface); }
.settings-profile > svg { flex-shrink: 0; color: var(--gold); }
.settings-profile p { font-size: calc(13px * var(--text-scale)); line-height: 1.7; }
.settings-profile small { display: block; color: var(--muted); font-size: calc(11px * var(--text-scale)); }
.settings-description { color: var(--muted); font-size: calc(13px * var(--text-scale)); line-height: 1.8; }
.settings-menu { display: grid; gap: 10px; }
.settings-menu-item { display: flex; gap: 12px; align-items: center; min-height: var(--control-height); padding: var(--control-padding); border: 1px solid var(--border); border-radius: 4px; background: var(--surface); }
.settings-menu-item:hover { background: var(--surface-raised); }
.settings-menu-item > span:nth-child(2) { min-width: 0; flex: 1; }
.settings-menu-item > svg { flex-shrink: 0; color: var(--gold); }
.settings-icon { display: grid; place-items: center; width: 38px; min-height: 44px; color: var(--gold); flex-shrink: 0; }
.settings-menu-item strong { display: block; font-size: calc(14px * var(--text-scale)); line-height: 1.5; font-weight: 500; }
.settings-menu-item small { display: block; margin-top: 5px; font-size: calc(11px * var(--text-scale)); line-height: 1.6; color: var(--muted); }
.settings-current { display: block; margin-top: 8px; color: var(--accent-ink); font-size: calc(11px * var(--text-scale)); line-height: 1.6; }
.settings-choices { min-width: 0; display: grid; gap: 10px; border: 0; padding: 0; margin: 0; }
.settings-choices legend { margin-bottom: 14px; color: var(--gold); font-size: calc(16px * var(--text-scale)); line-height: 1.5; }
.choice-card { display: flex; align-items: center; gap: 10px; min-height: var(--control-height); padding: var(--control-padding); background: var(--surface); border: 1px solid var(--border); border-radius: 4px; cursor: pointer; }
.choice-card.is-selected { border-color: var(--accent-ink); box-shadow: inset 3px 0 var(--accent-ink); background: var(--surface-raised); }
.choice-card > input { width: 18px; height: 18px; padding: 0; margin: 0; flex-shrink: 0; }
.choice-card > span:not(.color-swatch) { flex: 1; min-width: 0; }
```

```
23 .choice-card strong { font-size: calc(13px * var(--text-scale)); font-weight: 500; line-height: 1.6; }
24 .choice-card small { display: block; color: var(--muted); font-size: calc(11px * var(--text-scale));
   line-height: 1.7; margin-top: 5px; }
25 .choice-card > svg { flex-shrink: 0; color: var(--accent-ink); }
26 .color-swatch { display: block; width: 22px; height: 22px; flex-shrink: 0; border-radius: 50%; border:
   1px solid var(--foreground); }
27 .settings-preview { display: grid; gap: 12px; padding: 18px; border: 1px solid var(--border); background:
   var(--surface); border-radius: 4px; }
28 .settings-preview p:not(.eyebrow) { font-size: calc(13px * var(--text-scale)); line-height: 1.8; }
29 .settings-preview .preview-tags { color: var(--gold); }
30 .text-size-slider { display: grid; gap: 16px; color: var(--muted); font-size: calc(13px * var(--text-
   scale)); line-height: 1.7; }
31 .text-size-slider > span { display: flex; align-items: center; gap: 14px; }
32 .text-size-slider > span > span:last-child { font-size: calc(25px * var(--text-scale)); }
33 .text-size-slider input { flex: 1; width: 100%; min-height: var(--control-height); padding: 0; accent-
   color: var(--accent); }
34 .settings-toggle { display: flex; align-items: flex-start; gap: 12px; min-height: var(--control-height);
   padding: var(--control-padding); border: 1px solid var(--border); background: var(--surface); font-size:
   calc(13px * var(--text-scale)); line-height: 1.8; cursor: pointer; }
35 .settings-toggle small { display: block; color: var(--muted); font-size: calc(11px * var(--text-scale));
   }
36 .keyboard-shortcuts { width: 100%; border-collapse: collapse; table-layout: fixed; font-size: calc(12px *
   var(--text-scale)); line-height: 1.8; }
37 .keyboard-shortcuts caption { text-align: left; color: var(--gold); font-size: calc(16px * var(--text-
   scale)); margin-bottom: 12px; }
38 .keyboard-shortcuts th, .keyboard-shortcuts td { padding: 12px 6px; border-bottom: 1px solid var(--
   border); text-align: left; overflow-wrap: anywhere; }
39 .keyboard-shortcuts th { width: 45%; font-weight: 400; }
40 .keyboard-shortcuts kbd { display: inline-block; padding: 2px 5px; margin: 2px 0; border: 1px solid var(-
   -border); border-radius: 3px; background: var(--surface); color: var(--foreground); font: inherit; }
41 @media (max-width: 350px) {
42   .settings-menu-item { gap: 8px; }
43   .settings-icon { width: 27px; }
44 }
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 36 DE 50

package.json

package.json

Identidad del paquete, dependencias directas y comandos de desarrollo.

- React y React DOM componen la interfaz; React Router gestiona las rutas; Vite desarrolla y compila; ESLint comprueba el código.
- Los comandos son dev, build, test, test:browser, lint y preview. Las versiones con ^ son rangos declarados; package-lock.json concreta la resolución instalada.

Código completo

30 líneas · SHA-256 del original: `9bd88f61dd2f00d8cd60f4a9885a90e18f8772c2aa0751bca8a863a7d448f81b`

```
{
  "name": "proyecto-interdisciplinario---umbra",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "test": "node --test",
    "test:browser": "node scripts/browser-smoke.mjs",
    "lint": "eslint .",
    "preview": "vite preview"
  },
  "dependencies": {
    "react": "^19.3.1",
    "react-dom": "^19.3.1",
    "react-router": "^7.14.0"
  },
  "devDependencies": {
    "@eslint/js": "^10.0.1",
    "@types/react": "^19.3.0",
    "@types/react-dom": "^19.3.0",
    "@vitejs/plugin-react": "^6.1.2",
    "eslint": "^10.13.0",
    "eslint-plugin-react-hooks": "^7.1.1",
    "eslint-plugin-react-refresh": "^0.5.7",
    "globals": "^17.13.0",
    "vite": "^8.3.4"
  }
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 37 DE 50

vite.config.js

`vite.config.js`

Configuración del compilador y del servidor Vite.

- Activa el plugin React y excluye public/fonts de la vigilancia de archivos para evitar bloqueos EBUSY en Windows.
- No define proxy de API ni configuración de backend. El comentario enlazado a Vite forma parte del código original.

Funciones y elementos principales

Nombre	Qué hace	Línea
`defineConfig`	Ayudante de Vite que recibe la configuración exportada.	—

Código completo

13 líneas · SHA-256 del original: `7ea68df159e6dcf1a4da387c5eb828a35f402484ba3189819a79c6a5e52ad3bd`

```
1  import react from '@vitejs/plugin-react'
2  import { defineConfig } from 'vite'
3
4  // https://vite.dev/config/
5  export default defineConfig({
6    plugins: [react()],
7    server: {
8      watch: {
9        // Evita EBUSY al vigilar fuentes estáticas bloqueadas en Windows.
10       ignored: ['**/public/fonts/**'],
11     },
12   },
13 })
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 38 DE 50

eslint.config.js

`eslint.config.js`

Reglas de análisis estático para JavaScript y JSX.

- Ignora dist y combina recomendaciones de JavaScript, hooks de React y React Refresh.
- Declara globals de navegador y soporte para JSX. No ejecuta la app ni sustituye las pruebas de comportamiento.

Funciones y elementos principales

Nombre	Qué hace	Línea
`defineConfig`	Compone el array de reglas de ESLint.	—
`globalIgnores`	Indica los directorios que se excluyen del análisis.	—

Código completo

21 líneas · SHA-256 del original: `9f9b44b8e55768eaf86e417b96919510b12055d87806e0e2f0703c5a0eb64817`

```
import js from '@eslint/js'
import globals from 'globals'
import reactHooks from 'eslint-plugin-react-hooks'
import reactRefresh from 'eslint-plugin-react-refresh'
import { defineConfig, globalIgnores } from 'eslint/config'

export default defineConfig([
  globalIgnores(['dist']),
  {
    files: ['**/*.{js,jsx}'],
    extends: [
      js.configs.recommended,
      reactHooks.configs.flat.recommended,
      reactRefresh.configs.vite,
    ],
    languageOptions: {
      globals: globals.browser,
      parserOptions: { ecmaFeatures: { jsx: true } },
    },
  },
])
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 39 DE 50

characterStore.test.js

`tests/characterStore.test.js`

Pruebas del almacenamiento y las copias de personajes.

- Comprueba aislamiento de bibliotecas, persistencia de listas vacías, importación válida e inválida, ids repetidos y que un fallo de escritura conserve datos anteriores.

Funciones y elementos principales

Nombre	Qué hace	Línea
`memoryStorage`	Crea un almacenamiento de prueba basado en Map para no usar datos reales de navegador.	6

Código completo

43 líneas · SHA-256 del original: `66e4796c990f66f5da9eeda449a4a953c822d30de4ffee8d866dc8e2d95f12cd`

```
1  import assert from 'node:assert/strict'
2  import test from 'node:test'
3  import { initialCharacters } from '../src/data/characters.js'
4  import { importCharacters, normalizeCharacter, readCharacters, writeCharacters } from
   '../src/utils/characterStore.js'
5
6  function memoryStorage() {
7    const data = new Map()
8    return { getItem: key => data.has(key) ? data.get(key) : null, setItem: (key, value) => data.set(key,
   value) }
9  }
10
11 test('una biblioteca vacía persiste y las cuentas conservan bibliotecas independientes', () => {
12   const storage = memoryStorage()
13   assert.equal(readCharacters('guest', storage).length, 6)
14   writeCharacters('guest', [], storage)
15   assert.deepEqual(readCharacters('guest', storage), [])
16   assert.equal(readCharacters('account', storage).length, 6)
17   const character = normalizeCharacter({ ...initialCharacters[0], name: 'Nueva leyenda', favorite: true
   })
18   writeCharacters('account', [character], storage)
19   assert.deepEqual(readCharacters('account', storage), [character])
20   assert.deepEqual(readCharacters('guest', storage), [])
21 })
22
23 test('una copia restaura los datos y rechaza identificadores repetidos o contenido inválido', () => {
24   const character = normalizeCharacter(initialCharacters[0])
25   const backup = characters => JSON.stringify({ app: 'UMBRA', version: 1, characters })
26   assert.deepEqual(importCharacters(backup([character])), [character])
```

```
  assert.deepEqual(importCharacters(backup([])), [])
  assert.throws(() => importCharacters('no es JSON'))
  assert.throws(() => importCharacters(JSON.stringify({ version: 2, characters: [] })))
  assert.throws(() => importCharacters(backup([character, character])), /repetidos/)
  for (const changes of [{ level: 21 }, { stats: [1, 2, 3, 99] }, { traits: ['a', 'b', 'c', 'd', 'e', 'f', 'g'] }, { type: 'Inventada' }, { imageDataUrl: 'javascript:alert(1)' }]) {
    assert.throws(() => importCharacters(backup([{ ...character, ...changes }])))
  }
})

test('un fallo de almacenamiento no destruye la copia anterior', () => {
  const storage = memoryStorage()
  writeCharacters('guest', initialCharacters, storage)
  const before = readCharacters('guest', storage)
  const blocked = { ...storage, setItem: () => { throw new Error('QuotaExceeded') } }
  assert.throws(() => writeCharacters('guest', [], blocked))
  assert.deepEqual(readCharacters('guest', storage), before)
})
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 40 DE 50

localAccounts.test.js

`tests/localAccounts.test.js`

Pruebas de registro, verificadores, sesiones y recuperación.

- Comprueba correos duplicados, contraseñas erróneas, ausencia de secretos en texto plano, sesión temporal, Recordarme y cierre de sesión.
- Verifica que el código es necesario para recuperar y que cambiar la contraseña conserva el id de la cuenta.

Funciones y elementos principales

Nombre	Qué hace	Línea
`memoryStorage`	Simula localStorage y sessionStorage mediante Map.	5

Código completo

46 líneas · SHA-256 del original: `f9c74b3be3d6b928482ef505aff66a27eebf49e46cd6689ac47108c804688b8c`

```
import assert from 'node:assert/strict'
import test from 'node:test'
import { loginAccount, logoutAccount, readSession, recoverAccount, registerAccount } from '../src/utils/localAccounts.js'

function memoryStorage() {
  const data = new Map()
  return { getItem: key => data.get(key) ?? null, setItem: (key, value) => data.set(key, value), removeItem: key => data.delete(key) }
}

test('registro, verificación de contraseña, sesiones temporales y Recordarme', async () => {
  const storage = memoryStorage()
  const temporary = memoryStorage()
  const data = { name: 'Elara', email: 'Elara@Umbra.es', password: 'Historia1!' }
  const code = await registerAccount(data, storage)
  assert.equal(code.length, 32)
  const serialized = storage.getItem('umbra.accounts.v1')
  assert.ok(!serialized.includes(data.password))
  assert.ok(!serialized.includes(code))
  await assert.rejects(registerAccount({ ...data, email: 'elara@umbra.es' }, storage), /Ya existe/)
  await assert.rejects(loginAccount({ ...data, password: 'incorrecta' }, storage, temporary), /no coinciden/)
  const user = await loginAccount(data, storage, temporary)
  assert.equal(user.name, 'Elara')
  assert.deepEqual(readSession(storage, temporary), user)
  assert.equal(readSession(storage, memoryStorage()), null)
```

```
  await loginAccount({ ...data, remember: true }, storage, temporary)
  assert.deepEqual(readSession(storage, memoryStorage()), user)
  assert.equal(temporary.getItem('umbra.session.v1'), null)
  await loginAccount({ ...data, remember: false }, storage, temporary)
  assert.equal(storage.getItem('umbra.session.v1'), null)
  logoutAccount(storage, temporary)
  assert.equal(readSession(storage, temporary), null)
})

test('recuperación requiere el código y cambia la contraseña conservando la identidad', async () => {
  const storage = memoryStorage()
  const temporary = memoryStorage()
  const data = { name: 'Kael', email: 'kael@umbra.es', password: 'Antigua1!' }
  const code = await registerAccount(data, storage)
  const original = await loginAccount({ ...data, remember: true }, storage, temporary)
  await assert.rejects(recoverAccount({ ...data, recoveryCode: 'incorrecto' }, storage, temporary), /no
son correctos/)
  const changed = { ...data, password: 'OtraClave2!' }
  await recoverAccount({ ...changed, recoveryCode: code.toLowerCase() }, storage, temporary)
  assert.equal(readSession(storage, temporary), null)
  await assert.rejects(loginAccount(data, storage, temporary))
  assert.equal((await loginAccount(changed, storage, temporary)).id, original.id)
})
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 41 DE 50

preferenceStore.test.js

`tests/preferenceStore.test.js`

Pruebas de accesibilidad independiente por perfil.

- Comprueba dos cuentas e Invitado, restablecimiento aislado, migración de la antigua configuración global y rechazo de valores desconocidos.
- También verifica que un fallo de escritura no borre la configuración persistida anterior.

Funciones y elementos principales

Nombre	Qué hace	Línea
`memoryStorage`	Proporciona un almacén aislado para cada escenario de prueba.	5

Código completo

42 líneas · SHA-256 del original: `44f18b3026203a6c61dcb97cf28c841a10937f78486391c25a6e6e77810d4a53`

```
1  import assert from 'node:assert/strict'
2  import test from 'node:test'
3  import { defaultPreferences, normalizePreferences, readPreferences, writePreferences } from
   '../src/utils/preferenceStore.js'
4
5  function memoryStorage() {
6    const values = new Map()
7    return { getItem: key => values.get(key) ?? null, setItem: (key, value) => values.set(key, value) }
8  }
9
10 test('cada perfil y el invitado conservan todos sus ajustes independientes', () => {
11   const storage = memoryStorage()
12   const first = { ...defaultPreferences, theme: 'sepia', textSize: 'extra-large', contrast: 'high',
   accent: 'purple', elementSize: 'large', readAloud: true, readingRate: 0.75, enhancedFocus: true,
   announceNavigation: false, reduceMotion: true }
13   writePreferences('account-a', first, storage)
14   writePreferences('account-b', { ...defaultPreferences, theme: 'light', accent: 'blue' }, storage)
15   writePreferences('guest', { ...defaultPreferences, textSize: 'small' }, storage)
16   assert.deepEqual(readPreferences('account-a', storage), first)
17   assert.equal(readPreferences('account-b', storage).theme, 'light')
18   assert.equal(readPreferences('guest', storage).textSize, 'small')
19   assert.deepEqual(readPreferences('new-account', storage), defaultPreferences)
20   writePreferences('account-a', defaultPreferences, storage)
21   assert.equal(readPreferences('account-b', storage).theme, 'light')
22   assert.equal(readPreferences('guest', storage).textSize, 'small')
23 })
24
25 test('los ajustes globales antiguos se recuperan solo para el invitado', () => {
```

```
  const storage = memoryStorage()
  storage.setItem('umbra.accessibility.v1', JSON.stringify({ highContrast: true, largeText: true }))
  assert.equal(readPreferences('guest', storage).contrast, 'high')
  assert.equal(readPreferences('guest', storage).textSize, 'large')
  assert.deepEqual(readPreferences('account-a', storage), defaultPreferences)
  writePreferences('guest', defaultPreferences, storage)
  assert.deepEqual(readPreferences('guest', storage), defaultPreferences)
})

test('los valores desconocidos no se aplican y el fallo de guardado no sobrescribe ajustes anteriores', () => {
  const storage = memoryStorage()
  assert.deepEqual(normalizePreferences({ textSize: 'huge', theme: 'unknown', accent: 'invalid', readingRate: 99 }), defaultPreferences)
  assert.deepEqual(normalizePreferences(null), defaultPreferences)
  writePreferences('account-a', defaultPreferences, storage)
  assert.throws(() => writePreferences('account-a', { ...defaultPreferences, theme: 'sepia' }, { ...storage, setItem: () => { throw new Error('Storage blocked') } }))
  assert.deepEqual(readPreferences('account-a', storage), defaultPreferences)
})
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 42 DE 50

validateForm.test.js

`tests/validateForm.test.js`

Pruebas de las reglas compartidas de formularios.

- Incluye campos vacíos, correos, longitud y carácter especial de contraseña, confirmación, nivel entero entre 1 y 20 y casilla obligatoria.

Funciones y elementos principales

Nombre	Qué hace	Línea
`values`	Construye FormData a partir de un objeto sencillo para cada caso de prueba.	5

Código completo

50 líneas · SHA-256 del original: `f86dec842e30062c94bbfc7924821830f5054fcd34d816c21bdc79b58d2a24ee`

```
import assert from 'node:assert/strict'
import test from 'node:test'
import { validateForm } from '../src/utils/validateForm.js'

function values(entries) {
  const data = new FormData()
  for (const [name, value] of Object.entries(entries)) data.set(name, value)
  return data
}

test('un nombre vacío o compuesto de espacios requiere una corrección', () => {
  const fields = [{ name: 'name', label: 'Nombre', required: true }]
  assert.ok(validateForm(fields, values({ name: '   ' })).name)
  assert.deepEqual(validateForm(fields, values({ name: 'Elara Valen' })), {})
})

test('el correo exige usuario, arroba y dominio', () => {
  const fields = [{ name: 'email', type: 'email', required: true }]
  for (const email of ['elara', 'elara@', '@umbra.es', 'elara@umbra', 'elara @umbra.es']) {
    assert.ok(validateForm(fields, values({ email })).email, email)
  }
  assert.deepEqual(validateForm(fields, values({ email: 'elara@umbra.es' })), {})
})

test('el registro exige longitud suficiente, un carácter especial y contraseñas iguales', () => {
  const fields = [
    { name: 'password', required: true, minLength: 8, requireSpecial: true },
    { name: 'confirmPassword', required: true, matches: 'password' },
  ]
```

```
30   assert.ok(validateForm(fields, values({ password: 'corta!', confirmPassword: 'corta!' })).password)
31   for (const password of ['prueba123', 'prueba 123', 'contraseña123']) {
32     assert.ok(validateForm(fields, values({ password, confirmPassword: password })).password, password)
33   }
34   assert.ok(validateForm(fields, values({ password: 'prueba1!', confirmPassword: 'diferente'
})).confirmPassword)
35   for (const password of ['prueba1!', 'prueba1@', 'prueba1#', 'prueba1$']) {
36     assert.deepEqual(validateForm(fields, values({ password, confirmPassword: password })), {})
37   }
38 })
39
40 test('el nivel debe ser entero y estar entre 1 y 20', () => {
41   const fields = [{ name: 'level', type: 'number', required: true, min: 1, max: 20 }]
42   for (const level of ['', '0', '21', '2.5', 'letras']) assert.ok(validateForm(fields, values({ level
})).level)
43   for (const level of ['1', '8', '20']) assert.deepEqual(validateForm(fields, values({ level })), {})
44 })
45
46 test('una casilla obligatoria debe estar marcada', () => {
47   const fields = [{ name: 'consent', type: 'checkbox', required: true }]
48   assert.ok(validateForm(fields, values({})).consent)
49   assert.deepEqual(validateForm(fields, values({ consent: 'on' })), {})
50 })
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 43 DE 50

browser-smoke.mjs

`scripts/browser-smoke.mjs`

Recorridos de integración en Chrome o Edge mediante Chrome DevTools Protocol, sin dependencia de Playwright.

- Necesita la app ejecutándose y un navegador con CDP en el puerto 9222. Crea un contexto aislado y lo elimina al terminar; las limpiezas de almacenamiento ocurren dentro de ese contexto.
- Prueba personajes, filtros, retratos, favoritos, duplicado, eliminación, fallos de guardado, copias, cuentas y recuperación.
- Comprueba los cinco menús, los cambios de texto y contraste, el árbol de accesibilidad, el foco, los tamaños, el cambio entre perfiles y el diseño de 320/390 px.
- La voz se sustituye por una simulación en algunos recorridos para comprobar texto, velocidad y cancelación; esos casos no verifican la reproducción de sonido real ni un lector de pantalla externo.
- UMBRA_URL elige la dirección de la app. UMBRA_SCREENSHOT permite guardar una captura. Los fallos producen un código de salida distinto de cero.

Funciones y elementos principales

Nombre	Qué hace	Línea
`send`	Envía una orden CDP con id y sesión y relaciona su respuesta con una promesa.	27
`evaluate`	Ejecuta JavaScript en la página y devuelve el valor o su error.	34
`wait`	Consulta una condición de la página hasta cumplirla o agotar el plazo.	39
`click`	Enfoca y pulsa un elemento encontrado mediante selector.	46
`clickText`	Encuentra un botón o enlace por texto, lo enfoca y lo pulsa.	51
`fill`	Rellena un control identificado por name.	57
`fillSelector`	Utiliza el setter nativo y eventos para actualizar inputs o textareas de React.	60
`select`	Cambia un select y dispara su evento change.	63
`importFile`	Simula la selección de una copia JSON con File y DataTransfer.	66

Nombre	Qué hace	Línea
`go`	Carga una ruta y espera a que aparezca el contenido principal.	69
`reload`	Recarga la página y espera a que React vuelva a montarse.	73

Código completo

366 líneas · SHA-256 del original: `4e7869f75f0b3711184905afbd5d874ae70508b887415e3ccdc786aa6a716305`

```
// Requires Chrome/Edge with CDP on port 9222. Uses an isolated, disposable context.
import assert from 'node:assert/strict'
import { setTimeout as delay } from 'node:timers/promises'
import { writeFile } from 'node:fs/promises'
import { Buffer } from 'node:buffer'
import process from 'node:process'

const origin = process.env.UMBRA_URL || 'http://127.0.0.1:5173'
const browser = await (await fetch('http://127.0.0.1:9222/json/version')).json()
const socket = new WebSocket(browser.webSocketDebuggerUrl)
await new Promise((resolve, reject) => { socket.onopen = resolve; socket.onerror = reject })
let sequence = 0
let sessionId
let browserContextId
const pending = new Map()
const exceptions = []
socket.onmessage = event => {
  const message = JSON.parse(event.data)
  if (message.method === 'Runtime.exceptionThrown') exceptions.push(message.params.exceptionDetails)
  if (pending.has(message.id)) {
    const { resolve, reject } = pending.get(message.id)
    pending.delete(message.id)
    if (message.error) reject(new Error(message.error.message))
    else resolve(message.result)
  }
}
function send(method, params = {}, session = sessionId) {
  return new Promise((resolve, reject) => {
    const id = ++sequence
    pending.set(id, { resolve, reject })
    socket.send(JSON.stringify({ id, method, params, ...(session ? { sessionId: session } : {}) }))
  })
}
async function evaluate(expression) {
  const result = await send('Runtime.evaluate', { expression, returnByValue: true, awaitPromise: true })
  if (result.exceptionDetails) throw new Error(JSON.stringify(result.exceptionDetails))
  return result.result.value
}
async function wait(expression) {
  for (let index = 0; index < 100; index++) {
    if (await evaluate(expression)) return
    await delay(100)
  }
  throw new Error(`Timed out: ${expression}\n${await evaluate('document.body.innerText')}`)
}
```

```
async function click(selector) {
  await wait(`Boolean(document.querySelector(${JSON.stringify(selector)}))`)
  await evaluate(`(() => { const el = document.querySelector(${JSON.stringify(selector)}); el.focus(); el.click(); })()`)
  await delay(80)
}
async function clickText(text) {
  const expression = `[...document.querySelectorAll('button,a')].find(el => el.textContent.trim() === ${JSON.stringify(text)})`
  await wait(`Boolean(${expression})`)
  await evaluate(`(() => { const el = ${expression}; el.focus(); el.click(); })()`)
  await delay(80)
}
async function fill(name, value) {
  await fillSelector(`[name="${name}"]`, value)
}
async function fillSelector(selector, value) {
  await evaluate(`(() => { const el = document.querySelector(${JSON.stringify(selector)}); const proto = el.tagName === 'TEXTAREA' ? HTMLTextAreaElement.prototype : HTMLInputElement.prototype; Object.getOwnPropertyDescriptor(proto, 'value').set.call(el, ${JSON.stringify(value)}); el.dispatchEvent(new Event('input', { bubbles: true })); el.dispatchEvent(new Event('change', { bubbles: true })); })()`)
}
async function select(selector, value) {
  await evaluate(`(() => { const el = document.querySelector(${JSON.stringify(selector)}); el.value = ${JSON.stringify(value)}; el.dispatchEvent(new Event('change', {bubbles:true})); })()`)
}
async function importFile(contents) {
  await evaluate(`(() => { const transfer = new DataTransfer(); transfer.items.add(new File([${JSON.stringify(contents)}], 'backup.json', {type:'application/json'})); const input = document.querySelector('input[type=file]'); input.files = transfer.files; input.dispatchEvent(new Event('change', {bubbles:true})); })()`)
}
async function go(path) {
  await send('Page.navigate', { url: origin + path })
  await wait(`location.pathname === ${JSON.stringify(path)} && Boolean(document.querySelector('.main-content h1'))`)
}
async function reload() {
  await send('Page.reload')
  await delay(300)
  await wait("Boolean(document.querySelector('.main-content h1'))")
}

try {
  ;({ browserContextId } = await send('Target.createBrowserContext', { disposeOnDetach: true }, null))
  const { targetId } = await send('Target.createTarget', { url: 'about:blank', browserContextId }, null)
  ;({ sessionId } = await send('Target.attachToTarget', { targetId, flatten: true }, null))
  await send('Runtime.enable')
  await send('Page.enable')
  await send('Page.bringToFront')
  await send('Emulation.setDeviceMetricsOverride', { width: 390, height: 844, deviceScaleFactor: 1, mobile: true })
  await go('/inicio')
  await evaluate('localStorage.clear(); sessionStorage.clear()')
  await reload()
  assert.equal(await evaluate("document.querySelectorAll('.character-card').length"), 2)
  await click('.bottom-navigation a[href="/biblioteca"]')
  await wait("document.querySelectorAll('.character-card').length === 6")
```

```
  await fillSelector('input[type=search]', 'angelica')
  await wait("document.querySelectorAll('.character-card').length === 1")
  await fillSelector('input[type=search]', 'nada coincide')
  await wait("document.querySelectorAll('.character-card').length === 0")
  await clickText('Limpiar filtros')
  await wait("document.querySelectorAll('.character-card').length === 6")
  await clickText('+ Crear personaje')
  await clickText('Guardar personaje')
  await wait("Boolean(document.querySelector('[aria-invalid=true]'))")
  await fill('name', 'Ayla de Prueba')
  await fill('realm', 'Bosque de Niebla')
  await fill('story', 'Una historia que debe persistir.')
  await click('input[name="traits"][value="Leal"]')
  await evaluate("(() => { const transfer = new DataTransfer(); transfer.items.add(new File(['invalid'], 'portrait.svg', {type:'image/svg+xml'})); const input = document.querySelector('input[type=file]'); input.files = transfer.files; input.dispatchEvent(new Event('change', {bubbles:true})); })()")
  await wait("document.body.innerText.includes('Selecciona una imagen PNG')")
  await evaluate("(async () => { const canvas = document.createElement('canvas'); canvas.width = 4; canvas.height = 4; canvas.getContext('2d').fillRect(0,0,4,4); const blob = await new Promise(resolve => canvas.toBlob(resolve)); const transfer = new DataTransfer(); transfer.items.add(new File([blob], 'portrait.png', {type:'image/png'})); const input = document.querySelector('input[type=file]'); input.files = transfer.files; input.dispatchEvent(new Event('change', {bubbles:true})); })()")
  await wait("Boolean(document.querySelector('.portrait-editor img[src^=\"data:image/png\"]'))")
  await clickText('Guardar personaje')
  await wait("location.pathname === '/biblioteca' && document.querySelectorAll('.character-card').length === 7")
  await click('.character-card[aria-label*="Ayla"]')
  await wait("Boolean(document.querySelector('dialog[open]'))")
  assert.ok((await evaluate('document.querySelector("dialog").innerText')).includes('Una historia que debe persistir.'))
  assert.ok(await evaluate("document.querySelector('dialog img').src.startsWith('data:image/png')"))
  await clickText('Guardar en favoritos')
  await wait("document.querySelector('dialog [aria-pressed=true]') !== null")
  await send('Input.dispatchKeyEvent', { type: 'rawKeyDown', key: 'Escape', code: 'Escape', windowsVirtualKeyCode: 27, nativeVirtualKeyCode: 27 })
  await send('Input.dispatchKeyEvent', { type: 'keyUp', key: 'Escape', code: 'Escape', windowsVirtualKeyCode: 27, nativeVirtualKeyCode: 27 })
  await wait("!document.querySelector('dialog')")
  assert.ok(await evaluate("document.activeElement.classList.contains('character-card')"))
  await click('.preference-option input')
  await wait("document.querySelectorAll('.character-card').length === 1")
  await click('.preference-option input')
  await click('.character-card[aria-label*="Ayla"]')
  await clickText('Editar personaje')
  await wait("location.pathname.includes('/editar')")
  await fill('name', 'Ayla Editada')
  await clickText('Guardar cambios')
  await wait("location.pathname === '/biblioteca' && document.body.innerText.includes('Ayla Editada')")
  await reload()
  assert.ok((await evaluate('document.body.innerText')).includes('Ayla Editada'))
  await click('.character-card[aria-label*="Ayla"]')
  await clickText('Duplicar personaje')
  await wait("document.querySelectorAll('.character-card').length === 8")
  await click('.character-card[aria-label*="copia"]')
  await clickText('Eliminar personaje')
  await clickText('Conservar personaje')
  await clickText('Eliminar personaje')
  await click('dialog .primary-button')
```

```
  await wait("!document.querySelector('dialog') && document.querySelectorAll('.character-card').length === 7")
  await go('/crear-personaje')
  await fill('name', 'No debe guardarse')
  await evaluate("window.umbraSetItem = Storage.prototype.setItem; Storage.prototype.setItem = function() { throw new DOMException('Quota exceeded', 'QuotaExceededError') }")
  await clickText('Guardar personaje')
  await wait("document.body.innerText.includes('No se pudo guardar')")
  assert.equal(await evaluate('location.pathname'), '/crear-personaje')
  await evaluate('Storage.prototype.setItem = window.umbraSetItem')
  await clickText('Cancelar')
  await wait("Boolean(document.querySelector('dialog'))")
  await clickText('Descartar y salir')
  await wait("location.pathname === '/biblioteca' && document.querySelectorAll('.character-card').length === 7")
  await go('/inspiracion')
  const firstPrompt = await evaluate("document.querySelector('.inspiration-panel [role=status]').textContent")
  await clickText('Otra inspiración')
  assert.notEqual(await evaluate("document.querySelector('.inspiration-panel [role=status]').textContent"), firstPrompt)
  await clickText('Crear a partir de esta idea')
  await wait("Boolean(document.querySelector('textarea'))")
  assert.ok((await evaluate("document.querySelector('textarea').value")).length > 20)
  await go('/registro')
  await fill('name', 'Cuenta de prueba')
  await fill('email', 'prueba@umbra.es')
  await fill('password', 'Historia1!')
  await fill('confirmPassword', 'Historia1!')
  await click('input[type="checkbox"]')
  await clickText('Crear cuenta local')
  await wait("Boolean(document.querySelector('input[readonly]'))")
  const recoveryCode = await evaluate("document.querySelector('input[readonly]').value")
  assert.equal(recoveryCode.length, 32)
  await clickText('Ya he guardado el código. Iniciar sesión')
  await fill('email', 'prueba@umbra.es')
  await fill('password', 'Historia1!')
  await click('input[name="remember"]')
  await click('.login-submit')
  await wait("location.pathname === '/perfil' && document.body.innerText.includes('Cuenta de prueba')")
  assert.ok((await evaluate('document.body.innerText')).includes('6 personajes'))
  await reload()
  assert.ok((await evaluate('document.body.innerText')).includes('Cuenta de prueba'))
  await clickText('Cerrar sesión')
  await wait("document.body.innerText.includes('Exploras como invitado')")
  assert.ok((await evaluate('document.body.innerText')).includes('7 personajes'))
  await evaluate("window.umbraBackup = JSON.stringify({app:'UMBRA',version:1,characters:JSON.parse(localStorage.getItem('umbra.characters.v1.guest'))}); window.umbraCreateObjectURL = URL.createObjectURL; URL.createObjectURL = blob => { window.umbraDownload = blob.text(); return window.umbraCreateObjectURL(blob) }")
  await clickText('Exportar biblioteca (JSON)')
  const downloaded = await evaluate('window.umbraDownload')
  assert.equal(JSON.parse(downloaded).characters.length, 7)
  await importFile('archivo inválido')
  await wait("document.body.innerText.includes('JSON válido')")
  await importFile(JSON.stringify({ app: 'UMBRA', version: 1, characters: [] }))
  await wait("Boolean(document.querySelector('dialog'))")
  await clickText('Cancelar')
  assert.ok((await evaluate('document.body.innerText')).includes('7 personajes'))
```

```
  await importFile(JSON.stringify({ app: 'UMBRA', version: 1, characters: [] }))
  await wait("Boolean(document.querySelector('dialog'))")
  await clickText('Sustituir y restaurar')
  await wait("document.body.innerText.includes('0 personajes')")
  await go('/biblioteca')
  await reload()
  assert.equal(await evaluate("document.querySelectorAll('.character-card').length"), 0)
  await go('/perfil')
  await importFile(downloaded)
  await wait("Boolean(document.querySelector('dialog'))")
  await clickText('Sustituir y restaurar')
  await wait("document.body.innerText.includes('7 personajes')")
  await go('/recuperar')
  await fill('email', 'prueba@umbra.es')
  await fill('recoveryCode', recoveryCode)
  await fill('password', 'NuevaClave2!')
  await fill('confirmPassword', 'NuevaClave2!')
  await clickText('Cambiar contraseña')
  await wait("document.body.innerText.includes('Contraseña actualizada')")
  await clickText('Iniciar sesión')
  await fill('email', 'prueba@umbra.es')
  await fill('password', 'NuevaClave2!')
  await click('.login-submit')
  await wait("location.pathname === '/perfil' && document.body.innerText.includes('Cuenta de prueba')")
  await click('button[aria-label="Abrir menú"]')
  await click('.header-popover a[href="/accesibilidad"]')
  await wait("location.pathname === '/accesibilidad'")
  assert.equal(await evaluate("document.querySelectorAll('.settings-menu-item').length"), 5)
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.theme"), 'dark')
  await click('.settings-menu a[href="/accesibilidad/texto"]')
  await click('input[name="textSize"][value="large"]')
  const enlarged = await evaluate("parseFloat(getComputedStyle(document.querySelector('.settings-preview h2')).fontSize)")
  await fill('textSizeSlider', '3')
  await wait("document.querySelector('.app-frame').dataset.textSize === 'extra-large'")
  assert.ok(await evaluate("parseFloat(getComputedStyle(document.querySelector('.settings-preview h2')).fontSize)") > enlarged)
  await click('input[name="textSize"][value="large"]')
  const ax = await send('Accessibility.getFullAXTree')
  assert.ok(ax.nodes.some(node => node.role?.value === 'radio' && node.name?.value.includes('Grande') && node.properties?.some(property => property.name === 'checked' && property.value.value === 'true')))
  await go('/accesibilidad/contraste')
  await click('input[name="contrast"][value="high"]')
  assert.equal(await evaluate("getComputedStyle(document.querySelector('.app-frame')).backgroundColor"), 'rgb(0, 0, 0)')
  await click('input[name="contrast"][value="inverted"]')
  assert.equal(await evaluate("getComputedStyle(document.querySelector('.app-frame')).backgroundColor"), 'rgb(255, 255, 255)')
  await click('input[name="contrast"][value="high"]')
  await go('/accesibilidad/apariencia')
  await click('input[name="theme"][value="sepia"]')
  await click('input[name="elementSize"][value="extra-large"]')
  assert.ok(await evaluate("document.querySelector('.choice-card').getBoundingClientRect().height >= 60"))
  await click('input[name="accent"][value="blue"]')
  await click('input[name="reduceMotion"]')
  assert.equal(await evaluate("getComputedStyle(document.querySelector('.app-frame')).backgroundColor"), 'rgb(0, 0, 0)')
  await go('/accesibilidad/contraste')
```

```
  await click('input[name="contrast"][value="normal"]')
  assert.equal(await evaluate("getComputedStyle(document.querySelector('.app-frame')).backgroundColor"), 'rgb(239, 226, 200)')
  assert.equal(await evaluate("getComputedStyle(document.querySelector('.app-frame')).getPropertyValue('--accent').trim()"), '#12679a')
  await go('/accesibilidad/lectura')
  await click('input[name="readAloud"]')
  await click('input[name="announceNavigation"]')
  await select('select[name="readingRate"]', '0.75')
  assert.ok(!await evaluate("Boolean(document.querySelector('.app-frame > p.visually-hidden[role=status]'))"))
  await evaluate("window.umbraSpoken = []; window.umbraCanceled = 0; speechSynthesis.speak = speech => window.umbraSpoken.push({text:speech.text,rate:speech.rate}); speechSynthesis.cancel = () => window.umbraCanceled++")
  assert.equal(await evaluate('window.umbraSpoken.length'), 0)
  await clickText('Leer en voz alta')
  await wait("document.body.innerText.includes('Detener lectura')")
  assert.equal(await evaluate('window.umbraSpoken[0].rate'), 0.75)
  await clickText('Detener lectura')
  await go('/biblioteca')
  await evaluate("window.umbraSpoken = []; window.umbraCanceled = 0; speechSynthesis.speak = speech => window.umbraSpoken.push({text:speech.text,rate:speech.rate}); speechSynthesis.cancel = () => window.umbraCanceled++")
  await click('.character-card')
  await clickText('Leer en voz alta')
  await wait("document.body.innerText.includes('Detener lectura')")
  assert.ok((await evaluate('window.umbraSpoken[0].text')).includes('Elara Valen'))
  const cancellations = await evaluate('window.umbraCanceled')
  await click('dialog .dialog-close')
  await wait("!document.querySelector('dialog')")
  assert.ok(await evaluate('window.umbraCanceled') > cancellations)
  assert.equal(await evaluate("getComputedStyle(document.querySelector('.character-card')).transitionDuration"), '0s')
  await go('/accesibilidad/teclado')
  await click('input[name="enhancedFocus"]')
  await evaluate("document.querySelector('input[name=enhancedFocus]').focus()")
  await send('Input.dispatchKeyEvent', { type: 'rawKeyDown', key: 'Tab', code: 'Tab', windowsVirtualKeyCode: 9, nativeVirtualKeyCode: 9 })
  await send('Input.dispatchKeyEvent', { type: 'keyUp', key: 'Tab', code: 'Tab', windowsVirtualKeyCode: 9, nativeVirtualKeyCode: 9 })
  assert.equal(await evaluate('getComputedStyle(document.activeElement).outlineWidth'), '4px')
  await reload()
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.textSize"), 'large')
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.theme"), 'sepia')
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.accent"), 'blue')
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.elementSize"), 'extra-large')
  assert.ok(await evaluate("document.querySelector('.app-frame').classList.contains('enhanced-focus')"))
  await go('/perfil')
  await clickText('Cerrar sesión')
  await wait("document.body.innerText.includes('Exploras como invitado')")
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.theme"), 'dark')
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.textSize"), 'normal')
  await go('/accesibilidad/apariencia')
  await click('input[name="theme"][value="light"]')
  assert.equal(await evaluate("getComputedStyle(document.querySelector('.app-frame')).backgroundColor"), 'rgb(245, 245, 240)')
  await go('/registro')
  await fill('name', 'Segundo perfil')
  await fill('email', 'segundo@umbra.es')
```

```
  await fill('password', 'Historia2!')
  await fill('confirmPassword', 'Historia2!')
  await click('input[type=checkbox]')
  await clickText('Crear cuenta local')
  await wait("Boolean(document.querySelector('input[readonly]'))")
  await clickText('Ya he guardado el código. Iniciar sesión')
  await fill('email', 'segundo@umbra.es')
  await fill('password', 'Historia2!')
  await click('.login-submit')
  await wait("location.pathname === '/perfil' && document.body.innerText.includes('Segundo perfil')")
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.theme"), 'dark')
  await go('/accesibilidad/apariencia')
  await click('input[name="theme"][value="light"]')
  await click('input[name="accent"][value="green"]')
  await go('/accesibilidad')
  await clickText('Restablecer ajustes de este perfil')
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.theme"), 'dark')
  await go('/perfil')
  await clickText('Cerrar sesión')
  await wait("document.body.innerText.includes('Exploras como invitado')")
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.theme"), 'light')
  await go('/login')
  await fill('email', 'prueba@umbra.es')
  await fill('password', 'NuevaClave2!')
  await click('.login-submit')
  await wait("location.pathname === '/perfil' && document.body.innerText.includes('Cuenta de prueba')")
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.theme"), 'sepia')
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.textSize"), 'large')
  await go('/accesibilidad/contraste')
  await click('input[name="contrast"][value="normal"]')
  await send('Input.dispatchKeyEvent', { type: 'rawKeyDown', key: 'ArrowDown', code: 'ArrowDown',
windowsVirtualKeyCode: 40, nativeVirtualKeyCode: 40 })
  await send('Input.dispatchKeyEvent', { type: 'keyUp', key: 'ArrowDown', code: 'ArrowDown',
windowsVirtualKeyCode: 40, nativeVirtualKeyCode: 40 })
  await wait("document.querySelector('.app-frame').dataset.contrast === 'high'")
  await click('input[name="contrast"][value="normal"]')
  await go('/accesibilidad/texto')
  await evaluate("window.umbraSetItem = Storage.prototype.setItem; Storage.prototype.setItem = () => {
throw new DOMException('Storage blocked', 'SecurityError') }")
  await click('input[name="textSize"][value="extra-large"]')
  await wait("document.body.innerText.includes('no permite guardar tus preferencias')")
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.textSize"), 'extra-large')
  await evaluate('Storage.prototype.setItem = window.umbraSetItem')
  await reload()
  assert.equal(await evaluate("document.querySelector('.app-frame').dataset.textSize"), 'large')
  await go('/biblioteca')
  assert.ok(await evaluate('document.documentElement.scrollWidth <= innerWidth'))
  await send('Emulation.setDeviceMetricsOverride', { width: 320, height: 740, deviceScaleFactor: 1,
mobile: true })
  assert.ok(await evaluate('document.documentElement.scrollWidth <= innerWidth'))
  await go('/accesibilidad/texto')
  await click('input[name="textSize"][value="extra-large"]')
  for (const path of ['/inicio', '/biblioteca', '/crear-personaje', '/perfil', '/inspiracion', '/ayuda',
'/accesibilidad', '/accesibilidad/texto', '/accesibilidad/contraste', '/accesibilidad/lectura',
'/accesibilidad/teclado', '/accesibilidad/apariencia']) {
    await go(path)
    assert.ok(await evaluate('document.documentElement.scrollWidth <= innerWidth'), `Overflow at
${path}`)
  }
```

```
  await go('/ruta-que-no-existe')
  assert.ok((await evaluate('document.body.innerText')).includes('404'))
  await evaluate("window.dispatchEvent(new Event('offline'))")
  await wait("document.querySelector('.connection-status').textContent.includes('Sin conexión')")
  await evaluate("window.dispatchEvent(new Event('online'))")
  await wait("document.querySelector('.connection-status').textContent.includes('restablecida')")
  await send('Emulation.setDeviceMetricsOverride', { width: 390, height: 844, deviceScaleFactor: 1, mobile: true })
  await go('/accesibilidad')
  if (process.env.UMBRA_SCREENSHOT) {
    const shot = await send('Page.captureScreenshot', { format: 'png', captureBeyondViewport: true })
    await writeFile(process.env.UMBRA_SCREENSHOT, Buffer.from(shot.data, 'base64'))
  }
  assert.deepEqual(exceptions, [])
  console.log('Browser OK: CRUD, cuentas, copias, cinco menús de accesibilidad, texto y controles, tres contrastes, temas y acentos, voz, foco, tres perfiles independientes, persistencia, móvil, conexión y 404.')
} catch (error) {
  console.error(error)
  console.error('URL:', await evaluate('location.href').catch(() => 'unknown'))
  console.error('Page:', await evaluate('document.body.innerText').catch(() => 'unavailable'))
  process.exitCode = 1
} finally {
  if (browserContextId) await send('Target.disposeBrowserContext', { browserContextId }, null)
  socket.close()
}
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 44 DE 50

favicon.svg

`public/favicon.svg`

Icono vectorial de la pestaña del navegador.

- Lo referencia index.html. Se incluye su código SVG completo; los PNG y las fuentes se guardan como archivos binarios en el ZIP.

Código completo

4 líneas · SHA-256 del original: `9f9b30bbf7aa3dca61c8b928754abfa4ad370249b9b4d857bdcf1a28825ec5dc`

```
1 <svg xmlns="http://www.w3.org/2000/svg" viewBox="0 0 32 32">
2   <rect width="32" height="32" rx="6" fill="#090f0f"/>
3   <path d="m16 4 10 6-3 14-7 5-7-5-3-14Zm0 0v25m-5-18 1 9 4 4 4-4 1-9" fill="none" stroke="#a99770"
  stroke-linejoin="round"/>
4 </svg>
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 45 DE 50

ASSETS.md

`public/images/ASSETS.md`

Descripción y procedencia de los recursos gráficos incluidos.

- Documenta los prompts utilizados para los tres PNG. No genera imágenes al ejecutar la app.

Código completo

15 líneas · SHA-256 del original: `b47a7657182ce862ccd9e88a9b70ae4ae33ea49660a9d52875a18fdea6997172`

```
# Imágenes de UMBRA

Generadas con la herramienta integrada `image_gen` para esta pantalla. Las imágenes son interpretaciones del estilo de la referencia, no los archivos originales de la captura.

## umbra-castle.png

Prompt: Use case: stylized-concept. Create a landscape 1536x1024 dark fantasy illustration for a website hero background. A towering, intricate gothic castle with many narrow spires on a steep rocky hill, storm clouds, faint muted copper light on the horizon, atmospheric haze. Castle on the right half, cloudy sky and negative space left. Realistic painterly game concept art, restrained charcoal, desaturated green-black, aged gold, no bright colors. Similar mood to a dark medieval role-playing character library. No words, no text, no borders, no interface. This image will be saved as umbra-castle.png in a local React project.

## elara-valen.png

Prompt: Use case: stylized-concept. Square dark fantasy character portrait for a small character card. Elara Valen, pale adult sorceress with long wavy black hair, dark fitted medieval gown, silver pendant, calm serious expression, centered waist-up front view. Ancient gothic library with carved stone arches and candles behind her. Realistic painterly role-playing game illustration, charcoal black, muted aged gold, desaturated olive green. Soft dramatic light on face; dark atmosphere but facial features readable. No text, no borders, no UI. Save intended asset elara-valen.png.

## kael-grim.png

Prompt: Use case: stylized-concept. Square dark fantasy character portrait for a small character card. Kael Grim, rugged adult male rogue with short dark beard, dark hood, worn brown leather armor and belt, centered waist-up front view. Ancient gothic stone hall with tall arches in background. Realistic painterly role-playing game illustration, charcoal black, muted aged gold, desaturated olive green. Soft dramatic light on face; dark atmosphere but facial features readable. No text, no borders, no UI. Save intended asset kael-grim.png.
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 46 DE 50

.gitignore

`.gitignore`

Patrones que Git ignora para evitar añadir dependencias, compilados y archivos temporales.

- Incluye node_modules, dist, archivos de log y datos locales de herramientas. No elimina esos archivos del equipo.

Código completo

24 líneas · SHA-256 del original: `d50ab07e11fa4bf3b1c7c3312e1d7baeab56cca756832ccf40476823ad2a38b7`

```
# Logs
logs
*.log
npm-debug.log*
yarn-debug.log*
yarn-error.log*
pnpm-debug.log*
lerna-debug.log*

node_modules
dist
dist-ssr
*.local

# Editor directories and files
.vscode/*
!.vscode/extensions.json
.idea
.DS_Store
*.suo
*.ntvs*
*.njsproj
*.sln
*.sw?
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 47 DE 50

README.md

`README.md`

Guía breve original del repositorio con comandos, funciones y límites del almacenamiento local.

- Complementa este manual. El listado de código reproduce el archivo tal como estaba al generar el documento.

Código completo

60 líneas · SHA-256 del original: `7dda1427ac6139c9dc25225faff513b1b45813da65f1635402895bf1098ed103`

````
# UMBRA

Aplicación de personajes de fantasía con React, Vite y React Router. Interfaz oscura, navegación con teclado y almacenamiento local en el navegador.

## Ejecutar

```bash
npm install
npm run dev
```

En PowerShell con scripts deshabilitados, usa `npm.cmd` en lugar de `npm`.

```bash
npm test
npm run lint
npm run build
npm run preview
```

La comprobación de recorridos en navegador está en `npm run test:browser`. Requiere la app local activa y Chrome o Edge iniciado con depuración remota en el puerto 9222; utiliza un contexto temporal aislado y lo elimina al terminar. `UMBRA_URL` permite elegir otra dirección local y `UMBRA_SCREENSHOT` guarda una captura opcional. Comprueba altas, edición, borrado, favoritos, búsqueda, retratos, cuentas, recuperación, copias JSON, teclado, almacenamiento bloqueado y pantallas de 320/390 px.

## Funciones

- Inicio con personajes recientes y acceso a la creación de fichas.
- Biblioteca con búsqueda por nombre, clase, raza u origen, filtro de clase y favoritos, y orden por fecha, nombre o nivel.
- Crear, consultar, editar, duplicar y eliminar personajes. Las fichas incluyen historia, nivel, reino, rasgos, cuatro atributos y retrato propio PNG/JPG/WebP de hasta 750 KB.
- Inspiración con ideas narrativas y personajes aleatorios que se pueden llevar al editor.
- Registro de cuentas locales, inicio y cierre de sesión, opción Recordarme y recuperación mediante el código entregado al registrarse.
````

```
- Perfil con recuentos y exportación/importación de bibliotecas JSON. La importación valida el contenido y pide confirmación antes de sustituir datos.
- Accesibilidad por perfil mediante cinco menús: texto (cuatro tamaños y deslizador), contraste (normal, alto e invertido), lectores de pantalla (anuncios y lectura voluntaria en voz alta), teclado (foco reforzado y guía de atajos) y apariencia (oscuro, claro, sepia; tamaño de controles; seis acentos; movimiento reducido).
- Ayuda, información de privacidad, estado de conexión y página 404.

## Almacenamiento

La app funciona sin servidor de datos. Cada cuenta local y el invitado tienen una biblioteca independiente, inicialmente con seis personajes. Los cambios persisten en `localStorage`; una biblioteca vacía permanece vacía. Los datos del invitado se conservan al cambiar de cuenta. Usa exportar/importar para trasladar personajes entre bibliotecas.

Los ajustes de accesibilidad se guardan por identificador de perfil en `umbra.accessibility.v2.<id>`; Invitado tiene su propia configuración. Al cambiar de cuenta se cargan sus ajustes. Las antiguas preferencias globales se recuperan únicamente para Invitado. Restablecer afecta solo al perfil activo. Si el guardado está bloqueado, los ajustes se aplican durante la sesión y se muestra un aviso. El contraste tiene prioridad sobre los colores del tema, conservando la selección de tema y acento. La lectura en voz alta usa Web Speech cuando está disponible y se detiene al cerrar la ficha o salir de su pantalla; no sustituye al lector de pantalla del dispositivo.

Las cuentas solo existen en el navegador donde se crean. Las contraseñas se guardan como verificadores PBKDF2-SHA-256 con sal aleatoria por cuenta y 100 000 iteraciones; el código de recuperación se verifica mediante SHA-256. No se guardan contraseñas ni códigos de recuperación en texto plano. Es una organización local de bibliotecas, sin autenticación de servidor ni protección frente a personas con acceso al navegador. Web Crypto necesita un contexto seguro: localhost o HTTPS.

Recordarme conserva la sesión en `localStorage`; sin marcarlo se utiliza `sessionStorage`. El cierre de sesión elimina ambas referencias. No hay envío de correos ni sincronización entre dispositivos. Borrar los datos del sitio elimina las cuentas y las bibliotecas. Exporta copias periódicas desde Perfil; estas incluyen personajes y retratos, sin cuentas ni credenciales.

Los fallos de almacenamiento se muestran al usuario y las operaciones que no se pueden guardar no se presentan como completadas. Sin conexión puedes continuar con la app abierta; recargarla requiere acceso al sitio o al servidor local. No incluye un service worker.

## Estructura

- `src/pages/`: pantallas y formularios de cada ruta.
- `src/components/`: navegación, fichas, formularios, avisos y diálogos.
- `src/data/`: personajes iniciales, opciones y navegación.
- `src/hooks/`: bibliotecas locales y preferencias de accesibilidad.
- `src/utils/`: validación, cuentas locales y copias de seguridad.
- `tests/`: pruebas de validación, persistencia y cuentas.
- `public/`: imágenes y fuentes locales con sus licencias.

Las imágenes se documentan en [public/images/ASSETS.md](public/images/ASSETS.md). Las licencias de las fuentes están en `public/fonts/`.

## Publicación

`npm run build` genera `dist/`. El alojamiento debe servir `index.html` para las rutas de la aplicación, ya que se utiliza `BrowserRouter`. No incluyas las bibliotecas ni las cuentas locales en el despliegue; son datos propios de cada navegador.
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 48 DE 50

LICENSE

LICENSE

Licencia del proyecto incluida en el repositorio.

- Se reproduce íntegra en la documentación y se conserva también en el ZIP. Las fuentes tienen sus propias licencias en public/fonts.

Código completo

21 líneas · SHA-256 del original: `58e32531aacd2fa91b72e5c1cb037fe7b520989bc7ffdc66c40aa6537b184202`

```
MIT License

Copyright (c) 2026 Ixf

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 49 DE 50

Inter-OFL.txt

`public/fonts/Inter-OFL.txt`

Licencia de la fuente incluida en public/fonts.

- Se conserva íntegra junto con el recurso tipográfico correspondiente.

Código completo

93 líneas · SHA-256 del original: `f14f2b95a38f4f20cad4d27f7710593f37534c046641be0348da7c28365f4e39`

```
Copyright 2020 The Inter Project Authors (https://github.com/rsms/inter)

This Font Software is licensed under the SIL Open Font License, Version 1.1.
This license is copied below, and is also available with a FAQ at:
https://scripts.sil.org/OFL


-----------------------------------------------------------
SIL OPEN FONT LICENSE Version 1.1 - 26 February 2007
-----------------------------------------------------------

PREAMBLE
The goals of the Open Font License (OFL) are to stimulate worldwide
development of collaborative font projects, to support the font creation
efforts of academic and linguistic communities, and to provide a free and
open framework in which fonts may be shared and improved in partnership
with others.

The OFL allows the licensed fonts to be used, studied, modified and
redistributed freely as long as they are not sold by themselves. The
fonts, including any derivative works, can be bundled, embedded,
redistributed and/or sold with any software provided that any reserved
names are not used by derivative works. The fonts and derivatives,
however, cannot be released under any other type of license. The
requirement for fonts to remain under this license does not apply
to any document created using the fonts or their derivatives.

DEFINITIONS
"Font Software" refers to the set of files released by the Copyright
Holder(s) under this license and clearly marked as such. This may
include source files, build scripts and documentation.

"Reserved Font Name" refers to any names specified as such after the
copyright statement(s).

"Original Version" refers to the collection of Font Software components as
distributed by the Copyright Holder(s).

"Modified Version" refers to any derivative made by adding to, deleting,
or substituting -- in part or in whole -- any of the components of the
```

```
Original Version, by changing formats or by porting the Font Software to a
new environment.

"Author" refers to any designer, engineer, programmer, technical
writer or other person who contributed to the Font Software.

PERMISSION & CONDITIONS
Permission is hereby granted, free of charge, to any person obtaining
a copy of the Font Software, to use, study, copy, merge, embed, modify,
redistribute, and sell modified and unmodified copies of the Font
Software, subject to the following conditions:

1) Neither the Font Software nor any of its individual components,
in Original or Modified Versions, may be sold by itself.

2) Original or Modified Versions of the Font Software may be bundled,
redistributed and/or sold with any software, provided that each copy
contains the above copyright notice and this license. These can be
included either as stand-alone text files, human-readable headers or
in the appropriate machine-readable metadata fields within text or
binary files as long as those fields can be easily viewed by the user.

3) No Modified Version of the Font Software may use the Reserved Font
Name(s) unless explicit written permission is granted by the corresponding
Copyright Holder. This restriction only applies to the primary font name as
presented to the users.

4) The name(s) of the Copyright Holder(s) or the Author(s) of the Font
Software shall not be used to promote, endorse or advertise any
Modified Version, except to acknowledge the contribution(s) of the
Copyright Holder(s) and the Author(s) or with their explicit written
permission.

5) The Font Software, modified or unmodified, in part or in whole,
must be distributed entirely under this license, and must not be
distributed under any other license. The requirement for fonts to
remain under this license does not apply to any document created
using the Font Software.

TERMINATION
This license becomes null and void if any of the above conditions are
not met.

DISCLAIMER
THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT
OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL THE
COPYRIGHT HOLDER BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY,
INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL
DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING
FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM
OTHER DEALINGS IN THE FONT SOFTWARE.
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVO 50 DE 50

Cormorant-Garamond-OFL.txt

`public/fonts/Cormorant-Garamond-OFL.txt`

Licencia de la fuente incluida en public/fonts.

- Se conserva íntegra junto con el recurso tipográfico correspondiente.

Código completo

93 líneas · SHA-256 del original: `b2306305139223cbc000c2b9feffaa741687056066c227ba34de31c98eda0814`

```
Copyright 2015 the Cormorant Project Authors (github.com/CatharsisFonts/Cormorant)

This Font Software is licensed under the SIL Open Font License, Version 1.1.
This license is copied below, and is also available with a FAQ at:
https://scripts.sil.org/OFL


-----------------------------------------------------------
SIL OPEN FONT LICENSE Version 1.1 - 26 February 2007
-----------------------------------------------------------

PREAMBLE
The goals of the Open Font License (OFL) are to stimulate worldwide
development of collaborative font projects, to support the font creation
efforts of academic and linguistic communities, and to provide a free and
open framework in which fonts may be shared and improved in partnership
with others.

The OFL allows the licensed fonts to be used, studied, modified and
redistributed freely as long as they are not sold by themselves. The
fonts, including any derivative works, can be bundled, embedded,
redistributed and/or sold with any software provided that any reserved
names are not used by derivative works. The fonts and derivatives,
however, cannot be released under any other type of license. The
requirement for fonts to remain under this license does not apply
to any document created using the fonts or their derivatives.

DEFINITIONS
"Font Software" refers to the set of files released by the Copyright
Holder(s) under this license and clearly marked as such. This may
include source files, build scripts and documentation.

"Reserved Font Name" refers to any names specified as such after the
copyright statement(s).

"Original Version" refers to the collection of Font Software components as
distributed by the Copyright Holder(s).

"Modified Version" refers to any derivative made by adding to, deleting,
or substituting -- in part or in whole -- any of the components of the
```

```
Original Version, by changing formats or by porting the Font Software to a
new environment.

"Author" refers to any designer, engineer, programmer, technical
writer or other person who contributed to the Font Software.

PERMISSION & CONDITIONS
Permission is hereby granted, free of charge, to any person obtaining
a copy of the Font Software, to use, study, copy, merge, embed, modify,
redistribute, and sell modified and unmodified copies of the Font
Software, subject to the following conditions:

1) Neither the Font Software nor any of its individual components,
in Original or Modified Versions, may be sold by itself.

2) Original or Modified Versions of the Font Software may be bundled,
redistributed and/or sold with any software, provided that each copy
contains the above copyright notice and this license. These can be
included either as stand-alone text files, human-readable headers or
in the appropriate machine-readable metadata fields within text or
binary files as long as those fields can be easily viewed by the user.

3) No Modified Version of the Font Software may use the Reserved Font
Name(s) unless explicit written permission is granted by the corresponding
Copyright Holder. This restriction only applies to the primary font name as
presented to the users.

4) The name(s) of the Copyright Holder(s) or the Author(s) of the Font
Software shall not be used to promote, endorse or advertise any
Modified Version, except to acknowledge the contribution(s) of the
Copyright Holder(s) and the Author(s) or with their explicit written
permission.

5) The Font Software, modified or unmodified, in part or in whole,
must be distributed entirely under this license, and must not be
distributed under any other license. The requirement for fonts to
remain under this license does not apply to any document created
using the Font Software.

TERMINATION
This license becomes null and void if any of the above conditions are
not met.

DISCLAIMER
THE FONT SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND,
EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO ANY WARRANTIES OF
MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT
OF COPYRIGHT, PATENT, TRADEMARK, OR OTHER RIGHT. IN NO EVENT SHALL THE
COPYRIGHT HOLDER BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY,
INCLUDING ANY GENERAL, SPECIAL, INDIRECT, INCIDENTAL, OR CONSEQUENTIAL
DAMAGES, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING
FROM, OUT OF THE USE OR INABILITY TO USE THE FONT SOFTWARE OR FROM
OTHER DEALINGS IN THE FONT SOFTWARE.
```

Listado completo del archivo. Volver al índice de archivos.

ARCHIVOS PARA EXTRAER

Recursos y copia editable

El ZIP conserva el PDF, esta versión HTML y una carpeta codigo/ lista para abrir en el editor. Los archivos se leen del proyecto, sin alterar sus contenidos.

- Los PNG y TTF se incluyen en su formato original, junto con sus licencias.
- package-lock.json conserva la resolución de dependencias.
- El manifiesto enumera rutas y hashes para comprobar qué versión del código se documentó.
- Se excluyen dependencias instaladas, compilados, historial de Git y datos de los navegadores.

Para estudiar una función, usa primero el índice de archivos, consulta su explicación y localiza su número de línea. Para modificar o ejecutar el código, utiliza la copia editable del ZIP o el proyecto original.